ADIUVO

ENGINEERING AND TRAINING, LTD.

Mission Critical Design for FPGA & SoC

ADIUVO
ENGINEERING AND TRAINING, LTD.

Agenda

Session One
- Challenges, Environment, Development Processes, Requirements and Verification
- System level, Risks, System and Hardware Aspects
- Lab – Introduction to Libero – Lab Vivado Introduction (optional)

Session Two
- FPGA Design, Clocks and Reset, Clock Domain Crossing, IO Considerations, State Machines, Block RAMS, XADC / Sysmon
- Lab - Xilinx and Libero FSM Example

Session Three
- Optimization / Place and Route Issues, SRAM Configuration, Triple Modular Redundancy, Isolation Flow, TMR Support Xilinx, TMR MicroBlaze, Pin Placement, Timing Closure.
- Lab – Libero BRAM, Lab – Isolation Flow

Session Four
- Verification, Power Estimation, Multiboot and Fall back, Wrap Up
- Lab TMR Micro Blaze

ADIUVO
ENGINEERING AND TRAINING, LTD.

Introduction

Where do we need these systems?

Mission Critical FPGA requires the entire system to be mission critical

System solution is only as good as the weakest part

Why might we need to design these things

- Applications where life depends on it
- Safety
- Security

Many factors which need to be addressed to create a mission critical system

The challenge

Reliable Systems face several challenges

Could be to provide function across wide range of environmental conditions

- E.g. Aircraft, Automobile, Satellite, Industrial, Transportation

Alternatively it could be relatively benign environment

- E.g. Medical, Process Control

The Environment

ADIUVO
ENGINEERING AND TRAINING, LTD.

Environment Temperature

Temperature extremes:

- Effect timing – faster at lower temperatures, slower at high temperatures
- Effect power consumption – increased power demands when low
- May cause start up issues (e.g., oscillators)
- Effect reliability at higher temperatures
  - » High dissipation devices (processors, FPGA, etc.) could exceed rated junction temperature
- Need to use conduction, radiation and convection effectively within the system

Thermal Environment

Typically many people think of the cold temperatures of space however, for the payload engineering team the thermal challenge faced most often is ensuring components and equipment's stay within their de-rated junction / operating temperature range.

This is especially the case for high performance FPGA's like the Virtex 5QV which can require high currents when performing high end DSP applications.

- How can we address this and ensure the FPGA stays within the de-rated junction temperature range ?
- Why do we have a de-rated junction temperature range ?

Most satellite bus providers place limitations on the thermal density of the module on the satellite panels.

Thermal Environment

Both the equipment and FPGA will be required to operate across wide temperature ranges. It is typical for your design to have to meet three operating temperature ranges and still operate:

- Operating Temperature, the normal operating temperature in orbit (-20C to 60C) Applicable to all units
- Acceptance Temperature, this is the temperature at which the design analysis should be conducted this is normally operating temperature +/- 5C
- Qualification Temperature, this is the temperature the unit has to operate at during qualification normally operating temperature +/- 10C

These temperatures will depend upon the spacecraft bus your payload is flying on.

Thermal Environment

- Performance in the thermal environment is tested during thermal vacuum testing (TVAC)
- Why TVAC? A higher temperature rise will be exhibited in components than standard thermal testing by between 10 and 20C.
- This can have a significant effect upon both the power required and the timing performance of your design

Vibration and Shock

ADIUVO
ENGINEERING AND TRAINING, LTD.

Environment Vibration & Shock

- Random Vibration – many causes
  - » Vehicles – random vibration caused by interaction with road surface
  - » Aircraft – Wind Buffeting, Turbine / Jet Engines
- Sine Vibration
  - » Any reciprocal or regular cycling input
- Acoustic
  - » Caused by reflected sound waves – Jet Take off 100 dB(A)
  - » Experienced by electronic boxes as random vibration
- Shock - Short Duration High Acceleration Event
  - » Mechanical Release or Latching
  - » Equipment being dropped
- Can result in Failure of equipment due to physical failure

Dynamic Environment

The main shock and vibration environments occur during

- Launch
- Payload separation from the launcher
- Extension of solar panels
- Extension of antenna reflectors

- Depending upon the role of your equipment (e.g.,) platform or payload it will be either unpowered or powered during launch
- It is mainly platform subsystems which are powered during launch and therefore must operate correctly through out launch
- Unpowered equipment must be capable of operating post exposure to the dynamic environment

Dynamic Environment

- An example the vibration profile can cause, we recently had issues with an equipment a subcontractor supplied.
- Upon further investigation it was found vibration was causing bond wires within a FPGA to cause glitches and result in a un recoverable state.
- Often it is late in a project when the qualification is undertaken hence the cost of failures can be significant not only financially but also effecting launch dates.

How can we de-risk this ?

Dynamic Environment

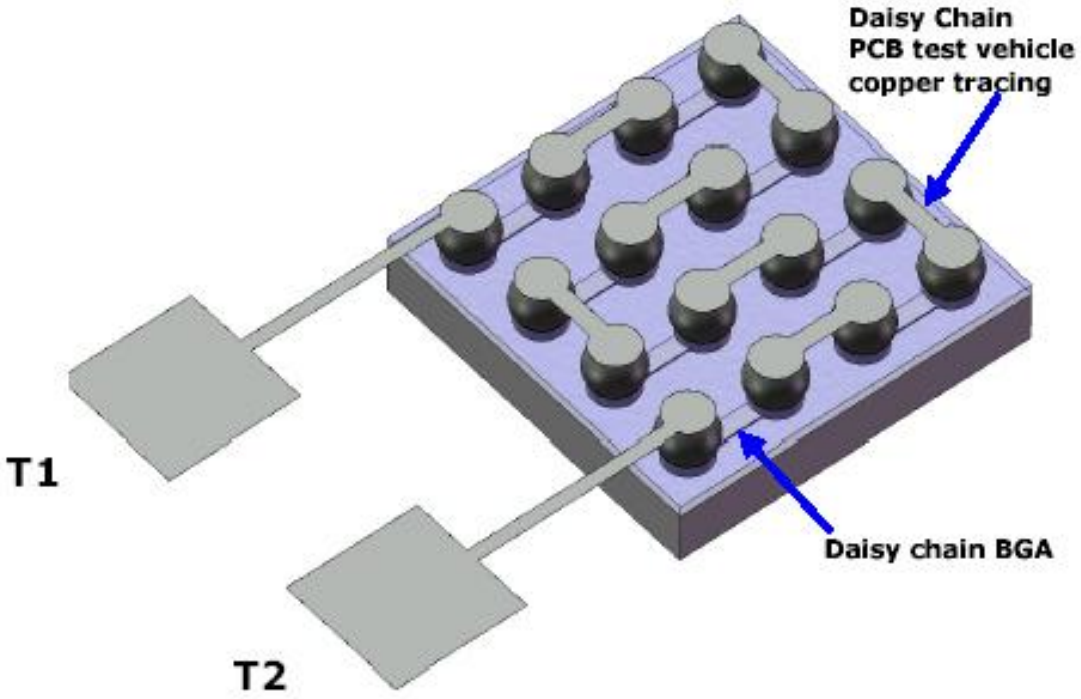
- Development of a structural model early in the programme which is representative of the final design.

- Daisy chain devices can be purchased from many component suppliers in the same packaging which allows you to monitor the package mounting to ensure it can survive or operate within the dynamic environment.

- While this can be tested early it is by no means cheap, but it is cheaper than finding out during the qualification programme.

Daisy Chain Devices

- If necessary we can chain multiple devices together
- Used to ensure the mounting does not fail under dynamic loads
- Bring out to external connectors. Test equipment can monitor the continuity.



EMC

ADIUVO
ENGINEERING AND TRAINING, LTD.

Environment – EMC / ESD

- The ability of an equipment, sub-system or system to share the electromagnetic spectrum and perform their desired functions without unacceptable degradation from or to the environment in which they exist
- Needs to be able to accept and function correctly when subject to radiated and conducted susceptibility
  - » Complete corruption of digital words
  - » Offsets in analogue signals
  - » Crosstalk in communication systems
  - » Equipment switch off/reset

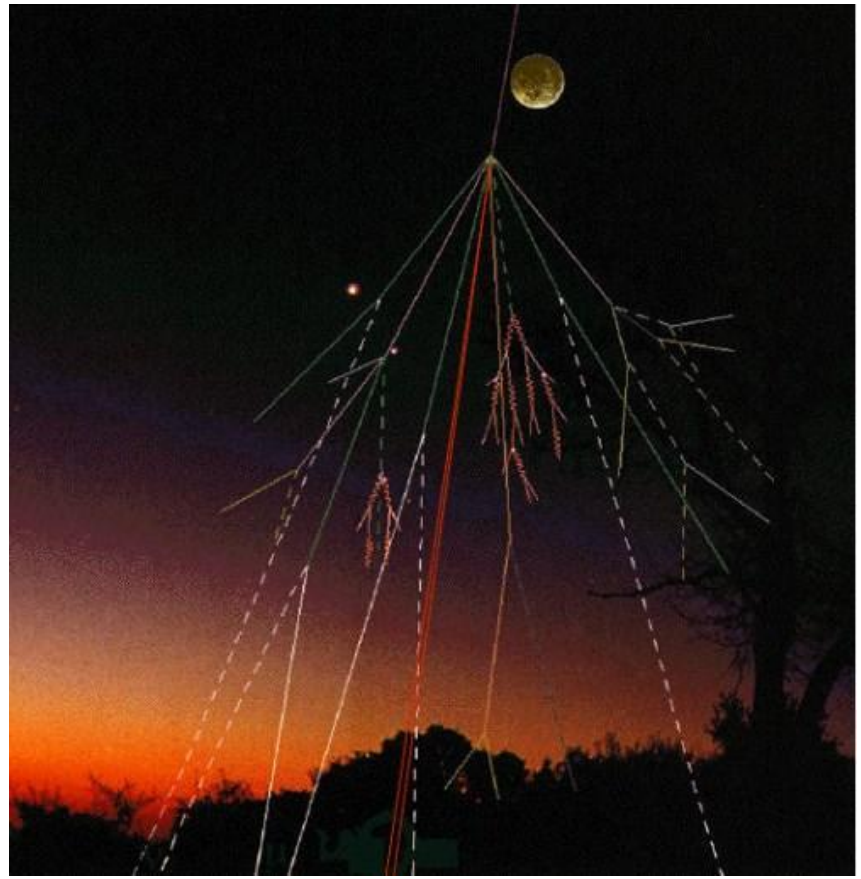
Radiation

ADIUVO
ENGINEERING AND TRAINING, LTD.

Environment – Atmospheric Neutrons

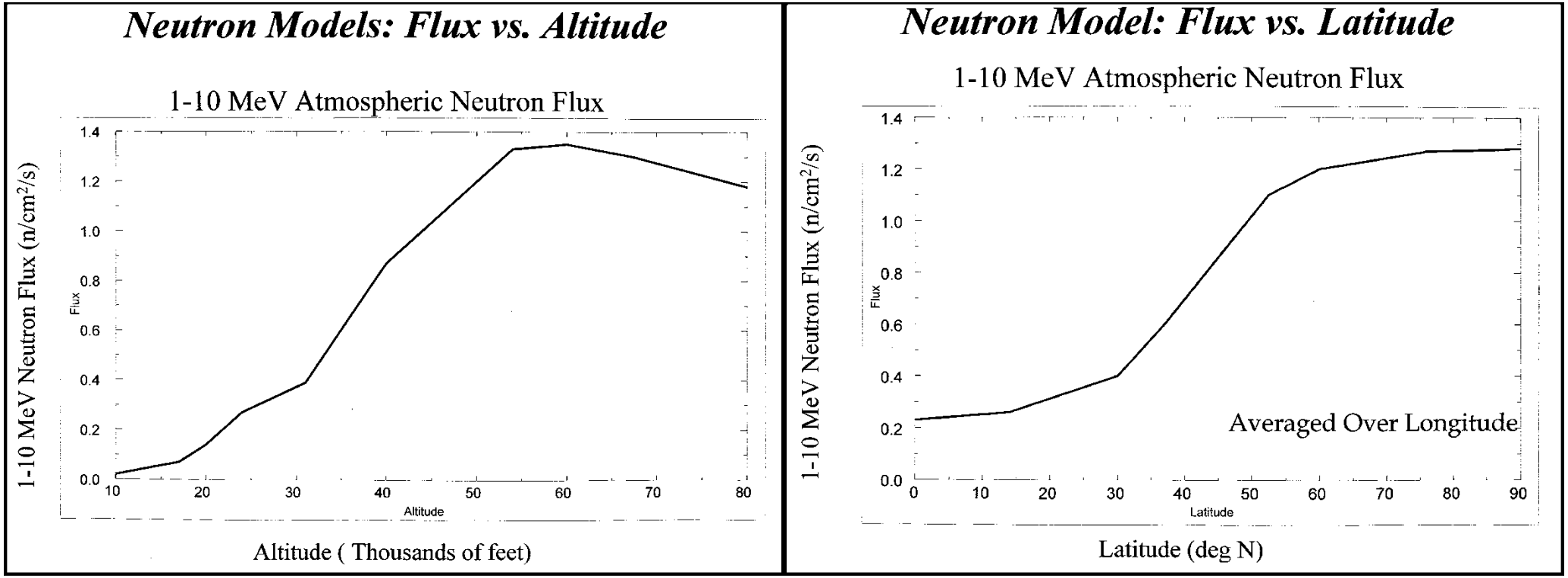
Atmospheric neutrons result from cosmic rays (actually high energy particles) when these high energy particles strike the earth's atmosphere they collide with other atoms present in the atmosphere creating neutron and proton showers.

These will be attenuated by the atmosphere but some will still strike the surface. This means that as the altitude you are operating at increases so does the probability of being subjected to a strike from an atmospheric neutron (it also varies with latitude too but we will look at that later on), the engineer and system designer must therefore take the operating altitude into account. For example you will have a better MTBF operating in New York City at sea level than you would in Denver Colorado (5280 feet above sea level).



Environment – Atmospheric Neutrons

Neutron Flux as it varies by altitude and latitude



Radiation Effects Environment

- Radiation falls into four types:
  - » Total Ionizing Dose (TID)
  - » Total Non Ionizing Dose (TNID)
  - » Single Event Effect (SEE)
  - » Deep Dielectric Discharge (DDD)
- The first three of these categories have the potential to effect electronic components while the fourth DDD is a more systematic effect.
- DDD allows a build up of charge in materials leading to an electrostatic discharge potentially leading to damage, for this reason all floating metal must be connected to ground

ADIUVO
ENGINEERING AND TRAINING, LTD.

Radiation Effects Space Environment

- Both TID and TNID are long term effects upon electronic components which result in the degradation of component parameters over time, for example Timing or Power parameters

- One problematic aspect of TID is Enhanced Low Dose Rate Sensitivity (ELDRS)

- ELDRS – Is when a device experiences worse performance degradation when subjected to a lower dose rate. This normally effects Bipolar or BiCMOS technologies

Radiation Effects

- Single Event Effects are different to total dose radiation being instantaneous and therefore occurring at any point within a mission

- SEE can be either destructive or non-destructive

- When designing engineers tend to focus upon the non-destructive effects. This is due to having worked with part and radiation engineers to ensure the parts selected are radiation hard in line with the radiation hardness assurance requirements.

ADIUVO
ENGINEERING AND TRAINING, LTD.

Radiation Effects

Description	Effect	Technology Affected	Destructive
Latchup – SEL	High Current	CMOS, BiCMOS	Yes
SnapBack –SESB	High Current	N Channel MOSFET, SOI Devices	Yes
Gate Rupture – SECR	Rupture of gate dielectric	Power MOSFET	Yes
Hard Error – SHE	Unalterable change of memory element	Memories, Latches, Registers	Yes
Burn Out – SEB	Destructive Burn Out	BJT, N Channel Power MOSFET	Yes
Upset – SEU	Register or Memory Corruption	Memories, Registers and Laches	No
Multi Bit Upset – MBU	As SEU but multiple bits affected	Memories, Registers and Laches	No
Functional Interrupt - SEFI	Loss of normal operation	Complex device e.g. FPGA	No
Transient – SET	Impulse Response	Analog & Mixed Circuitry	No
Disturb – SED	Momentary Corruption of information stored in a bit	Combinatorial Logic	No

Radiation Effects

While we are focusing upon the effects on electronic components it is worth noting that radiation also effects other materials within the design.

Radiation requirements by mission type:

Radiation	Galactic Heavy Ions	Solar Flare Heavy Ions	Solar Flare Protons	Trapped Protons	Trapped Electrons	Gamma Photons
Launcher	X	X	X	X		
	0.1 – 100 MeV.cm^2/mg		0.1 – 500 MeV			
Telecoms (GEO)	X	X	X	X	X	X
	0.1 – 100 MeV.cm^2/mg		0.1 – 500 MeV		10KeV – 10MeV	
Observation (LEO / MEO)	X	X	X	X	X	X
	0.1 – 100 MeV.cm^2/mg		0.1 – 500 MeV		10KeV – 10MeV	
Interplanetary	X	X	X			
	0.1 – 100 MeV.cm^2/mg		0.1 – 200 MeV			

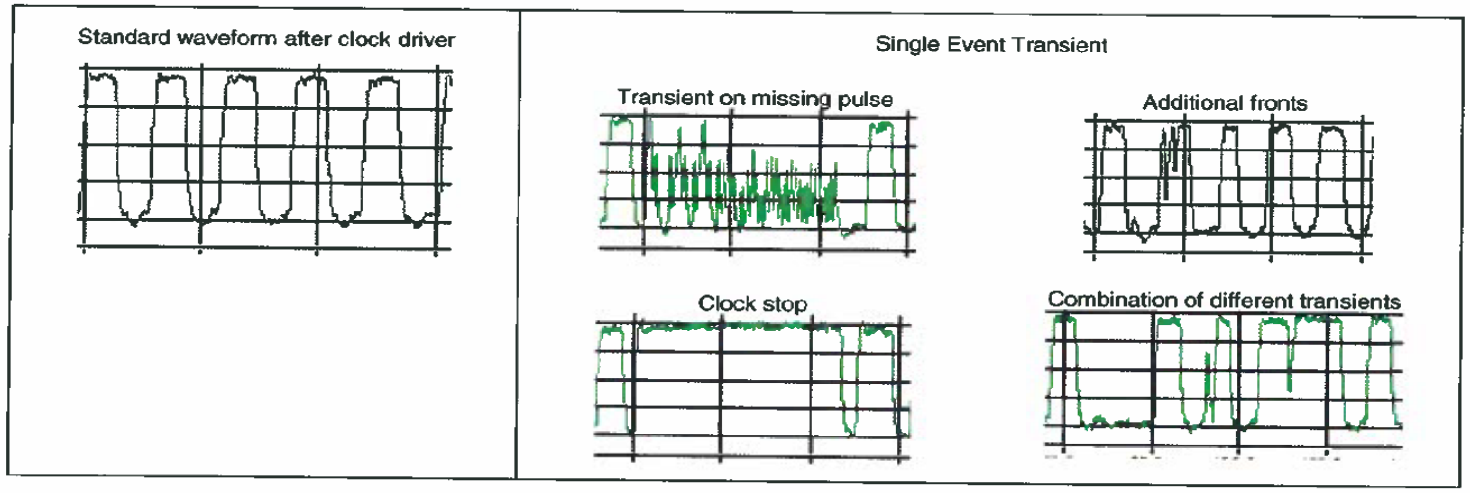
Radiation Effects

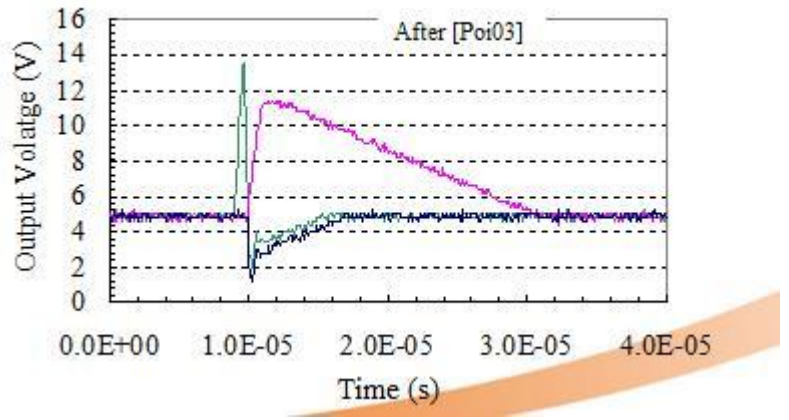
Where do these effects come from?

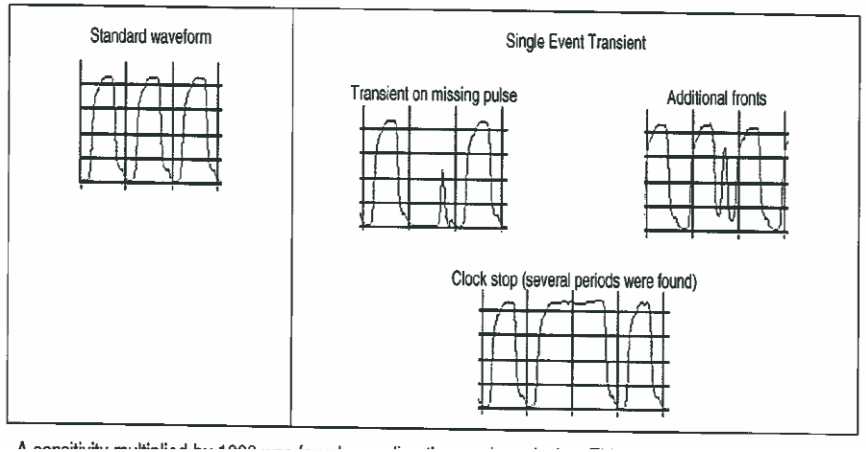
- Radiation comes from either our own sun or from a source outside of our galaxy
- Radiation can therefore be grouped into one of three sources:
  - Solar Particles – Ejected during a solar flare event
  - Galactic Cosmic Rays – High Energy charged particles (e.g., Supernova 1987A)
  - Trapped Particles – Particles trapped in the Earth's radiation belts
- Radiation consists of electrons, protons, heavy ions, UV rays and neutrons

Radiation Effects

Effects of radiation upon Op Amp, Oscillator







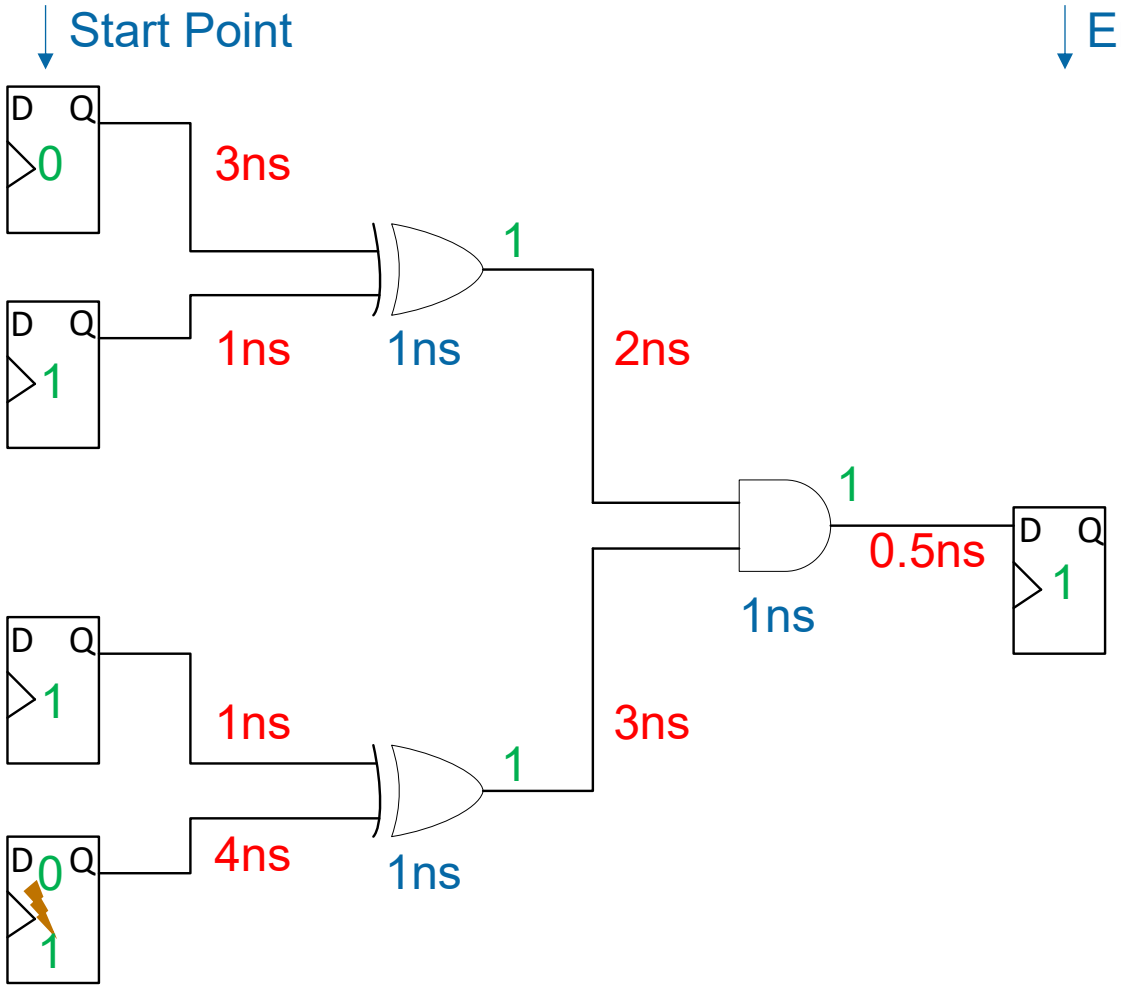
Single Event Upset

- Results in Register or Memory Corruption
- Bit Flips between state
- In FPGA SEU can corrupt FSM, Counters, Stored Data & Values
- Mitigation schemes can be deployed to minimize and tolerate SEU effects
- What is the probability SEU will be captured downstream and impact the design.

Single Event Transient

- Occurs when routing or combinatorial gate is struck
- Results in a glitch on the signal line
- SET has the potential to be clocked in and latched as a SEU
- What is the probability of the SET becoming a SEU?
- Will SET -> SEU or SEU dominate in a design ?

Single Event Upset / Transient



Will a corruption in a start point propagate ?

Depends upon when it happens

Path delay = 4+1+3+1+0.5
Path delay = 9.5ns

If the register is corrupted between

0ns and (Clk Period-Path Delay)ns

The SEU will propagate, otherwise it is filtered out

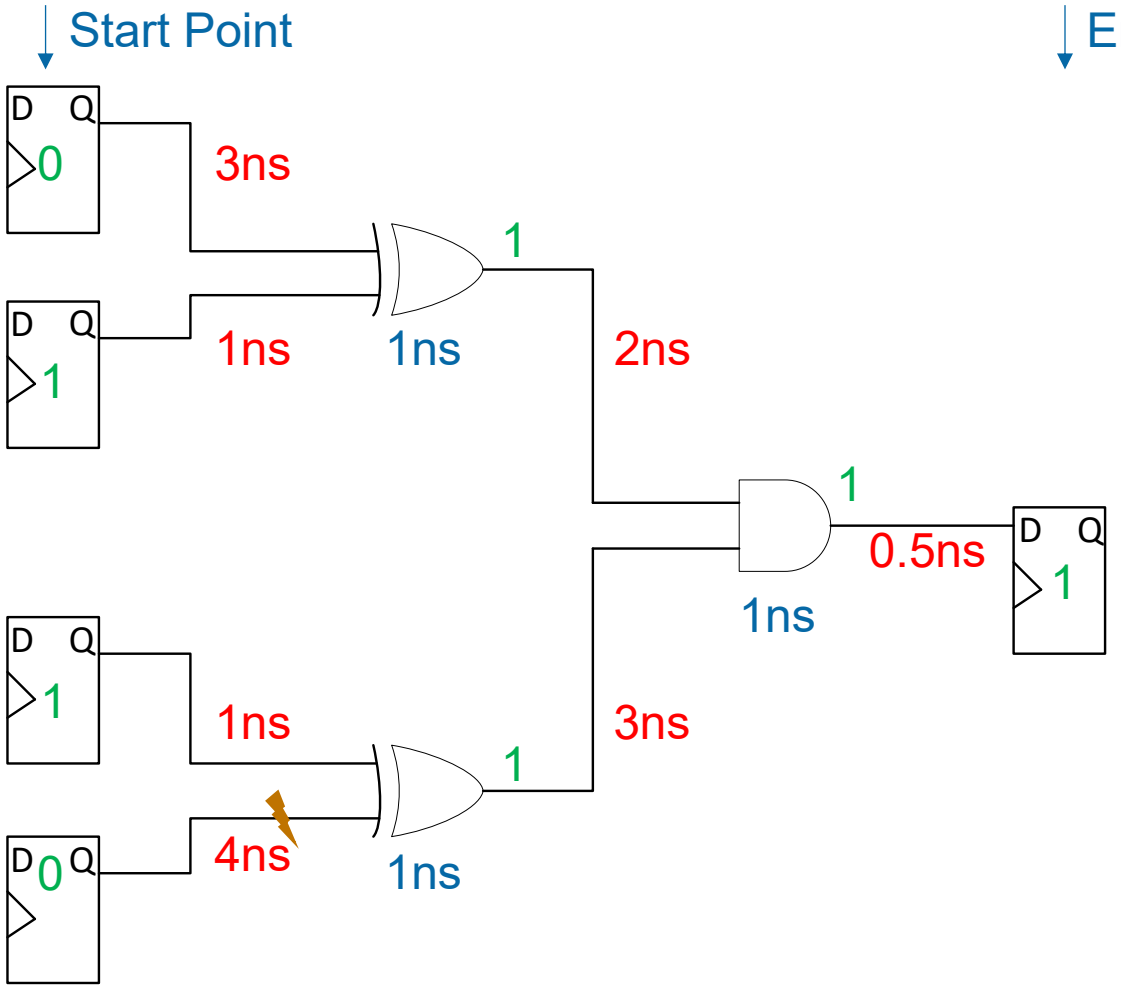
Single Event Upset / Transient

- Capture percentage of a Start point SEU being captured by the end point is therefore

$$Capture\ Percentage\ of\ Clock = 1 - Path\ Delay\ * Clock\ Frequency$$

- What is the probability of a SET occurring in the logic path being captured?

Single Event Upset / Transient



Will a SET in a signal or gate be captured?

Width of SET depends greatly on the Silicon Technology and Node – 50ps & 1000ps

Capture % of Clock Period = Twidth * FS

Time to capture

T < (Tclk –Tdly) + Twidth
T > (Tclk –Tdly) - Twidth

Single Event Upset / Transient

- SEU and SET capture is therefore Frequency dependent
- As frequency increases
  - Probability of SEU propagation from Flip Flop to Flip Flop decreases
  - Probability of SET capture by Flip Flop Increases

Hands on Example

- Create a simple spreadsheet – Demonstrates the SEU / SET propagation percentage

- Parameters are
  - Tdelay = 3.5 ns
  - Twidth = 1000ps
  - FS variable between 0 and 200 Mhz

Hands on Example Solutions

Slow Clock Speed SEU – SEU Dominates

Clock (FS)	6	MHz	
Delay	3.50E-09	Width	1.00E-09
Capture SEU %	0.9790	Capture SET %	0.0060
	97.9000%		0.60%
t	1.63E-07	tmax	1.64E-07
		tmin	1.62E-07

Higher Clock Rates SET -> SEU increases

Clock (FS)	200	MHz	
D=Delay	3.50E-09	Width	1.00E-09
Capture SEU %	0.3000	Capture SET %	0.2000
	30.0000%		20.00%
t	1.50E-09	t max	2.50E-09
		t min	5.00E-10

Very Variable, depends upon technology, implementation (delay), SET Width

Section Quiz

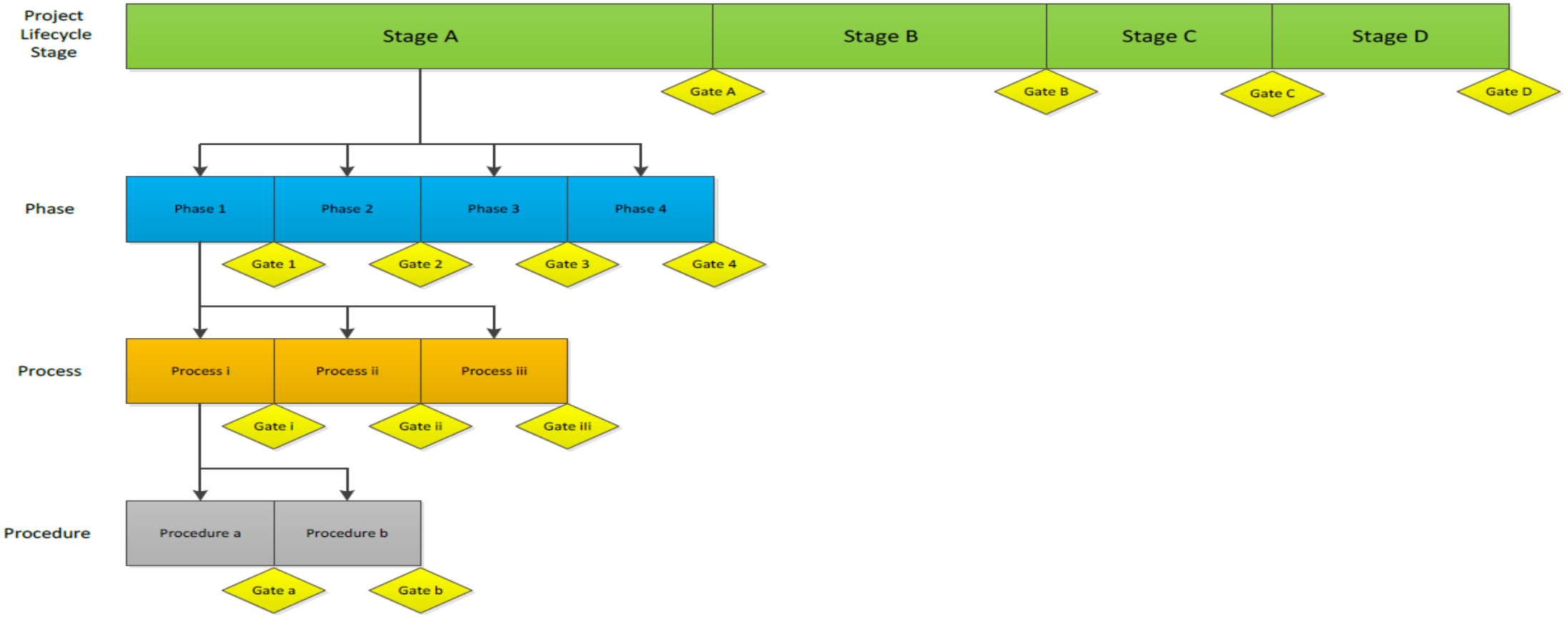
- How can ambient temperature effect our system
- How can acoustic noise be seen by electronic equipment
- What are the kinds of Vibration which might occur
- Do these effects just result in physical failure or can they manifest in other ways
- What is the role of structural model
- What kinds of Radiation do we face in space environment
- What kinds of Radiation do we face in terrestrial environment

Development process

ADIUVO
ENGINEERING AND TRAINING, LTD.

ADIUVO
ENGINEERING AND TRAINING, LTD.

Engineering Governance



Engineering Governance

Stage Review

Module Review

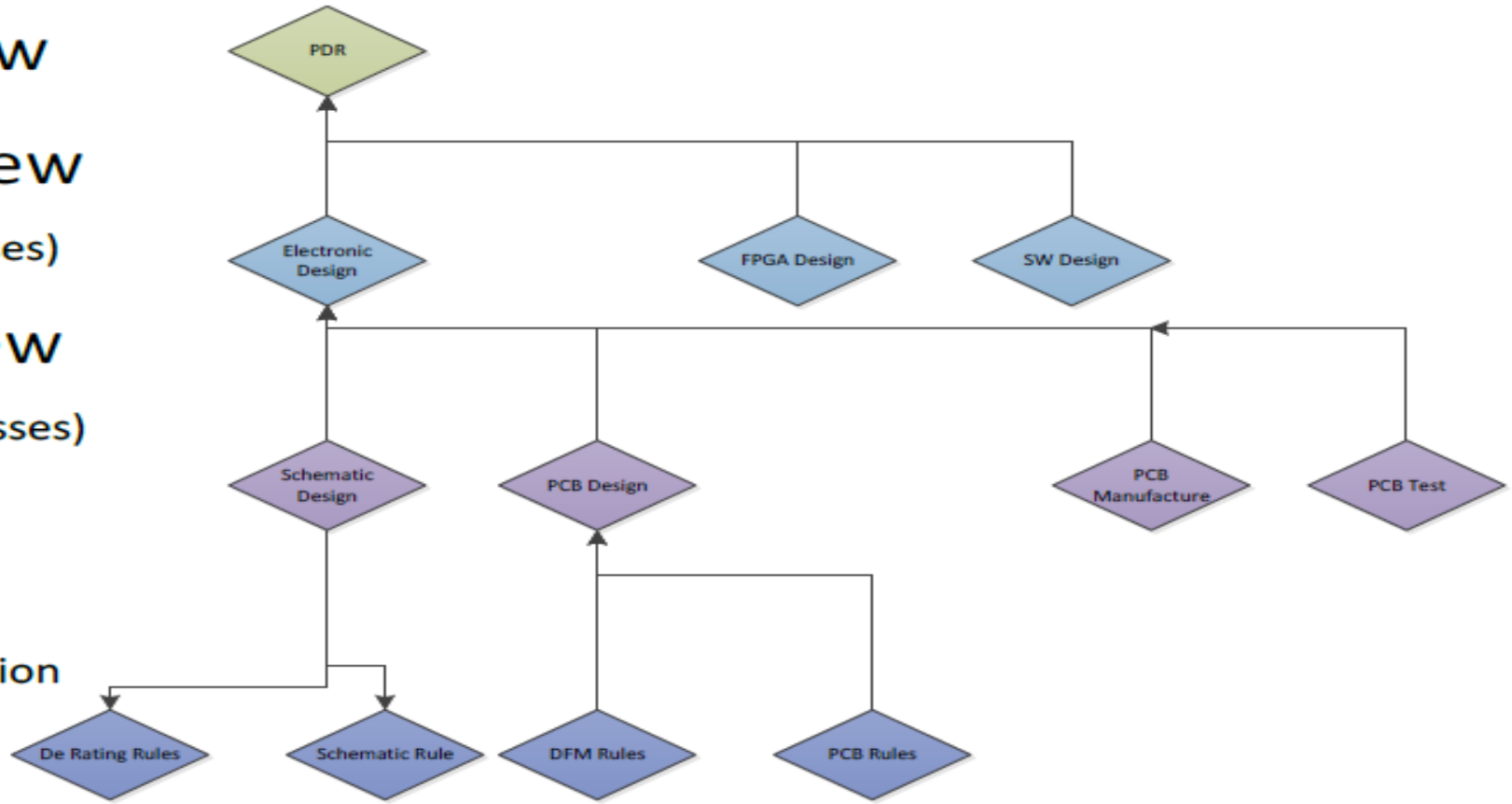
(Resulting from phases)

Design Review

(Resulting from Processes)

Element Reviews

(resulting from application of Procedures)



Stage Reviews – ESA-M-ST-10

Stage	Phase	Objective
System Requirement Review		• Release of updated technical requirements specifications • Assessment of the preliminary design definition • Assessment of the preliminary verification program
Preliminary Design Review		• Verification of the preliminary design of the selected concept and technical solutions against project and system requirements • Release of final management, engineering and product assurance plans. • Release of product tree, work breakdown structure and specification tree • Release of the verification plan (including model philosophy).
Critical Design Review		• Assess the qualification and validation status of the critical processes and their readiness • Confirm compatibility with external interfaces • Release the final design

Stage Reviews – ESA-M-ST-10

Stage	Phase	Objective
Manufacturing Readiness Review		• Release flight hardware/software manufacturing, assembly and testing
Test Readiness Review		• Release assembly, integration and test planning
Test Review Board		• To confirm that the verification process has demonstrated that the design, including margins, meets the applicable requirements. • To verify that the verification record is complete at this and all lower levels in the customer-supplier chain. • To verify the acceptability of all waivers and deviation

Standards

Numerous standards depending upon the end application

IEC61508 – Functional Safety of Electrical, Electronic, Programmable Electronic Safety Systems. Introduces the famous SIL level, lead to many variants :-

- IEC62279 – Railway Implementation
- IEC61511 – Process Industry Implementation
- IEC61513 – Nuclear Industry Implementation
- IEC62061 – Machinery Implementation
- ISO26262 – Automotive Electrical / Electronic Systems

DO254 – Design Assurance Guidance for Airborne Electronic hardware

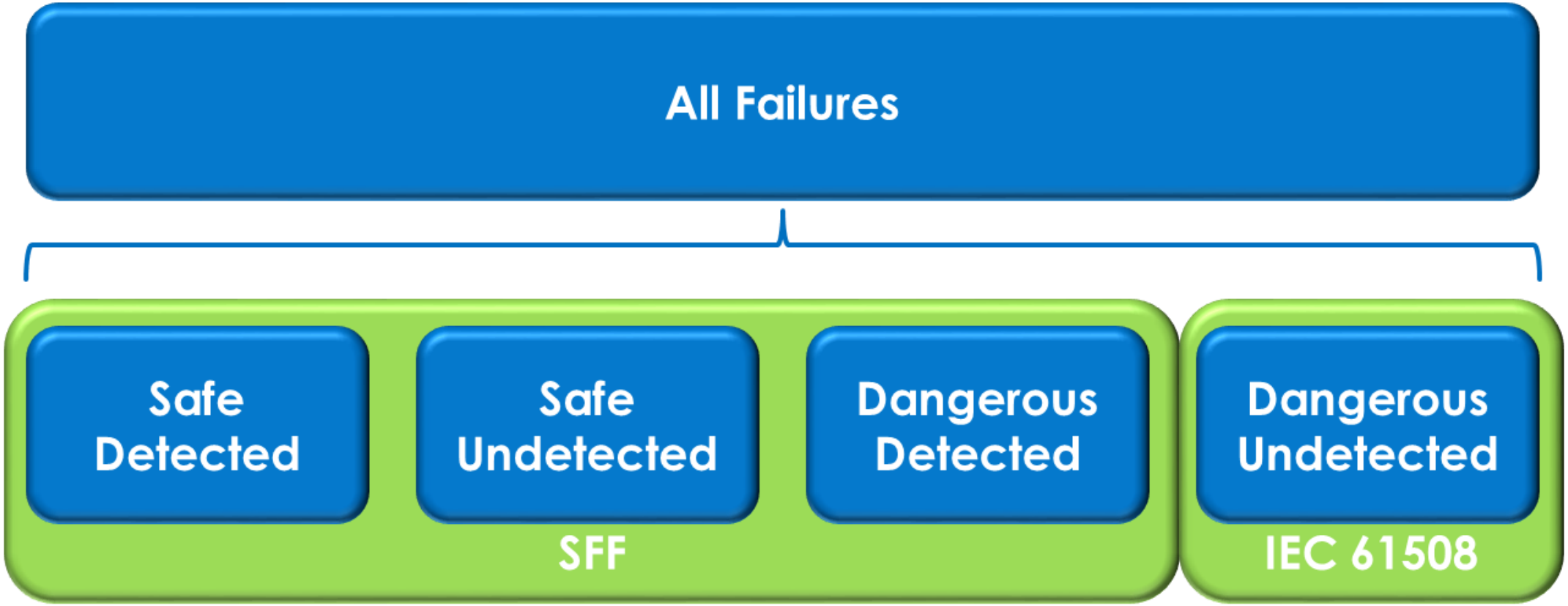
ADIUVO
ENGINEERING AND TRAINING, LTD.

SIL: Continuous Operation

SIL	Time To Failure (hours)	Failure in Time (FIT)
4	100,000,000	10
3	10,000,000	100
2	1,000,000	1,000
1	100,000	10,000

ADIUVO
ENGINEERING AND TRAINING, LTD.

Failure Space



Safe Failure Function

Proportion of all safe and detected failures based on the total amount of failure

FME(D)A (Failure Modes, Effects and Diagnostics Analysis)
Methods of analysis for quantitative determination of types of failure and failure rates

			Device type A				Device type B			
SIL-Level			Safe Failure Fraction (SFF)							
	High Demand Mode	Max. acceptable failure of the safety system	<60%	60...90%	90...99%	>99%	<60%	60...90%	90...99%	>99%
	$10^{-5} \le PFH < 10^{-4}$	One risk of failure every 10,000 hours								
1	$3 \cdot 10^{-6} \le PFH < 10^{-5}$	One risk of failure every 1,250 days	HFT 0				HFT 1	HFT 0		
	$10^{-6} \le PFH < 3 \cdot 10^{-6}$	One risk of failure every 115.74 years								
2	$10^{-7} \le PFH < 10^{-6}$	One risk of failure every 115.74 years	HFT 1	HFT 0			HFT 2	HFT 1	HFT 0	
3	$10^{-8} \le PFH < 10^{-7}$	One risk of failure every 1,157.41 years	HFT 2	HFT 1	HFT 0	HFT 0		HFT 2	HFT 1	HFT 0
4	$10^{-9} \le PFH < 10^{-8}$	One risk of failure every 11,574.1 yeras		HFT 2	HFT 1 HFT 2	HFT 1 HFT 2			HFT 2	HFT 1 HFT 2

Hardware Fault Tolerance

SIL level depends not only upon PFH but also SFF and HFT

HFT = 0 or 1oo1 - One hardware element voting one of one needed to work

HFT = 1 or 1oo2 – Two hardware elements voting one of two needed to work

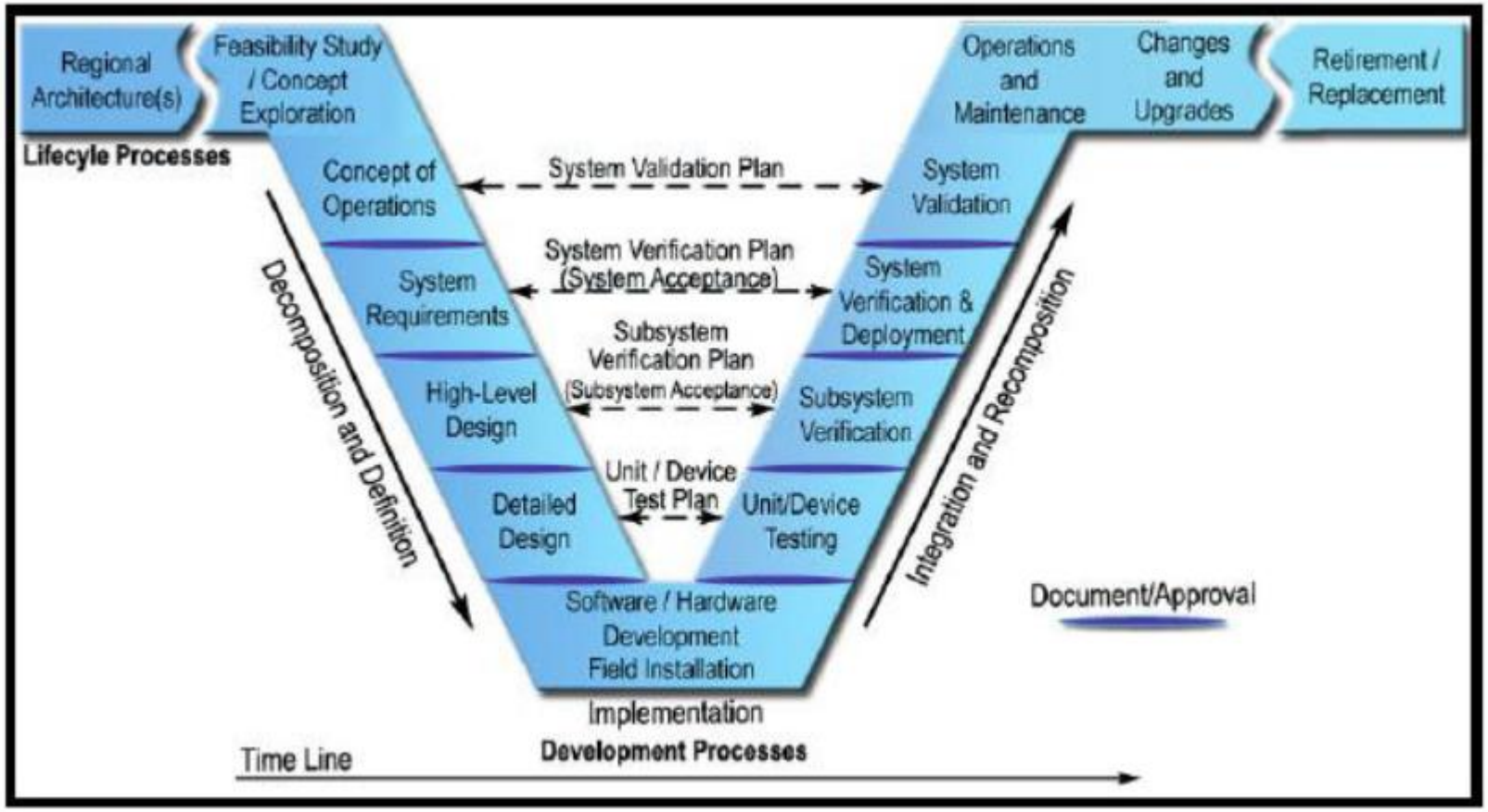
HFT = 2 or 1oo3 – Three hardware elements voting one of three needed to work

To determine HFT use equation M-N when given as MxxN

Voting is the number of paths which must work out of total paths available, M in above equation.

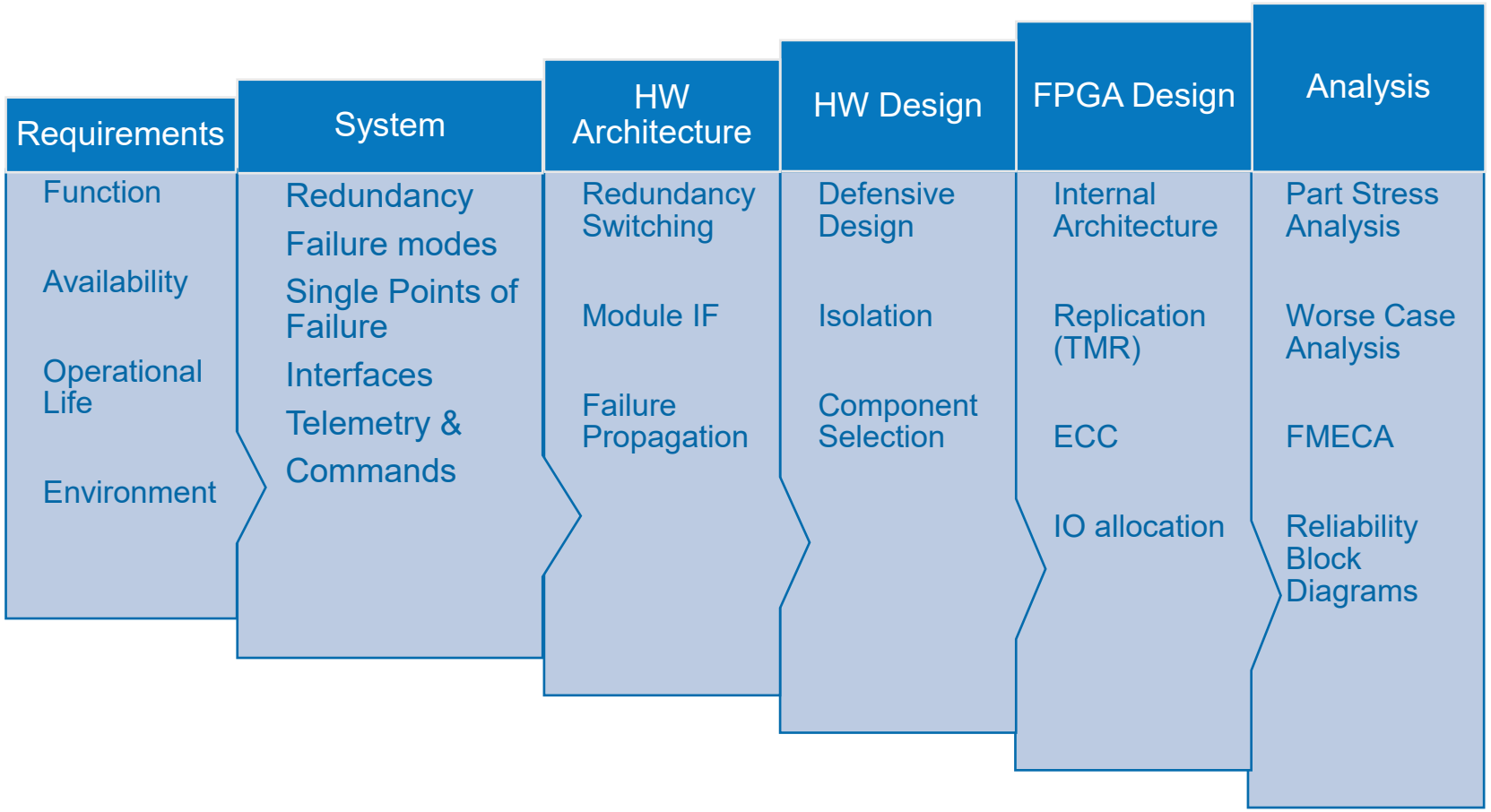
Relationship between redundancy and HFT is more complicated need to draw reliability block diagram to understand redundancy

V Model


Regional Architecture(s)
Feasibility Study / Concept Exploration
Lifecyle Processes
Concept of Operations
System Requirements
High-Level Design
Detailed Design
Software / Hardware Development Field Installation
Implementation
Development Processes
Decomposition and Definition
System Validation Plan
System Verification Plan (System Acceptance)
Subsystem Verification Plan (Subsystem Acceptance)
Unit / Device Test Plan
Operations and Maintenance
Changes and Upgrades
Retirement / Replacement
System Validation
System Verification & Deployment
Subsystem Verification
Unit/Device Testing
Integration and Recomposition
Document/Approval
Time Line

ADIUVO
ENGINEERING AND TRAINING, LTD.

Elements of the Process



Model Philosophy

Developing equipment for a satellite will require a number of models being developed prior to delivery of the final flight models.

- Engineering Model, the initial design representative in size and function to the final version by may use commercial parts
- Engineering Qualification Model, the model subjected to the qualification campaign. This usually includes design updates from the EM and uses identical components to flight however, they do not need to be screened to flight standard
- Proto Flight Model, A flight model subjected to the same qualification levels as the EQM. Sometimes for cost issues this is also flown
- Flight Model, the models actually flying these are tested to acceptance level testing.

Model Philosophy

This model philosophy will effect the choice of FPGA used

- Engineering Model, FPGA design is likely to change often so either a socketed device (if final device is one time programmable) or commercial reprogrammable device is used. Most suppliers of space grade components offer a low cost prototype solution.
- Engineering Qualification Model, the FPGA should be a representative of the flight version in every way although it may be screened to a lower level e.g. Xilinx B Flow or RTAX Proto
- Flight Model, the final FPGA screened to the required levels needed for the mission.

ADIUVO
ENGINEERING AND TRAINING, LTD.

Section Quiz

- What are the key reviews of a project
- What is the model philosophy
- What is the V Model
- What are the elements of the failure space
- What is the Safe Failure Fraction

Requirements & Verification

ADIUVO
ENGINEERING AND TRAINING, LTD.

Requirements

Requirements

Requirements define the system you are developing get it wrong at the this stage and the project will have issues.

Requirements should therefore be:-

1. Necessary – Success cannot be achieved without these requirements
2. Verifiable – You must be able to ensure the requirement can be implemented via inspection, test, analysis or demonstration
3. Achievable – Technically possible, with the constraints
4. Traceable – Can be traced to from lower level requirements, traces to higher level requirements
5. Unique – Prevents contraction between requirements
6. Simple and Clear – Each requirement specifies one function

Requirements

Language used to identify requirements needs to be clear and concise.

SHALL – Denotes a mandatory requirement

"The equipment SHALL have a 1553B Interface"

WILL – Denotes the purpose of the part or system

"The equipment WILL be used to encrypt the tele-commands"

SHOULD – Denotes a non mandatory requirement

"The equipment SHOULD be able to store 10 encryption keys"

Requirement Documentation

Typical requirement documentation through out a project life cycle

- **Customer Requirements** – High level of requirements, all requirements will trace to these.
- **Applicable Standards** - EMC, Analysis, Derating, Electrical etc
- **Equipment Requirements** – These expand on the customer requirements to define the equipment performance
- **Statement of Compliance** – Identifies compliance or not against each of the requirements, can be used to raise request for deviations, request for waivers
- **Interface Control documents** - Define the electrical interfaces of the module along with the protocols required to communicate with the equipment
- **Mechanical Interface Document** – Defines the equipment dimensions, mass, centre of gravity etc
- **Sub System Segmentation** – Identifies which requirements are implemented in hardware, software, FPGA etc
- **Architectural Design Description** – Describes how the requirements are to be addressed for the Hardware, software FPGA etc
- **Verification Matrix** – Defines how each of the requirements will be tested analysis, inspection, test or demonstration
- **Test Specification** – Details how each of the requirements is to be tested

Requirement Documentation

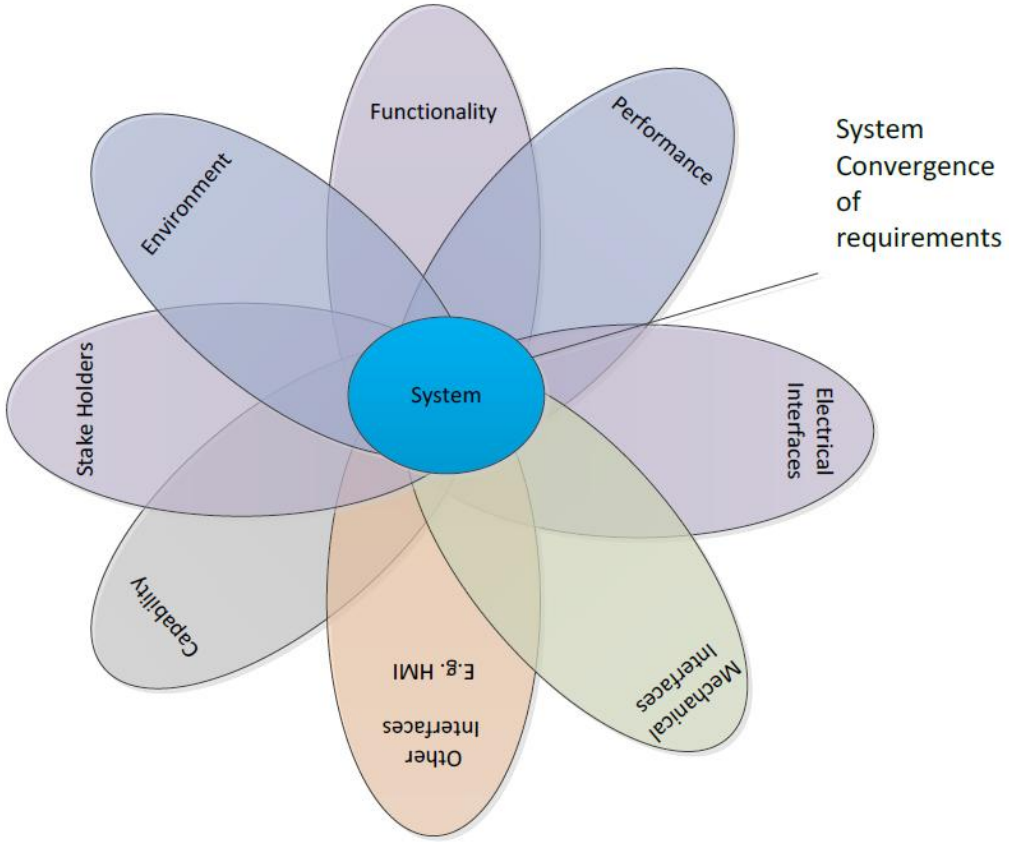
- **Test Procedure** – Details how the tests in the test specification are to be implemented
- **Engineering Budgets Document** – Defines the power, mass, memory budgets available for the mission
- **EMC Control Plan** – Defines the approach taken to ensure the EMC requirements will be achieved
- **Safety and Compliance Plan** – Defines how the safety and regulatory marking requirements
- **Transportation and Storage Plan** – Defines the requirements for safe transportation and storage of the equipment
- **Reliability Analysis** – Analysis which determines

Requirements

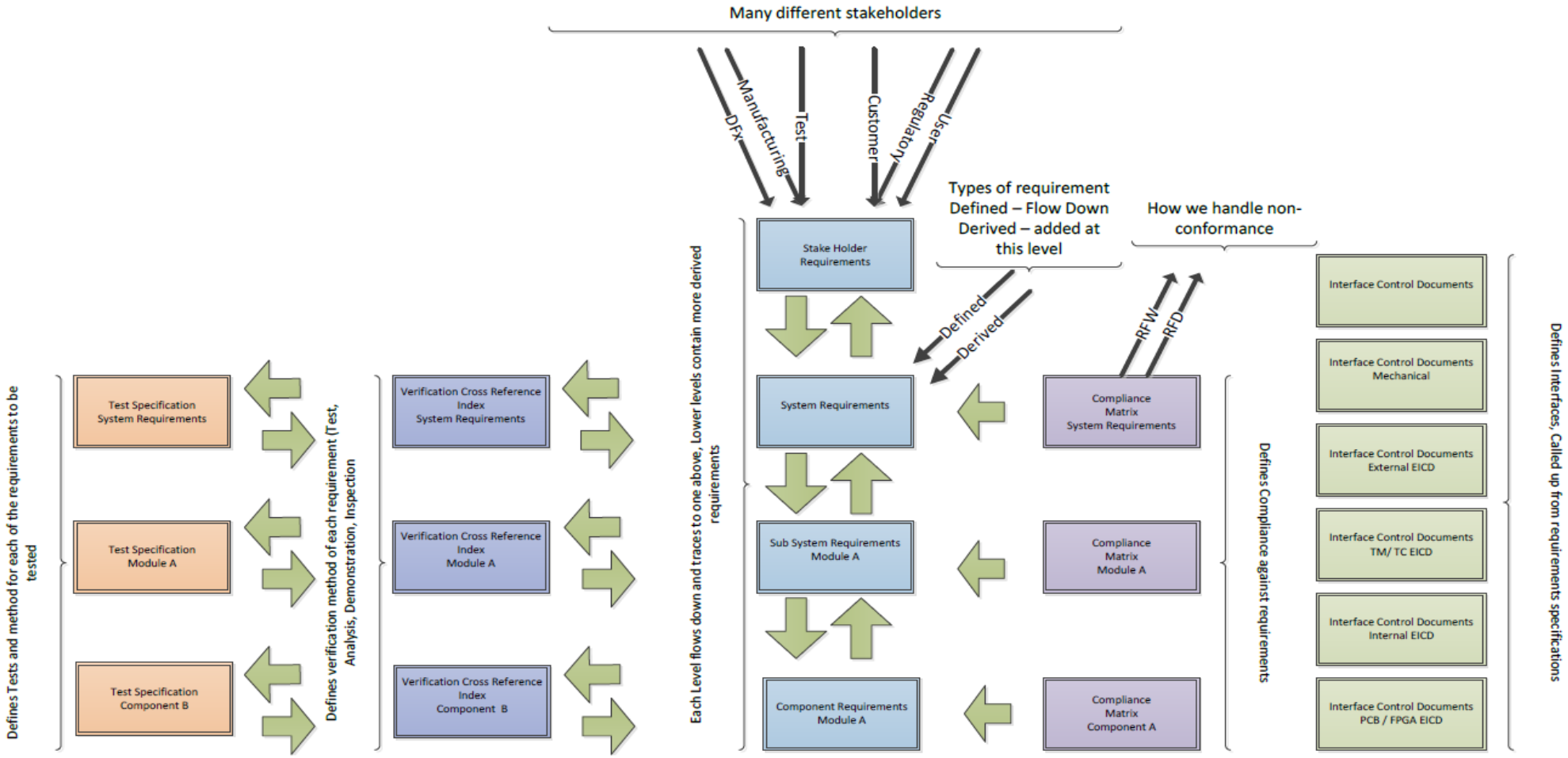
Requirements shall contain

- System life time
- Operating environments
- Temperature – Maximum, Minimum, Rate of Change
- Shock and Vibration
- Radiation
- EMC / EMI / ESD
- MTBF or even better Probability of Success
- Permissible Down time
- SIL and Probability of Failures / Success

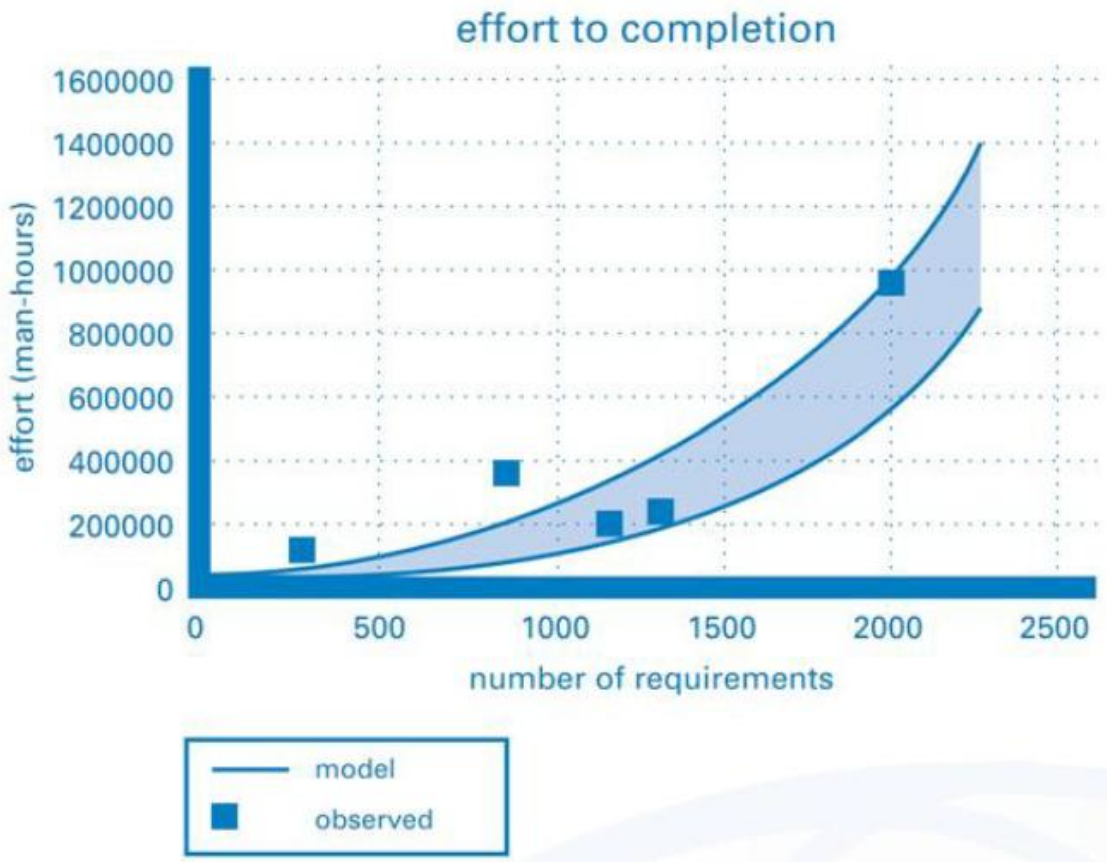
Also need to be written in SMART Manner



Requirements



Good Estimate to Effort



ESA Development Flow

ADIUVO
ENGINEERING AND TRAINING, LTD.

ESA Q60 Development

Commonly used Space FPGA development Framework Phases

- Definition Phase
- Architectural Phase
- Detailed Design Phase
- Layout Phase
- Implementation Phase
- Verification Phase

Definition Phase

"The aim of this development step is to establish an ASIC and FPGA requirements specification, a feasibility and risk analysis report and an ASIC and FPGA development plan."

Process

- Examine Feasibility of the development
- Identify Technical and project risks associated with the development
- Create FPGA development plan
- Generate FPGA requirement Specification

Outcomes

- FPGA requirements specification
- Feasibility and risk analysis
- ASIC and FPGA development plan

Architectural Phase

"During the architectural design phase, the architecture of the chip shall be defined, verified and documented down to the level of basic blocks implementing all intended functions, their interfaces and interactions."

Process

- Create Architecture of the device
- Define Interfaces
- Generate Verification Strategy
- Generate Models

Outcomes

- FPGA Architecture Document
- Verification Plan
- Models

Detailed design

“During this phase the high-level architectural design is translated into a structural description on the level of elementary cells of the selected technology and library.”

Process

- Create the Detailed Design
- Create the test benches
- Generate the netlist
- Generate the detailed Design Report

Outcomes

- RTL of the design
- Design Report
- Validation Plan

Layout Phase

"The layout shall generate the placement and routing information to meet the design rules, timing and other constraints."

Process

- Synthesize the Design
- Place and Route to the targeted technology
- Verify the timing constraints are all achieved.

Outcomes

- Programming File for the FPGA
- Layout Report

Prototyping Phase

"In this phase chips are manufactured and packaged, or FPGA's programmed, and the prototypes are tested."

Process

- Program the FPGA
- Verify Correctness of FPGA programming

Outcomes

- Programmed Test Devices
- Production Test Results
- Burn-in Reports

Design validation and release

"Final release of the FPGA"

Process

- Perform Validation as per the Validation Plan
- Compare measured parameters with specified requirements
- Validate performance across qualification requirements

Outcomes

- Validation Report
- Release Report

System Level

ADIUVO
ENGINEERING AND TRAINING, LTD.

What is Mean Time Between Failure and What does it mean

If a system has a MTBF of 8760 Hours (1 year) does that guarantee the system will operate without error for a year?

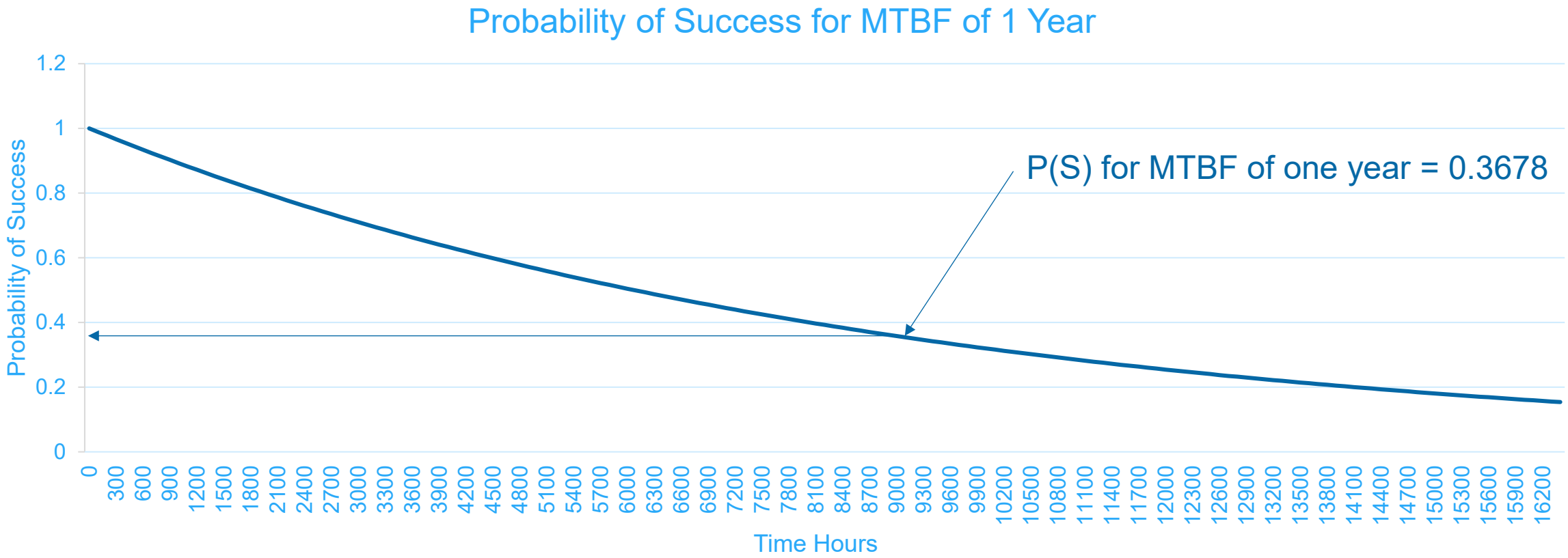
MTBF

NO the MTBF is used as an input to calculate the probability of success [P(s)] of a mission for a specified duration but on its own it does not ensure the system will operate error free for the MTBF. The probability of success is given by

$$P(s) = E^{(-t/MTBF)}$$

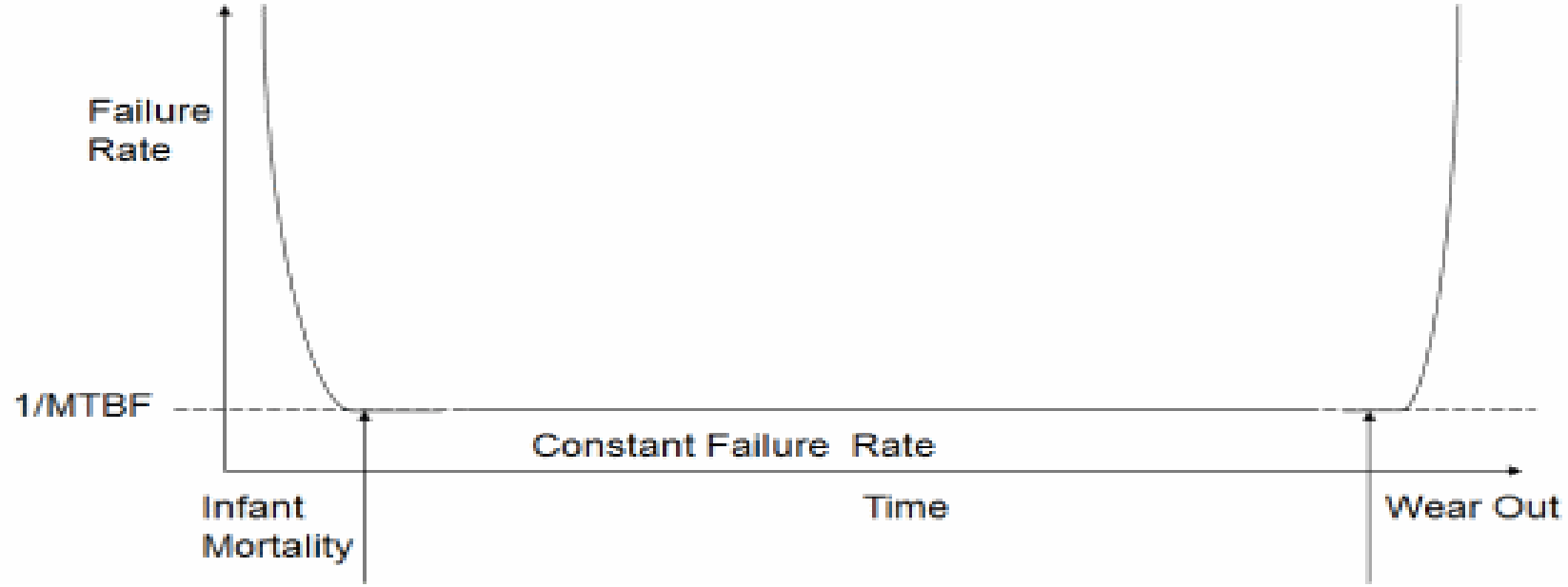
Where t is the mission time in hours

Probability of Success



MTBF

The MTBF is also only valid for the constant failure rate period of the equipment and does not include infant mortality or wear out phase.



For this reason units are normally burnt in to weed out infant mortality

MTBF – Where on the curve are we ?

The failure rate is only valid during the constant failure period, how do we determine when that is the case?

Weibull analysis of a batch of units tested for a specified duration, recording the times at which these modules failed (be warned it gets more complicated if not all modules fail)

Humidity and temperature can be used to accelerate the life of these modules. Acceleration factors can be determined using Arrhenius (increased temperature) and Coffin Manson (temperature cycle acceleration) – Humidity is also needed for none hermetic packages – due to moisture ingress

Weibull analysis can be used to determine the distribution of these failures and hence location on the curve

Weibull

What we are really interested in is the shape parameter referred to as β this indicates where on the bath tub you are

β < 1 indicates you are in the infant mortality period and have a decreasing failure rate.

β = 1 Indicate a constant failure rate – failure rate is valid for this region

β > 1 Indicates wear out period

3 factors affecting failure rate

Common Reference to calculate failure rate is MIL HND Book 217 – Not been updated for many years

Electrical Stresses – Identified by Part Stress Analysis

Quality Level – Defined by Application or Customer

Temperature – Defined by operating environment and thermal analysis

Component Quality Level

Standard Commercial, Extended and Industrial

Differing standards for components

Type	Standard	Military	Space
Integrated Circuits	MIL-PRF-38535	QML Q (Class B)	QML V (Class S)
Hybrids	MIL-PRF-38536	Class H	Class K
Discrete	MIL-PRF-19500	JAN TXV	JAN S

Introduction of non-hermetic is leading to development of

Class L – Hybrids

QMV Y – Integrated Circuits

FPGA Failure Rates

Failure Rate Summary

Table 1-18: **Summary of the Failure Rates**

Process Technology	Device Hours at TJ = 125°C	FIT (1)
16 nm	1,411,811	10
20 nm	1,054,884	13
28 nm	1,133,597	11
40 nm	1,133,274	10
45 nm	1,027,456	11
65 nm	2,036,838	6
90 nm	7,405,362	2
130 nm	2,256,834	5
150 nm	2,032,788	6
180 nm	1,086,123	11
220 nm/180 nm	1,003,249	12
350 nm/250 nm	1,187,017	10
350 nm	1,013,887	12

ADIUVO
ENGINEERING AND TRAINING, LTD.

PFH Calculation

- Perform Part Stress Analysis
- Perform FMECA using Part Stress Data takes account of HFT
- SIL Verification Report, Calculate PFH, SFF hence SIL level

ADIUVO
ENGINEERING AND TRAINING, LTD.

What are we protecting against?

Systematic – Faults arising from the design or manufacturing phase

Random – Occur due to stresses, aging or the environment

- Can be permanent faults e.g. an input stuck at a level
- Temporary e.g. memory corruption due to SEU

How do we design to address these failures

Avoidance

- How to AVOID (or to reduce the Probability of) the failure occurrence? No single point failures, best Hi Rel products

Tolerance

- How to TOLERATE failures? Recognize Avoidance is not to achieved and include Redundancy scheme in mitigation

Avoidance and Tolerance

Examples of avoidance include

Functional Analysis, Derating, Design Rules, Parts Quality Level, Worse case analysis, Quality Standards.

Example of Tolerance include

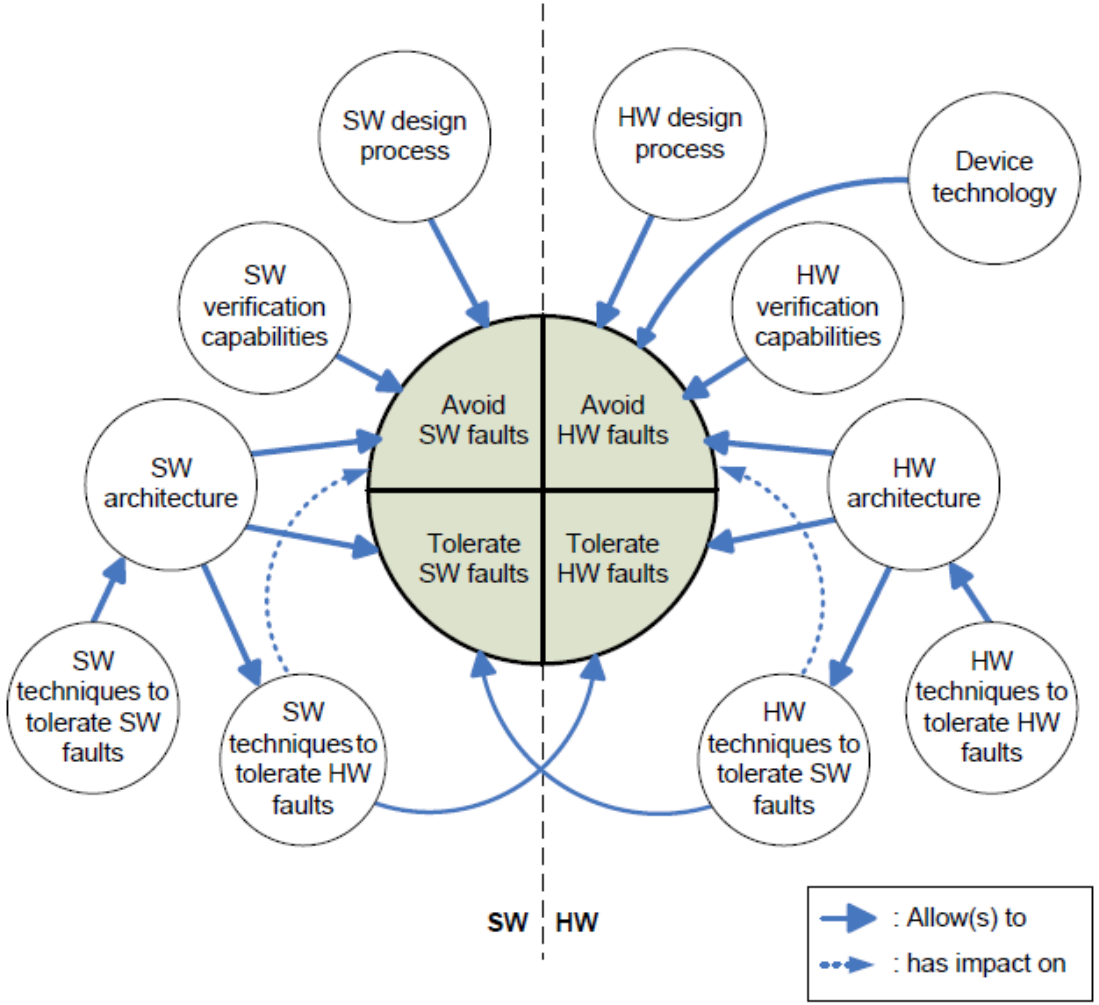
Detection – Identify a failure in the system

Localisation – Identify the failed element of the system

Isolation – Limit failure propagation

Reconfigure – Recover from the failure – e.g. redundant paths

Holistic Approach Required



Analysis at the HW Level

Analysis

- Parts Stress – Determines electrical stress on components, increases reliability by ensuring components are not operating close to the maximum stresses
- FMECA – Determines the failure modes of the design taking into account the electrical stresses, component failure modes and the criticality of these effects upon the design.
- Worse Case – Determines the design will work across worst case conditions – temperature, tolerances, aging, drift and radiation effects
- Reliability Analysis – Shows the reliability block diagram from the system and the results from the FMECA to determine the overall failure rate and probability of success for the mission.
- Thermal Analysis – Determines the junction temperatures are acceptable for the operating conditions of the unit.
- Dynamic Analysis – Determines the module can survive the dynamic loads applied throughout the mission.

Thermal Analysis

Thermal Analysis shows the junction temperature of components are within the de-rated maximum allowed at the maximum base plate temperature.

This will also consider the thermal interface between the module and the base plate to ensure the maximum base plate temperature can be maintained with the thermal load presented by the module.

This is normally given in Watts/Metre^2 this is calculated by dividing the modules maximum power dissipation by the area of the module. For example a 40 Watt unit with a area of 100 cm^2 has a thermal interface of 4000 W/M^2

Failure Rate Standards

There are a number of specifications for failure rate

Mil-Hnd Book 217F Notice 2

Telcordia SR332

Siemens Norm

FIDES

UTE 80-810

While Mil-Hnd BK 217 is the most well used it has not been updated for a number of years and as such is not accurate for modern advanced technology

ADIUVO
ENGINEERING AND TRAINING, LTD.

Component Failure Mechanisms

Component	Failure Mode
Attenuator	Open Circuit
Capacitor	Open & Short Circuit
Digital IC	Input or Output Stuck High Input or Output Stuck Low Output Stuck Tri State Supply Failure
Diode	Open & Short Circuit
FET	Gate to Drain Short Circuit Gate to Drain Short Circuit Source to Drain Short Circuit Gate to Drain Open Circuit Gate to Source Open Circuit Source to Drain Open Circuit
Filter	Open & Short Circuit
Inductor	Open & Short Circuit
Operational Amplifier	Output Shorted to +ve supply Output Shorted to –ve supply
Relay	Open & Short Circuit
Resistor	Open Circuit
Transformer	Open & Short Circuit
Transistor	Base to Emitter Short Circuit Base to Collector Short Circuit Collector to Emitter Short Circuit Base to Emitter Open Circuit Base to Collector Open Circuit Collector to Emitter Open Circuit

Two Approaches FMECA vs FTA

FMECA:

- Single event (-)
- Exhaustivity (+)
- Volume (-)
- Easiness (+)
- Extension to SW errors and operators errors (+)

FAULT TREE:

- Events combination (+)
- Based on analyst skill (-)
- Optimised (+)
- Difficult to perform (-)
- Extension to SW errors and operators errors (+)

FMECA Example – Ground Based Crypto

(a) No		(b) Item	(c) Block	(d) Function	(e) Assumed Failure Mode	(f) Local Effects	(g) End Effects	(h) Observable Symptoms	(i) Corrective Action	(j) Severity	(k) Remarks
1	1	AC/DC Converter		Provides power to unit from mains	Part failure causing input shorts to case	Fuse will blow	No output	No Output	Other AC/DC converter will provide power	2R	
1	2				Part failure causing input to fail open	No power from AC/DC converter	No output	Unit is not powered	Other AC/DC converter will provide power	2R	
1	3				Part failure causing over Current	Fuse will blow	N/A	Blown Fuse	Other AC/DC converter will provide power	2R	No effect as redundant AC/DC converter will provide power
1	4				Part failure causing over voltage	Over voltage provided to front panel	Unit will shut down	Telemetry will provide over voltage data	Power cycle or replace CGU if necessary	2SP	
1	5				Part failure causing under Voltage	Under voltage provided to front panel	Unit will shut down	Telemetry will provide under voltage data	Power cycle or replace CGU if necessary	2SP	
2	1	Front Panel			Part failure causing On/off switch fault	Cannot turn unit on/off	Unit is stuck on or off	Unable to switch on/off	Replace Unit	2SP	
2	2				Failure in key fill part	No Keys Loaded	Cannot enable encryption	Cannot enable encryption	Replace Unit	2SP	No effect until keys need loading
2	3				Part failure causing reset switch fault	none	Unable to reset CGU	Reset will not happen if switch is activated	Replace Unit	2SP	No effect until reset is required

Failure Mode Effect and Criticality Analysis

Purpose:

- To ASSESS the consequences of the elementary failures (parts, functional block...) on the system
- To DESIGN (building) and DEMONSTRATE (verification) the efficiency of the FDIR policy.
- To IDENTIFY Critical Items
- To SUPPORT contingency analysis
- To SUPPORT safety analysis

Phase:

- Preliminary: functional approach
- Detailed Design: detailed approach

Fault Tree Analysis

Purpose:

- To IDENTIFY the causes of hazardous events
- To IDENTIFY Critical Items
- To SUPPORT Safety analysis

Preliminary Design: functional approach (system level)

Discussion Point

How might we analyze an internal RTL module for the Failures ?

```
entity fsm is
  port (

    rst : in std_logic; --!clock
    clk : in std_logic; --!reset

    data_in : in std_logic_vector(31 downto 0); --!data in
    start   : in std_logic; --! 0 = idle 1 = start transaction
    mode    : in std_logic; --!0 = read 1 - write

    data_out : out std_logic_vector(31 downto 0)--!data out
  );
end fsm;
```

Discussion Point

Treat the module as a digital IC consider

- Inputs stuck high
- Inputs stuck low
- Outputs Stuck low
- Outputs Stuck High
- Outputs Stuck Tristate if external
- Corrupted data on data bus

- When developing modules – Good practice is to ensure signals are active on an edge not a level.

ADIUVO
ENGINEERING AND TRAINING, LTD.

Question

- What errors do you think can be considered for a FPGA IP Module
- What are the two fault analysis methods
- What are the two methods we use to make a reliable system
- Component quality has big effect on overall reliability true / false
- How doe MTBF, P(S) relate to each other

Hardware Level – Derating

All manufacturer component data sheets provide absolute maximum and operating electrical stresses for the device in question

If the engineer designed the system such that the device was operating with an electrical stress just below the absolute maximum the reliability of the design would be considerably lowered as it would be operating outside the recommended operating conditions

Reducing the electrical stress upon the design therefore enables the engineer to produce reliable equipment.

Hardware Level - Derating

Following a failure within the equipment.

To prevent propagation of the fault components downstream of the failure should not be stressed beyond the manufacturers maximum operating conditions.

Hardware Level - Derating Standards

Depending upon end user / application

- Mil-STD-975 – NASA
- Mil-STD-1547 – DoD
- AS4613 – US Navy
- Nav Sea TE000-AB-GTP-010 – US Navy
- ESCC-Q-30-11A – ESA
- MSFC-STD-3012 – NASA Marshall Space Centre

ADIUVO
ENGINEERING AND TRAINING, LTD.

Hardware Level - Derating

Component	Style	Parameter	De-Rating	Comment
Capacitor	Ceramic	Voltage	0.6	Reduce 0.3 @110C
Capacitor	Glass	Voltage	0.6	Reduce 0.3 @110C
Capacitor	Plastic	Voltage	0.6	Reduce 0.3 @110C
Capacitor	Tantalum	Voltage	0.6	Reduce 0.3 @110C
Capacitor	Feed Through	Voltage	0.5	
Capacitor	Variable	Voltage	0.5	
Connector	Non-RF	Voltage Current	0.5 0.5	
Connector	RF	RF Power Voltage	0.75 0.5	
Diode	Signal /RF / Rectifier / Schottky / PIN	Forward Current Reverse Current Dissipated Power	0.75 0.75 0.5	
Diode	Zener	Current Dissipated Power	0.65 0.65	

ADIUVO
ENGINEERING AND TRAINING, LTD.

Hardware Level - Derating

Component	Style	Parameter	De Rating	Comment
Inductor		Voltage	0.5	
Logic IC		Voltage Output Current	0.95 to 1.05 0.8	
Linear IC		Supply Voltage Input Voltage Output Current Transients	0.9 0.7 0.8 Not Exceed Max Rating	
Relays		Coil Pulse Duration	3 times match time or 40 mS which ever is greater	
Resistor		Voltage Power	0.8 0.5	
Resistor		Voltage Power	0.8 0.6	
Thermistor		Power	0.5	
Transistor	Bipolar	Vce Vcb Veb Collector Current Base Current Power	0.75 0.75 0.75 0.75 0.75 0.65	
Transistor	FET	Vds Vgs Drain Current Power	0.75 0.75 0.75 0.65	

Risk Analysis

ADIUVO
ENGINEERING AND TRAINING, LTD.

Risk Analysis

Need to consider a risk analysis for the development

Consider not just the FPGA but the system and team

Elements to consider include

- Maturity – Device, Tools, Design Libraries
- Available experienced development team
- Stability of Requirements – Rapidly changing.
- Operating Environment – Radiation, Temperature etc.
- Resource Utilisation – Will it fit in the design with margin.
- Technology Readiness Level – What elements of the design are low TRL and higher risk

Risk Analysis

Create Technical and Project Risk Matrix

Each entry should have a mitigation action

Each entry should be costed – financially and time scales

Each entry should be weighted by probability and impact

Factored risk budget can then be held by the project

Each entry should have a run out date

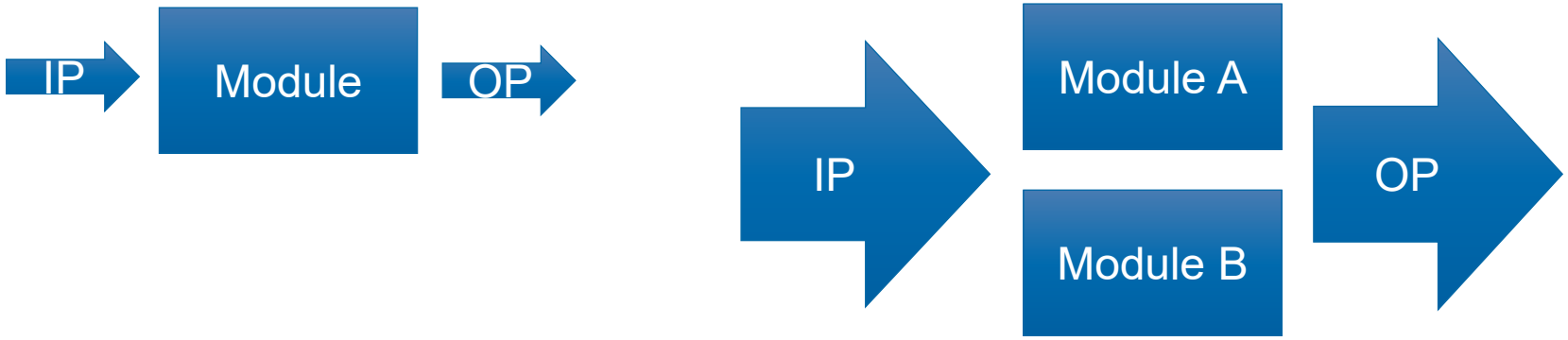
- » Enables creation of a risk retirement chart

Architectural Choices

ADIUVO
ENGINEERING AND TRAINING, LTD.

Redundancy

- Introduced when a module cannot be demonstrated to meet the probability of success required or when other architectural constraints apply, for example required SIL is above that permitted by HFT+SFF characteristics..



- Requires Independence between module A and B, especially on Input and Output
- Has significant increase in probability of success

Redundancy

Depends on architecture

- FAIL Safe - failure detected and goes to a safe state operation does not continue
- Continue operation – continues to operate following a failure – e.g. satellite

Different to HFT as we discussed earlier

Active Redundancy – transition is seamless no break in operation

Cold Spared – Redundant part powered down, takes time to get up and running

Benefit of cold spared is it ages at 1/10th the rate of the active redundancy

Redundancy

What can affect redundancy

- Electrical PROPAGATION - Ensure electrical isolation depending on failure mode
- Physical PROPAGATION - Ensure separation of redundant channels
- COMMON CAUSE – Effects all redundant channels

Redundancy

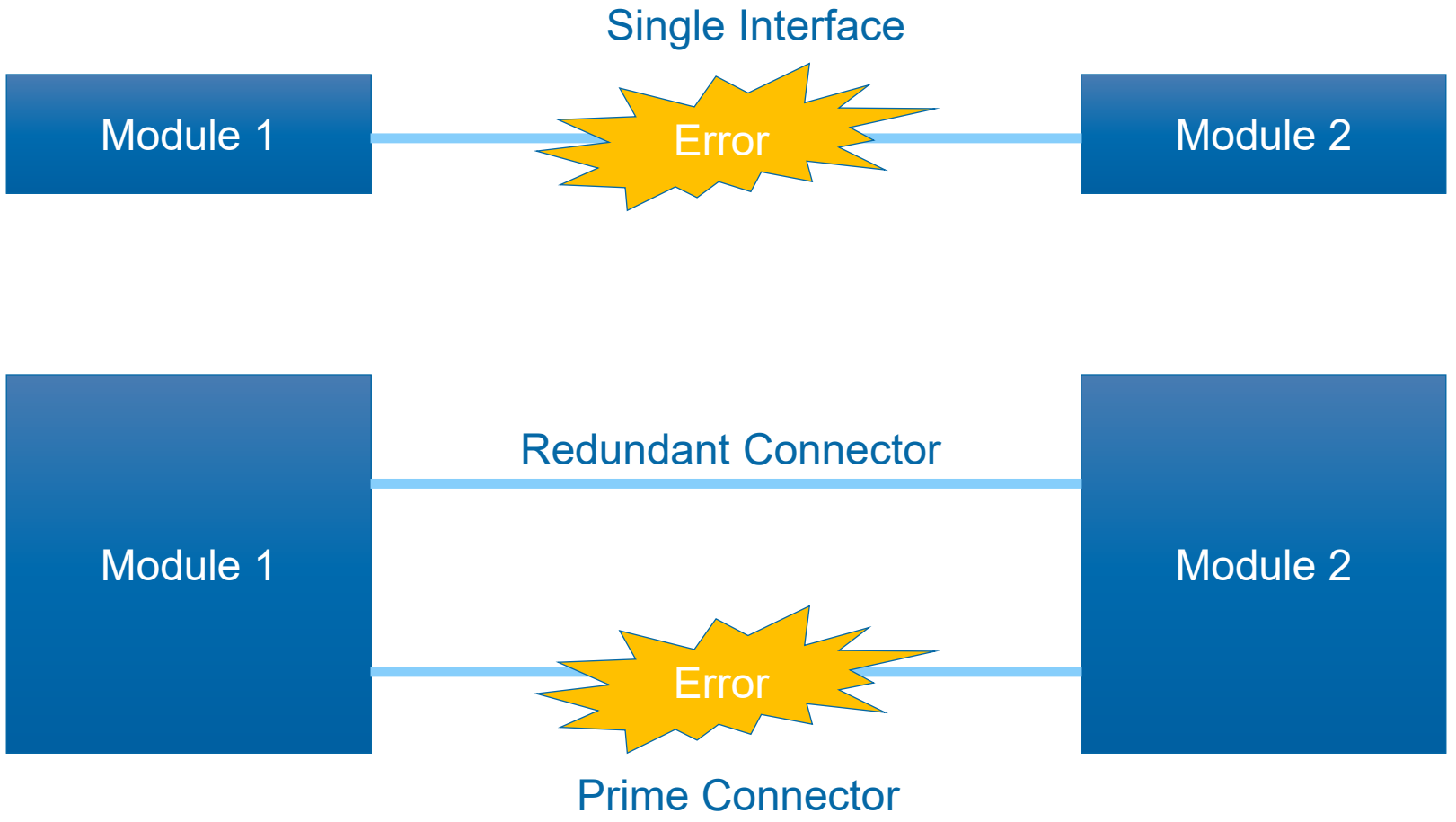
- Should we introduce redundancy
- Consider how critical the unit is to the mission.
- Consider the “predicted” failure rate of the unit and the resultant effect on the reliability.
- Consider the effect of the time delay in enabling the redundant unit. (often referred to as “Downtime”)
- Consider the impact on the power budget to have redundant unit “powered-up”.
- Consider the mass implications of having two or more units.

Common Cause failures

Within a electronic system there exist a number of Common Cause Failures

- Power Supplies and sequencing
- Clocks
- Resets
- Memories and Configuration memories
- IO interfacing – Share but need to be careful about fault propagation
- Using Component from the same batch

Connectors


Single Interface
Module 1
Error
Module 2
Module 1
Redundant Connector
Error
Module 2
Prime Connector

Error Correcting Codes

Code	Error Detecting	Error Correcting	Comment
Parity	X		Errors can be masked
N of M	X		Not suitable to multiple bit
CRC	X		Good for burst errors
BCH	X	X	Easy to Decode
Hamming	X	X	One of First EDC
Reed Solomon	X	X	Special case of BCH

Built In Test

Often used to demonstrate the testability of a system and contribute to the SFF
Significant contributor to the FMECA/FMEDA process

Generally three types

- Power on BIT (PBIT) – Runs at power on to ensure system is operational
- Continuous BIT (CBIT) – Reduced tests run continuous monitoring key parameters
- Demand BIT (DBIT) – Run on demand before critical operations for a more extensive test of the system

Testability therefore needs to be considered from day one of the development

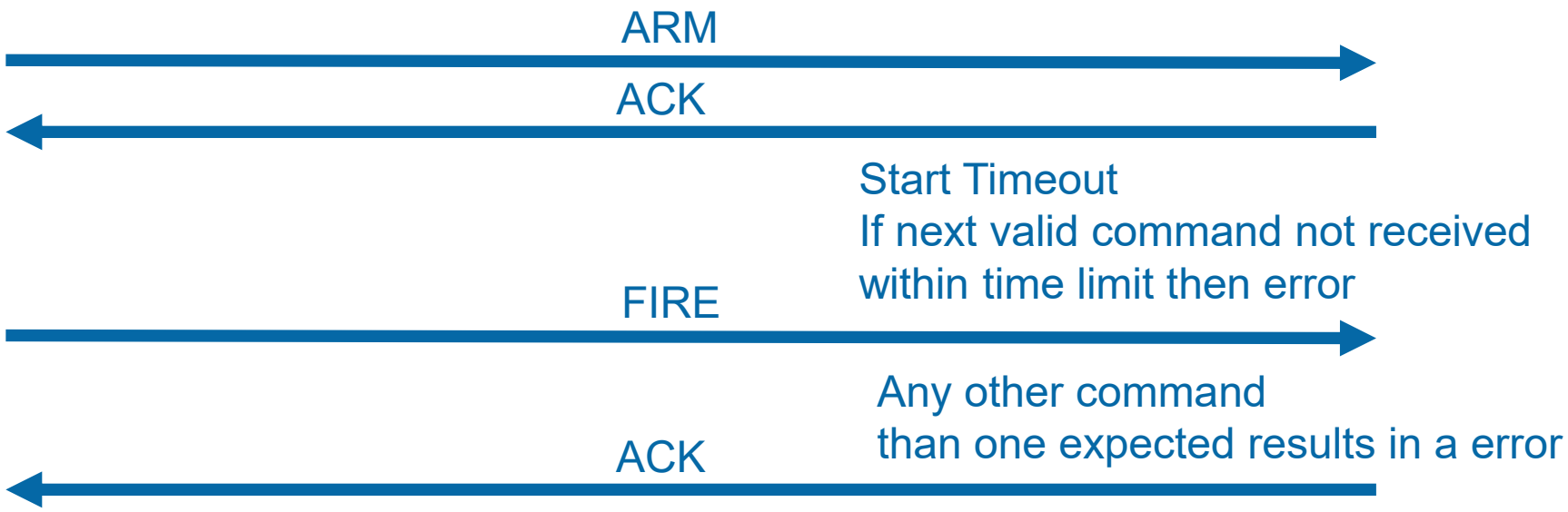
ADIUVO
ENGINEERING AND TRAINING, LTD.

BIT Parameters to consider

Parameter	Comment
Temperature	Unit and Critical Component
Current	Drawn from main supply
Voltages	The health of sub regulated voltages within the system
Redundancy Switching	Reports position of any switches used to route Prime /Redundant signals
Processing Status	Results of CRC, Over Under flow of calculations signals out of range etc.

Arm & Fire

Most command sequences require a single command, However critical sequences can utilise a ARM / FIRE approach. Two commands need to cause an action.



ADIUVO
ENGINEERING AND TRAINING, LTD.

Make use of Hamming Distance

Very useful when transferring commands across communication links

Along with Error Detection and correction on the message packet

Ensure the hamming distance is sufficient that a single bit flip cannot change one command to another dangerous command

010	FIRE
001	ARM
000	IDLE

Stale word detection

Risk of Stale Data within the data communicated outside the system.

- For example update mechanism becomes stuck, but read out still functions

Stale word indication can be vital to indicate the data is now longer being refreshed

Stale word indication toggles a bit in a word between 1 and 0 each time data is passed from the update mechanism to the read out function.

Therefore if the update mechanism becomes stuck the stale bit does not change and the receiving modules knows that data it is receiving is out of date.

Mitigation Schemes

- Parity, capable of detecting single bit errors
- Cyclic Redundancy Code, capable of detecting errors
- Hamming code, normally capable of detecting two and correcting one bit error
- Reed-Solomon code, capable of correcting multiple symbol errors
- Convolutional codes, capable of correcting burst of errors
- Watchdog timer, capable of detecting timing and scheduling errors
- Voting, capable of detecting and selecting most probable correct value
- Repetition in time, capable of overcoming transient errors

Originally compiled by Kenneth A. LaBel, NASA

ADIUVO
ENGINEERING AND TRAINING, LTD.

Section Quiz

- What is stale word detection why might we use it
- How can we use Hamming Distance to our benefit
- What is cross strapping
- What parameters might we want to consider for BIT
- What is the largest challenge of redundancy

Hardware Architecture

ADIUVO
ENGINEERING AND TRAINING, LTD.

Hardware Level - Connectors

Connectorisation

How does the system connect to the wider world or system

- Different connector styles to prevent incorrect connections
- Separation between powers and grounds
- Derating by use of multiple pins
- Correctly addressing unused pins
- Prime and Redundant connectors
- Mating Cycles
- Selecting the correct Plug / Socket end for your application

Hardware Level - Connectors

Use of different connector types: Correct use of styles and keying can prevent incorrect mating of connectors when the system is assembled. An incorrect assembly and power application could result in many hours of design analysis to prove no parts have been subject to electrical overstress

Connector pin out: Can a power pin short to a ground pin that will affect the overall power distribution network? It is a good idea to ensure separation of power and ground and treat the pins separating them correctly.

Pin derating: A component reaches maximum reliability by reducing the electrical stress placed on it. However, there are many different standards for this: ESA, NASA, US military, etc. The basic idea is to reduce the voltage and current applied to the pins

Hardware Level - Connectors

Correctly Addressing Unused Pins: Unused pins present to hazards the first being as metallic if they are left floating they can build up a charge which could result in Deep Dielectric Discharge issues. It is therefore common to connect unused pins to ground via a resistor – this limits the current which can flow if a power pin gets shorted to a unused pin.

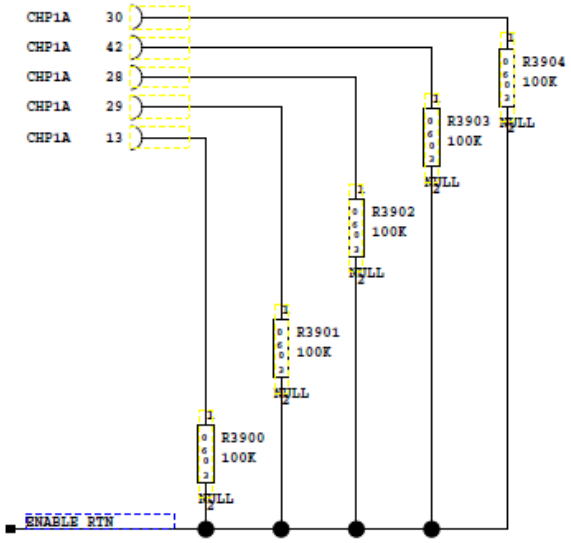
Prime /Redundant connectors: If a module has internal redundancy then control and power interfaces must be redundant.

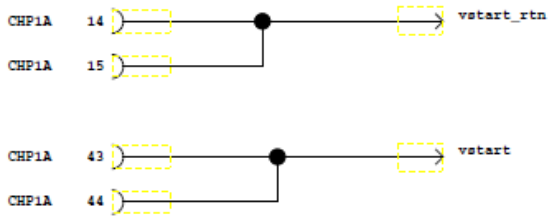
Number of mating cycles: The number of times the modules are mated/de-mated from the system has to be recorded. For this reason, many designers use *connector savers* that can be connected to the system and reduce the number of mating cycles.

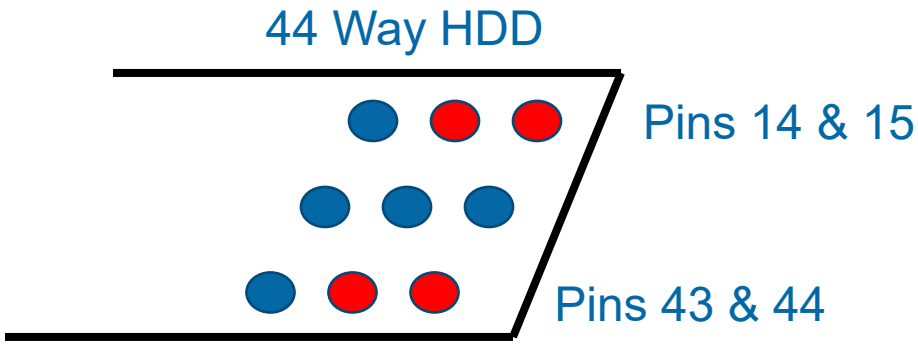
Selecting the correct Plug / Socket end for your application : If you are connecting power to a module ensure the module has a plug mating such that the cable connecting to it will be a socket hence reducing the risk of shorts or electrocution

Hardware Level - Connectors

Critical pin / signals – Ensure critical pins are surrounded by guard pins to prevent shorting other signals, power and ground e.g.



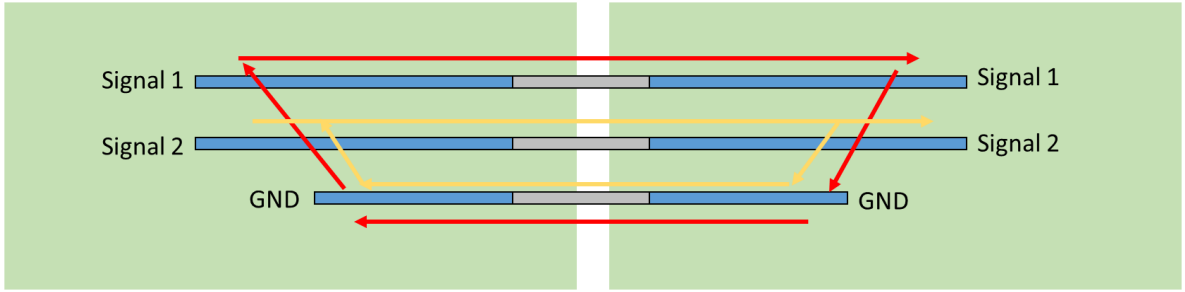




Hardware Level - Connectors

Consider return currents carefully

- For example, if there are two signal pins and only one return pin the return currents will allow flow back through the same pin creating noise in each other as shown below.



Decrease the mutual inductance of the signal by positioning the return signal as close as possible to the signal pins, in the above example the gnd signal should be between signal 1 and 2. Though this will still result in noise from cross talk.

Adding extra grounds around the signal pins will split the return current, with most current flowing through the closest ground. In the example above adding extra ground above signal 1 would reduce the ground current by a half dropping the mutual inductance and hence cross talk induced noise

Adding additional grounds between signals makes a large difference to cross talk reduction

Isolation of supplies

Power convertors, can be classified into one of two types

1. Isolating – Input voltage is not electrically connected to the output voltage.

2. Non-Isolating – Input voltage is electrically connected to the output voltage.

Isolating supplies use transformers hence providing galvanic isolation

Isolation of supplies

Why do we need isolating supplies?

Isolation barrier prevents electric shocks reduces fire risks.

Isolates down stream faults from impacting the supply voltage – very important in reliable systems.

Breaks Ground Loops

Many spacecraft requirements, specify isolating power supplies to be connected to the space craft bus voltage.

Isolation of supplies

The downside of using Isolating power supplies means each secondary return is floating therefore it is common to enable the isolating Dc Dc convertors return output to be connected to chassis.

This enable modules to all have the same ground reference and hence communicate correctly.

If this is not the case then the returns in different modules will be at different potentials and result in communication between modules being intermittent.

Isolation of supplies

Three different types of isolation

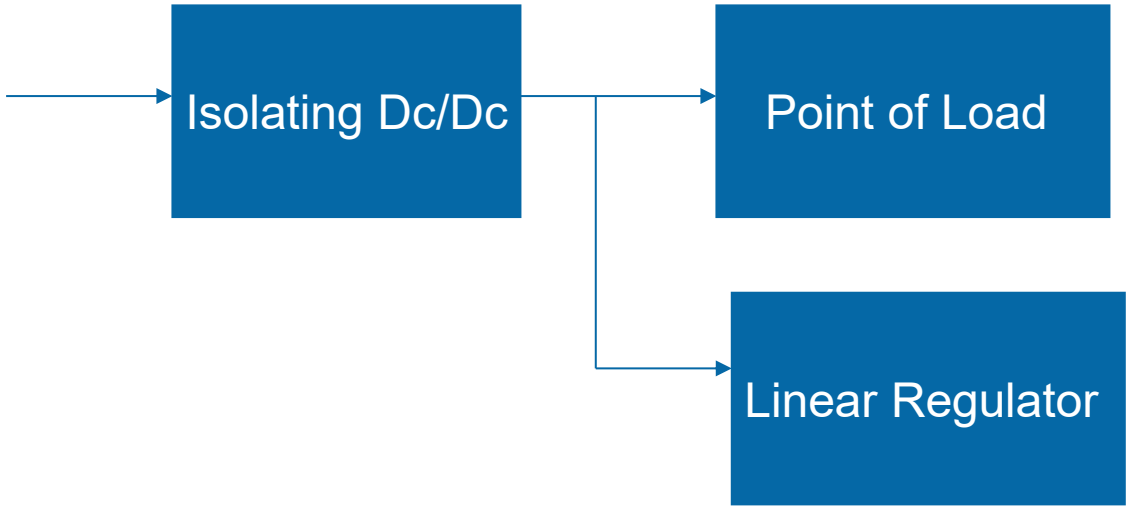
1. Functional Isolation – Primary and Secondary winding are on top of each other. Isolation is provided by the lacquer between windings.

2. Basic Isolation – Primary and Secondary windings are NOT wound on top of each other and are separated by an isolating material.

3. Reinforced Isolation – Primary and Secondary windings are NOT wound on top of each other and are separated by a physical barrier. The core is also coated with an isolating material.

Isolation of supplies

Most reliable applications will use a isolating Dc Dc supply followed by using point or load and linear regulators to generate the secondary rails required. As demonstrated below



Power Reliable Rails - Creating

In some applications a reliable auxiliary rail is required for instance for under and over voltage detection on secondary rails or providing power to redundant switching circuits.

The simplest method of creating this supply is to use schottky diodes to OR the two rails together however, is one diode on each rail enough ?

This will create a reliable rail which is roughly regulated which can be used downstream. – with current limiting circuits were used to prevent collapsing the rail.

Power Reliable Rails – Current Limiting

Current limits can be implemented in one of three commonly used methods

Constant Current: Operates as a constant current control loop, so that the output voltage varies, but the output overload current remains constant.

Example: The maximum current is 2A at 15V. Once the load current reaches 2A, then any further increase to the load current demand will fail to go beyond 2A, as the power supply is now a constant current regulator set to 2A. The output voltage falls, but the current stays at exactly 2A, even when the output is a perfect short circuit. With a short circuit, all the electrical power going in is converted to heat, hence a massive heatsink!

Power Reliable Rails – Current Limiting

Current Fold back - As the maximum output current is reached, the output voltage starts to decrease, but the greater the overload, the lower the overload current becomes.

Example: The maximum current is 2A at 15V. Once the load current reaches 2A, then any further increase to the load current demand, hence lower resistive loading, the overload current reduces to, for example 1A at 7V, and a short circuit output current may be as low as 200mA

Power Reliable Rails – Current Limiting

Current Limit: Similar to Constant Current, but not as precise, and not so good overload current regulation.

Example: The maximum current is 2A at 15V. Once the load current reaches 2A, the output voltage starts to collapse, but the overload current is not precisely regulated. It may reach, for example 2.5A when the output has a short circuit.

Power Reliable Rails – Capacitors

Most applications require decoupling and bulk capacitance on the power rails to both act as filtering by providing a low impedance AC path to ground and supply instantaneous power needs of the down stream circuits.

When you are dealing with primary power supplies which are connected to the main power supply – you need to ensure a fault cannot propagate and effect the supply.

This is also the same for reliable supplies.

ADIUVO
ENGINEERING AND TRAINING, LTD.

Power Reliable Rails - Capacitors

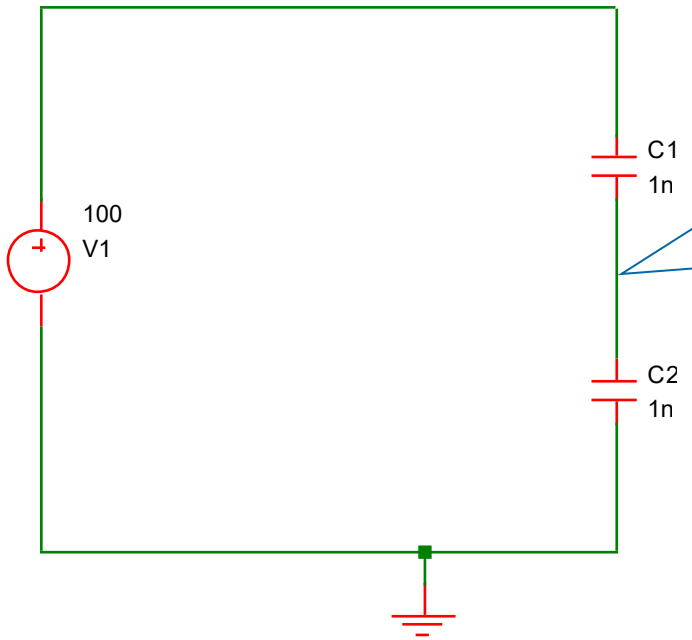
When dealing with primary supplies and reliable supplies anything connected to it must not be capable of pulling the rail down.

This is key for capacitors – which must be connected in series on primary voltages and reliable rails. This will reduce the capacitance hence you many need twice as many capacitors as you initially thought.

You need to ensure that should one capacitor fail short the other capacitor is not over stressed by the voltage applied – Remember as this is following a failure derating rules do not apply it just has to be within the manufacturers rated voltage.

Power Reliable Rails - Capacitors

When putting two capacitors in many engineers think the rated voltage doubles, as the potential voltage between them is divided.

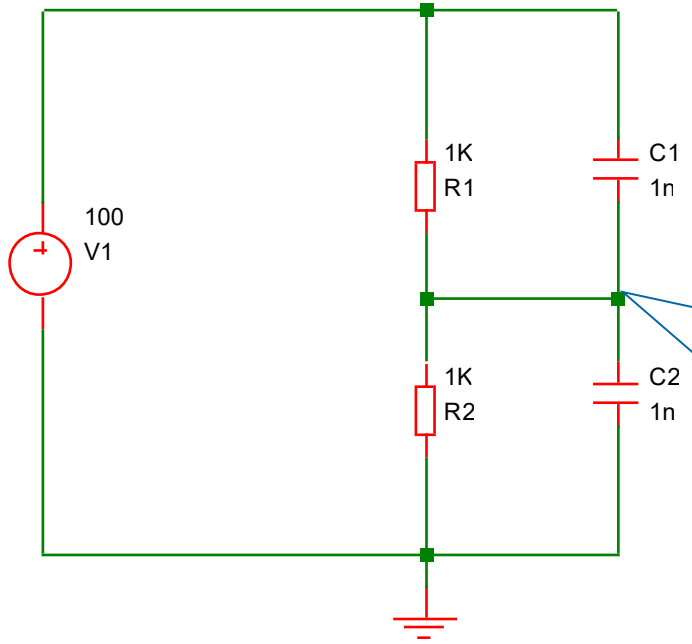


Voltage at this point is approximately half the rail voltage.

This enable 100V capacitors to be used across the rail and meet derating requirements as they are then below the 50% of the rated voltage required by the PSA rules.

Power Reliable Rails - Capacitors

However while approach shown on the previous slide does not take into account the drifts of components parameters being at different rates. Therefore the mid point of the divider will change as the circuit ages, potentially breaking the derating rules.



Addition of potential divider allows more accurate balancing across each capacitor

This also acts as a drain circuit for when the system is powered down.

This circuit is only used when voltage rating is close to maximum limit.

ADIUVO
ENGINEERING AND TRAINING, LTD.

Power -Sequencing and Reset

Power rail sequencing is often required with complex digital devices to ensure the device is configured correctly or that the current drawn during power on is not excessive.

This is often combined with the reset, allowing power rails (including downstream Point of Load regulators) to be up and stable before the reset is released.

Ideally reset will also include time for oscillators to become stable.

Power -Sequencing and Reset

Resets ensure the power rails are present and within limits and all oscillators are stable before the device starts operating.

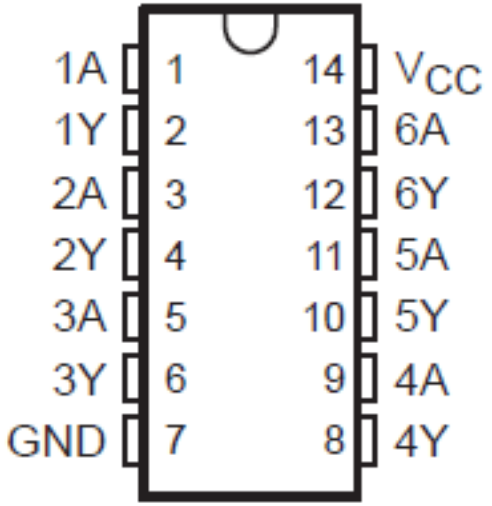
It also ensures the device being reset starts from a known configuration

- Reset will be Asynchronous to the clock domain in the FPGA
- Therefore removal needs to ensure metastability cannot occur on reset release
- Also need to ensure the slew rate of the reset is compatible with the device input requirements.

Power - Sequencing and Reset

If you are using one of the power sequencer outputs for a reset to a microprocessor or FPGA it is best to ensure a fast clean transition on the reset applied to that device. For this reason a Schmitt trigger buffer should be used to ensure the rise /fall time of the signal is compliant with the device input requirements.

For instance a AC14 Hex Schmitt inverter



Power – Over & Under Voltage Detection

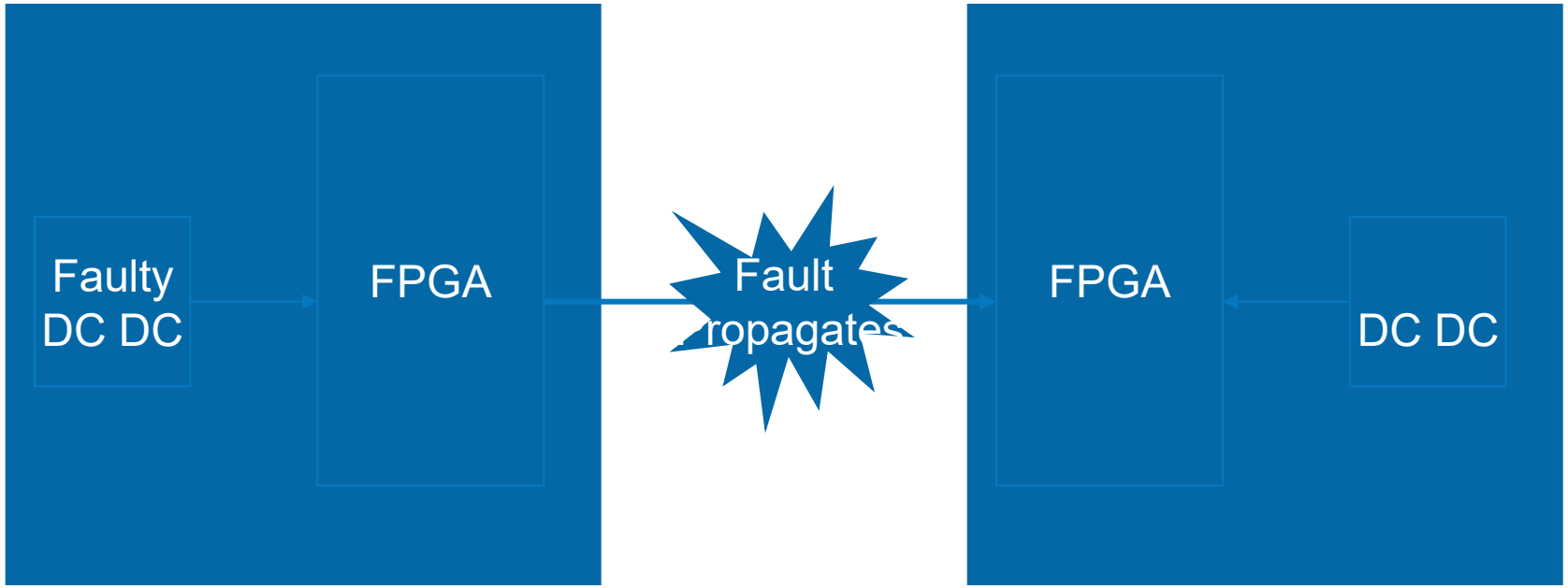
Over and Under Voltage detection monitors the output voltage of Dc Dc convertors and detects when it is out of specification.

Upon detecting this out of range voltage, the regulator or power supply (more likely) is disabled, why is this protection required?

Main reason is to prevent thermal issues should an over voltage event occur. For instance take a 1V0 FPGA core which require 10A (10W) if the output voltage (due to a failure in the Dc Dc) raises too high the component will be damaged. Worse still the power dissipated by the now failed device will increase and could present thermal issues which could effect a redundant side or result in the module catching fire.

Power – Over & Under Voltage Detection

Worse still if the device with the failed (high) power supply interfaces externally to the module. There is the risk of the fault propagating external to the module.



Power – Over & Under Voltage Detection

Protection prevents this from happening, provided the response time is fast enough.

At the very least over and under voltage protection should be included on voltage rails which power external interfaces. Unless the interface is isolated in another manner e.g. Transformer or AC coupled.

Under voltage protection is used to detect a event which is collapsing the voltage rail and disables the power supply system.

Power – Redundancy Architectures

Despite the protection provided by Isolating Dc Dc convertors, Reliable rails, Over, Under and short circuit protection we have not yet looked at reliable power switching architectures.

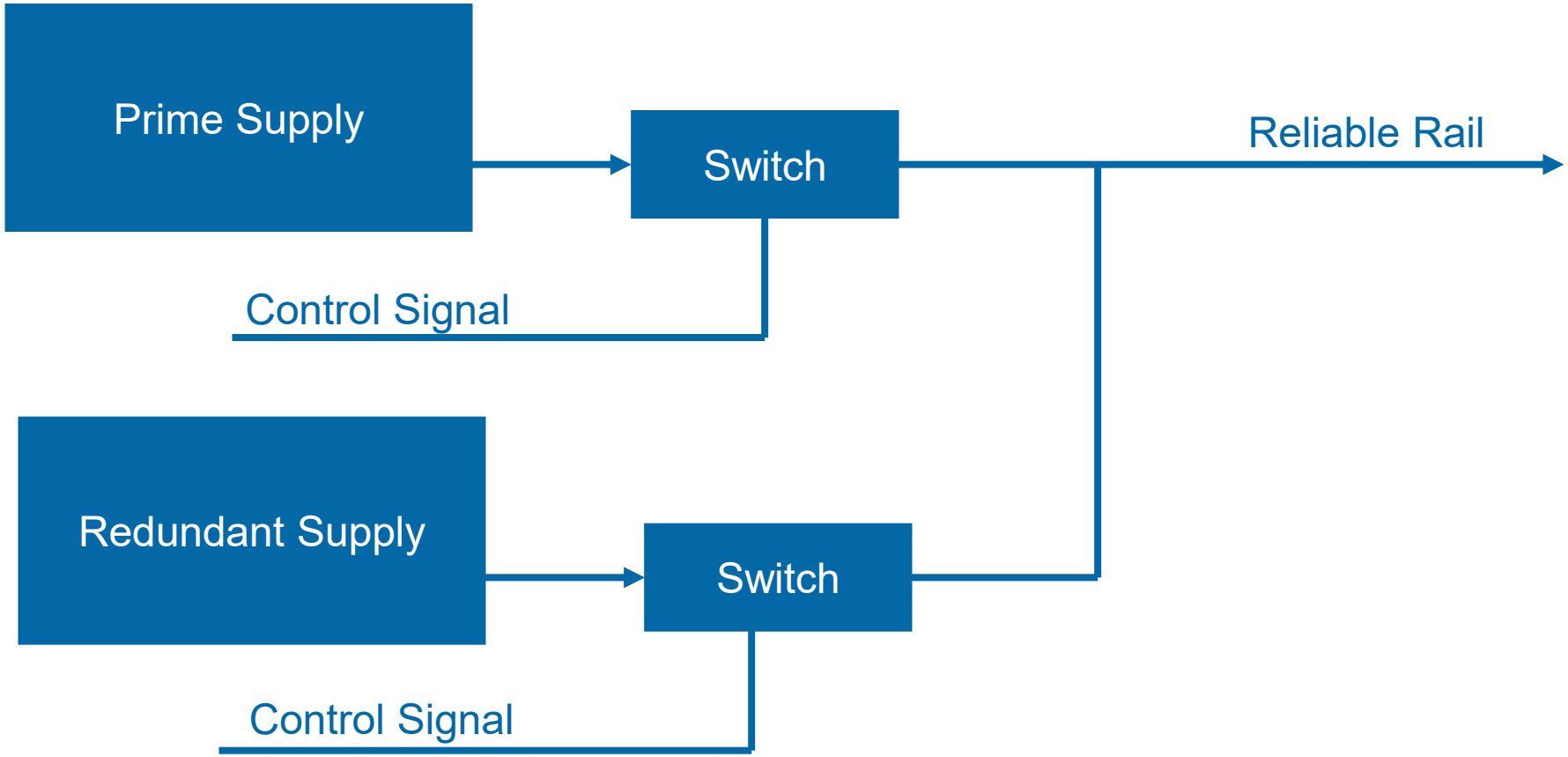
Common Architectures included

1) Prime / Redundant power supplies

2) Redundancy at module or block level requires less switching

Power – Prime / Redundant Supplies

Basic architecture as below



Power – Prime / Redundant Supplies

With Prime / Redundant power supplies as shown on previous slide the following need to be considered.

Both prime and redundant controllers must be able to control both switches.

Both switches must be designed to fail open i.e. disconnecting the supply from the rail.

Downstream modules supplied by the rail must be not be capable of pulling down the rail for instance they need to have current limiters.

Power - Linear Regulators

Commonly used in place of switching regulators where low noise is required.

Not as efficient as the switched mode supplies dissipation across the pass transistor can be considerable

They are not a magic bullet to removal of noise on the rail either.

Higher drop out voltage better noise rejection but efficiency is reduced.

Need to select linear regulator with good PSRR at frequency of interest

ADIUVO
ENGINEERING AND TRAINING, LTD.

Section Quiz

- What is the role of under and over voltage detection
- How do we decouple safely with capacitors on reliable rails
- What connector level considerations do we face
- Are Linear Regulators ideal devices

FPGA Design

ADIUVO
ENGINEERING AND TRAINING, LTD.

Benefit of FPGA

FPGA's offer several benefits to the system designer

- Flexibility of Design – performance, upgrades
- Reduction in NRE and Cost.
- Reliability – One Device as opposed to lots if using discrete devices - increased reliability with reduced solder connections.
- Time to market can be reduced.
- Maintainability – ability with some FPGA to update in the field.

Device Selection - SRAM, OTP or FLASH ?

SRAM: The FPGA program is stored in an external memory and loaded into the FPGA each time it is powered.

FLASH: The FLASH architecture of the FPGA also contains the program; no external memory device is needed.

One Time Programmable (OTP): The FPGA is applied by blowing fuses in the device. Once programmed, it cannot be modified.

Device Selection

SRAM

- Higher Performance (+)
- Higher Capacity (+)
- Needs configuration at power up (-)
- Higher Power (-)
- Susceptible to Configuration Corruption (-) – but there are mitigation schemes

OTP

- Live at power up (+)
- Cannot be updated / fixed design (-)
- Lower Power (+)
- Lowest Performance (-)

Flash based

- Live at power up (+)
- No Configuration Corruption (+)
- Updateable in the field (+)
- Middling Performance (-)

Device Selection

Typical use cases

SRAM

- Software defined radio, image processing, satellite communications, EW, AI etc.

OTP

- Control and communication, security functions e.g. crypto

Flash

- Control & Communication, Crypo, image processing, SDR

ADIUVO
ENGINEERING AND TRAINING, LTD.

Tools

- Requirements Capture tool
- Editors – Ideal with ability to LINT and check structural issues e.g HDL Creator
- Synthesis Tool – Third Party or Vendor Supplied
- Implementation – Vendor supplied – some open source for lattice
- Simulation – Third Party or Vendor
- Source Control – Subversion, GIT
- Configuration control – often called PLM tool
- Specialist e.g. Fault Injection

ADIUVO
ENGINEERING AND TRAINING, LTD.

Certified Tool Chains

T1: tools that cannot directly or indirectly contribute to the executable code of the system

T2: tools supporting verification or test of the design or the executable code; errors in these tools can fail to reveal defects

T3: tools generating outputs that can directly or indirectly contribute to the executable code of the system

Code Quality

ADIUVO
ENGINEERING AND TRAINING, LTD.

Code Quality

What do we mean by code quality

- Readable and Maintainable
- Comply with coding standards – either internal or external
- Ensuring no simulation / synthesis mismatch
- Enforce RTL clarity and reduce complexity
- Enable Testability and Traceability of the code
- Specific checks include
  - FSM without terminal states, unmapped states
  - Long If Then Else (ITE) chains
  - Control Signal test
  - Ensuring Structural correctness – e.g. used, undriven nets.
  - Gated Clocks
  - Allowable Types - e.g. std logic, unsigned etc

How can we achieve code quality?

Stringent set of coding rules – Including

- Naming conventions (Architectures, Packages, Test Benches, Signals) etc.
  - Signal should indicate bi-directional, active low and synchronisation signals
- Using Constants in place of hard coded numbers – in common packages if shared
- Use of enumerated types for FSM
- Rules on Signal vs Variable use
- Rules on Synthesis Attributes

Bad coding practice example

```
11  architecture rtl of fsm is
12
13  type state_type is (state0, state1);
14  signal current_state : state_type;
15
16  begin
17
18  process(clk,reset)
19  begin
20      if reset = '1' then
21          current_state <= state_type'left;
22      elsif rising_edge(clk) then
23        y <= '0';
24        case current_state is
25          when state0 =>
26              if x = '0' then
27                  current_state<= state1;
28                  y <= '1';
29              end if;
30          when state1 =>
31              if x = '1' then
32                  current_state <= state_type'left;
33              end if;
34         end case;
35      end if;
36  end process;
37
38  end architecture;
```

How can we enforce code quality

Linting tools – able to check the code against language and other rules

RTL Analysis e.g. Blue Pearl – Visual Verification Suite

- Enables Linting checks against standard
- Structural Design Checking
- FSM Checking
- Clock Domain Crossing Analysis
- False and Multi Cycle Path Analysis

DO254 Rules

ADIUVO
ENGINEERING AND TRAINING, LTD.

Number of Rules

Very simple and sensible approach split into 4 sections

- Coding Practices
- Clock Domain Crossing
- Safe Synthesis
- Design Reviews

ADIUVO
ENGINEERING AND TRAINING, LTD.

Coding Practice

Coding Rule Name	Rule Number	BPS Check
Avoid Incorrect VHDL Type Usage	CP1	Should be a compilation error
Avoid Duplicate Signal Assignments	CP2	PREV_ASSIGN
Avoid Hard-Coded Numeric Values	CP3	HCCC
Avoid Hard-Coded Vector Assignment	CP4	UNIFORM_OTHERS_ASSIGNMENT
Ensure Consistent FSM State Encoding Style	CP5	FSM_NO_HCC
Ensure Safe FSM Transitions	CP6	RST, TERM_STATE, UNREACHABLE_STATE
Avoid Mismatching Ranges	CP7	MOS, MBA
Ensure Complete Sensitivity List	CP8	CSL
Ensure Proper Sub-Program Body	CP9	ACCESS_GLOBAL_VAR
Assign Value Before Using	CP10	UBA
Avoid Unconnected Input Ports	CP11	UNCONNECTED_UNDRIVEN_UNUSED_PORT
Avoid Unconnected Output Ports	CP12	UNCONNECTED_DRIVEN_UNUSED_NET/PORT
Declare Objects Before Use	CP13	UBA
Avoid Unused Declarations	CP14	UNCONNECTED_UNDRIVEN_UNUSED_NET/PIN, UNUSED_VAR, UNUSED_PARAM

Safe Synthesis

Coding Rule Name	Rule Number	BPS Check
Implied Latches	SS1	MCD, MEB, ETB, MCA,MIA,MEA,UBA,LATCH_CREATED
Safe Case Synthesis	SS2	DCI, OCI, MCI, CASE_X, NO_XZ_CASE_ITEMS
Combinatorial feedback	SS3	CLPR
Avoid Latch Inference	SS4	MCD, MEB, ETB, MCA,MIA,MEA,UBA,LATCH_CREATED
Avoid Multiple Waveforms	SS5	NO_DELAY
Avoid Multiple Drivers	SS6	MDR
Avoid Uninitialized VHDL Deferred Constants	SS7	
Avoid Clock Used as Data	SS8	CLK_PROP_DFF_DATA, DIFF_CLK_PROP_DFF_DATA
Avoid Shared Clock and Reset Signal	SS9	RESET_CLOCK
Avoid Gated Clocks	SS10	CKGT
Avoid Internally Generated Clocks	SS11	IGCK
Avoid Internally Generated Resets	SS12	RSTMOD, RST_INTERNAL
Avoid Mixed Polarity Reset	SS13	RESET_POLARITY
Avoid Unresettable Registers	SS14	RST, NO_SR
Avoid Asynchronous Reset Release	SS15	SYNCH_DEASSERT_RST
Avoid Initialization Assignments	SS16	NO_INITIAL
Avoid Undriven and Unused Logic	SS17	UNCONNECTED_UNDRIVEN_UNUSED_NET/PIN,
Ensure Register Controllability	SS18	
Avoid Snake Paths	SS19	COMB_LEVELS, Path Analysis
Ensure Nesting Limits	SS20	ITE_DEPTH
Ensure Consistent Vector Order	SS21	RANGE_CHECK

ADIUVO
ENGINEERING AND TRAINING, LTD.

Design Review

Coding Rule Name	Rule Number	BPS Check
Use Statement Labels	DR1	COMMENT_END_STMNTS, COMMENT_NET_DEC, COMMENT_PORT_DEC
Avoid Mixed Case Naming for Differentiation	DR2	
Ensure Unique Name Spaces	DR3	
Use Separate Declaration Style	DR4	MULT_PORTS
Use Separate Statement Style	DR5	SLCC
Ensure Consistent Indentation	DR6	CHECK_INDENTATION
Avoid Using Tabs	DR7	NO_TABS
Avoid Large Design Files	DR8	MAX_LINES_PER_MODULE
Ensure Consistent Signal Names Across Hierarchy	DR9	
Ensure Consistent File Header	DR10	FILE_HEADER
Ensure Sufficient Comment Density	DR11	Use SLOC report
Ensure Proper Placement of Comments	DR12	
Ensure Company Specific Naming Standards	DR13	NAMING_RULES

Clocks and Reset

ADIUVO
ENGINEERING AND TRAINING, LTD.

Clocks

Very simple approach which is easily often to get wrong

- Consider Oscillator start up time in the design analysis
- Carefully Plan the use of Clock Global and Local Clock resources
- Be very careful about crossing clock domains
- Ensure Asynchronous inputs are synchronised to the clock domain

Clocking Example

NASA WideField Infrared Explorer

“The control logic design utilized a synchronous reset to force the logic into a safe state. However, the start-up time of the crystal clock oscillator was not taken into consideration, leaving the circuit in a non-deterministic state for a significant period of time. Likewise, the start-up characteristics of the FPGA were not considered”

ADIUVO
ENGINEERING AND TRAINING, LTD.

Clock Quality in Xilinx FPGAs

Clock monitoring capability of Clocking Wizard

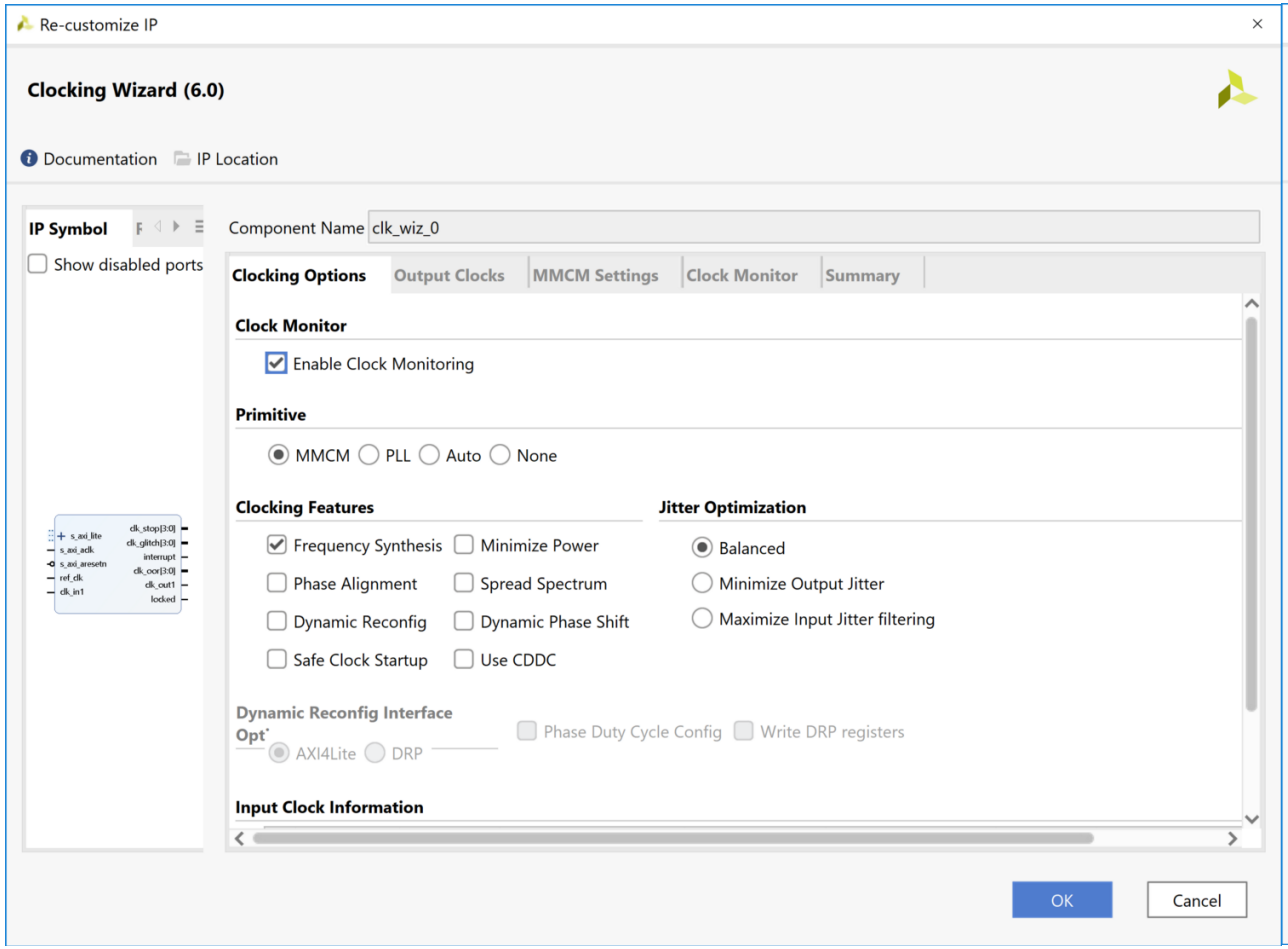
Using clock monitoring, we are able to observe:

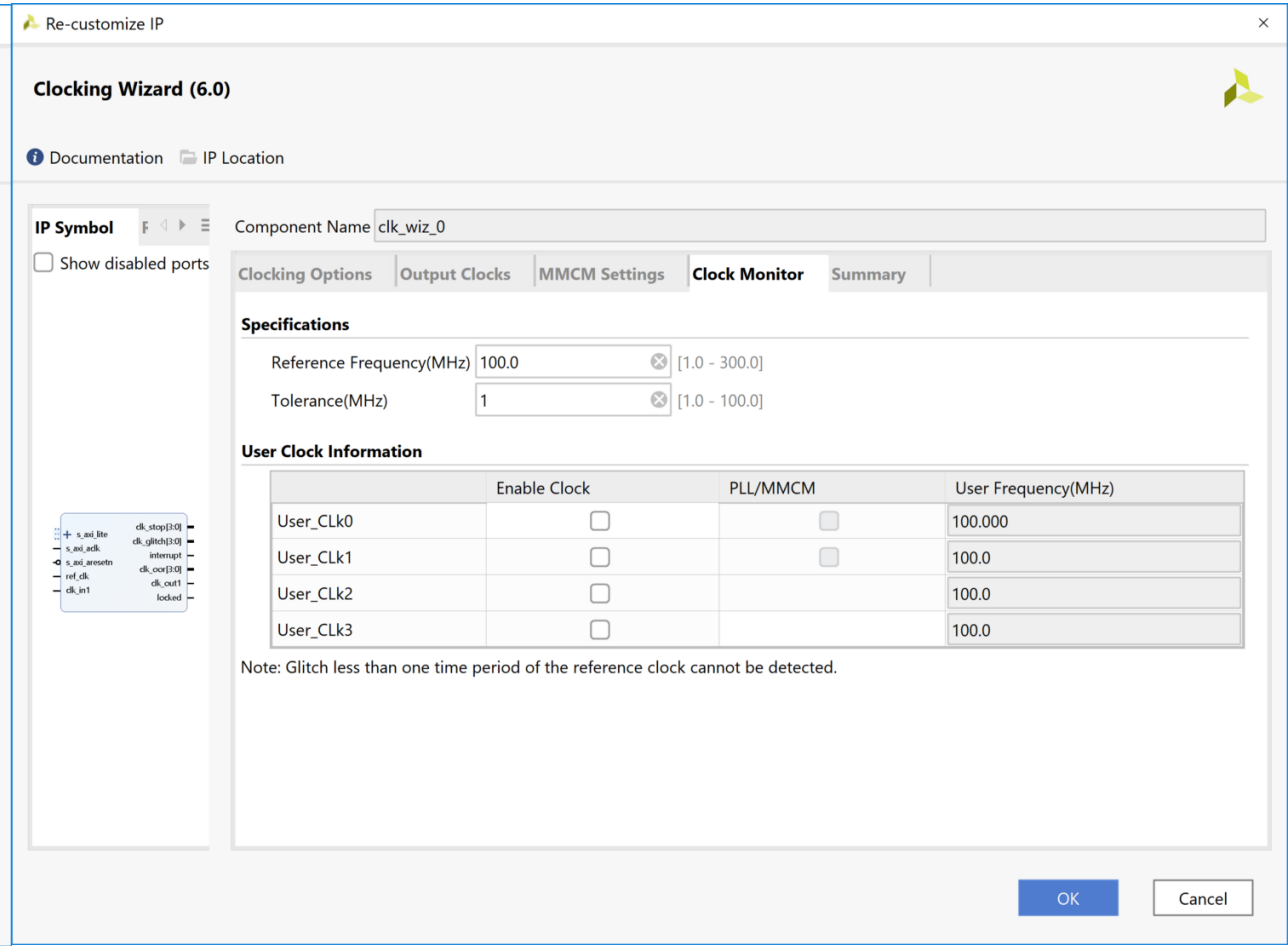
- If a monitored clock stops
- If a monitored clock glitches
- If a monitored clocks frequency goes out of range

Clock Wizard allows us to be able to monitor up to four clocks.

The reference clock is the clock used to monitor the four clocks, it is assumed the reference clock is stable and error free

Clock Monitoring





Clock Monitoring

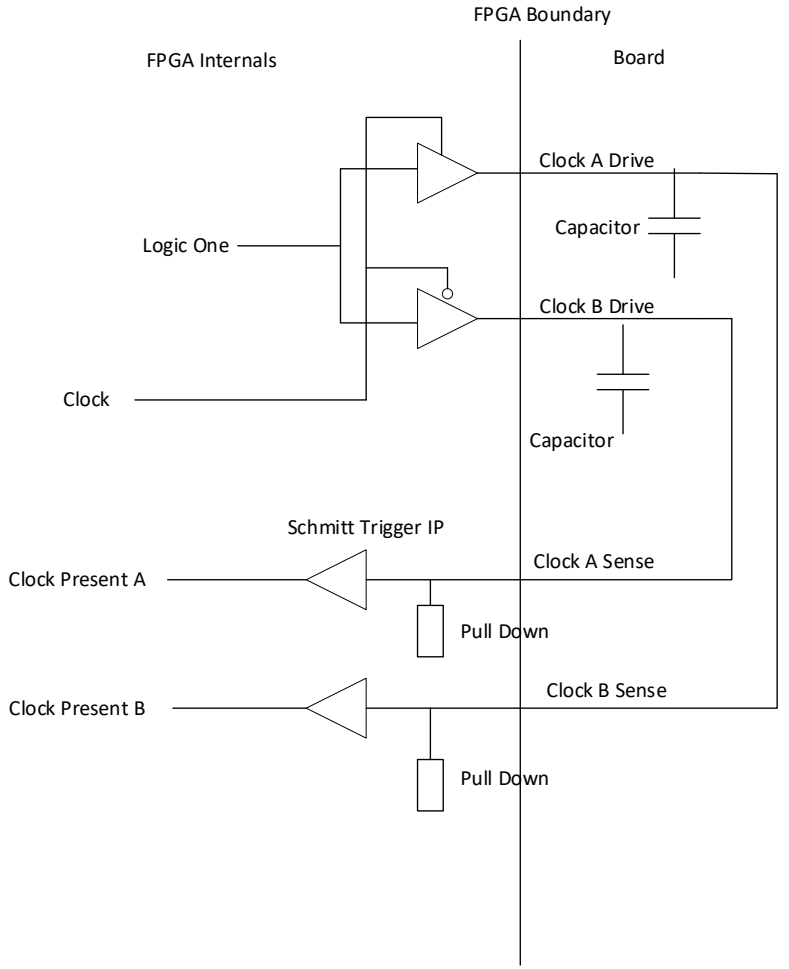
Uses Tristate buffer and charging capacitor on the board and fed back into the FPGA to detect a clocks presence.

Capacitor charged when Tristate buffer enabled every half clk cycle.

This achieves steady state in logic ‘1’ on input indicating clock present

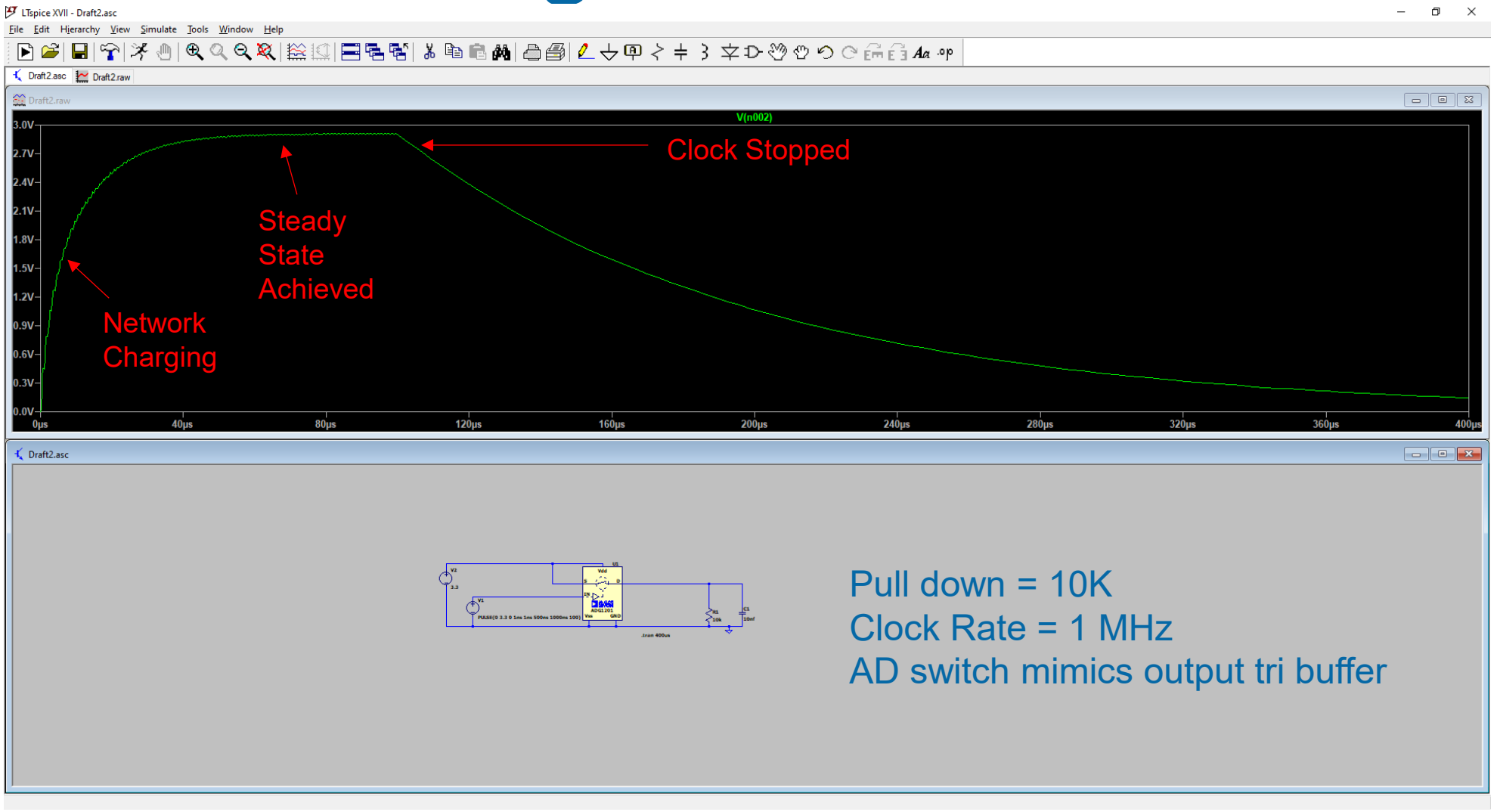
If clock stops the Capacitor discharges and clock present signal goes to a logic low.

Use Schmitt buffer internally or externally



If FPGA does not Support Schmitt Trigger input, these can be located discrete on the board

Clock Monitoring



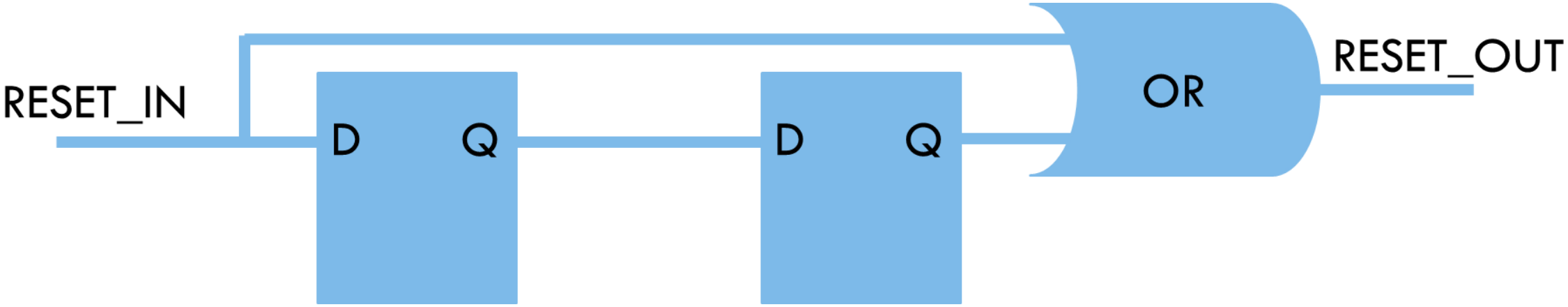
Resets

Do not rely upon the default value at power up for critical aspects

Reset needs to be Asynchronous Assertion – See clock start up time

Synchronous De Assertion – Prevents the risk of meta-stability when cleared

Ensure Outputs are reset to de assert peripherals especially bi directional busses



Reset

Ensure reset assertion time is longer than the oscillator start up time

If RC network used for reset generation

- Consider the slow rising edge and relation to logic levels.
- Use Schmidt trigger to ensure rise / fall time compliant edges

Do not use feedback from the FPGA to indicate when the device is powered up

TMR or Not TMR Clocks

Reducing Glitches – clock insertion into buffers

Clock Domain Crossing

ADIUVO
ENGINEERING AND TRAINING, LTD.

Clock Domain Crossing

Ideal solution uses one clock and the entire design is synchronous.

Modern devices have multiple clocks to address different clock domains e.g. ADC / DAC clocks, source synchronous interfaces.

Brings with it the need to transfer data, and signals safely and reliability between the clock domains

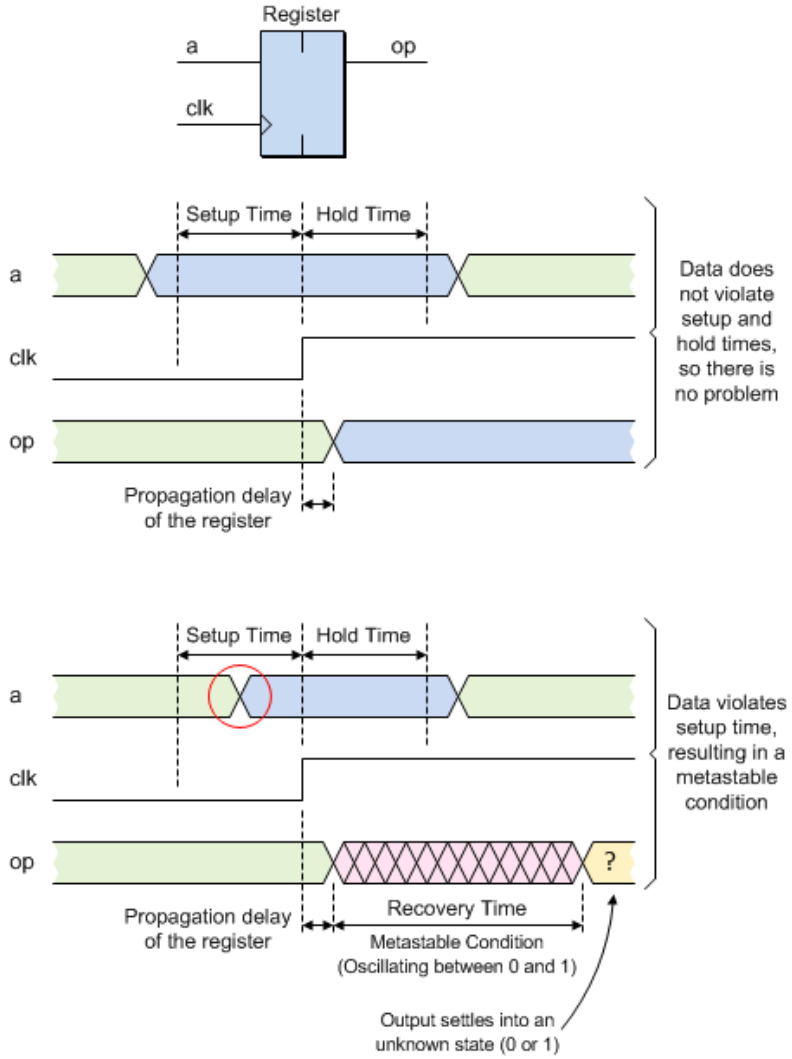
Metastability

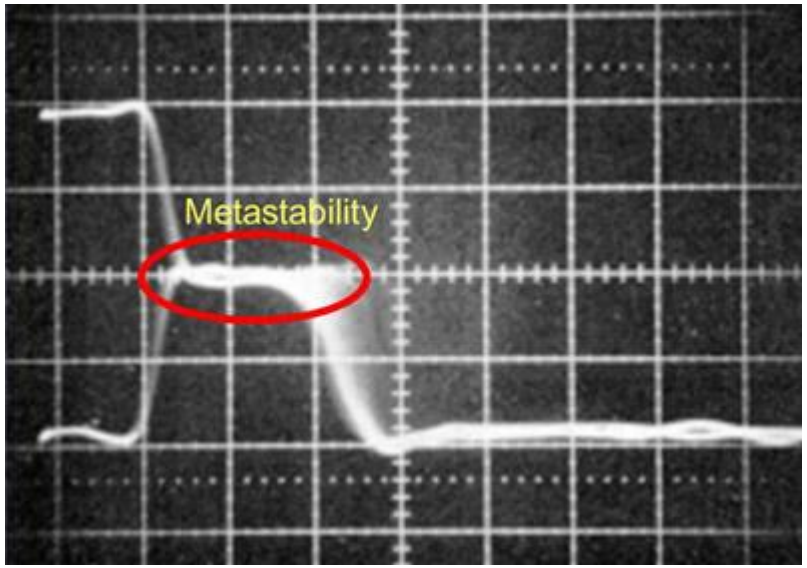
One issue which can arise with incorrect domain crossing is metastability

This can lead to corruption of data or incorrect behaviour

Occurs when a flip flops set up or hold time is violated

Metastability





Clock Domain Crossing

Several Techniques which can be used depending upon what needs to be transferred

- Two stage synchroniser – Ideal for single bit data
- Grey Code Synchroniser – Encodes data bus in grey code and transfer between domains – Ideal for counters as input to be converted to grey code can only decrement / increment by one from previous value
- Hand shake synchroniser – Transfers data bus between two clock domains using handshake signals
- Pulse synchroniser – transfer pulse from one clock domain to another
- Asynchronous FIFO – transfers data from one domain to another, useful for high throughput / burst transfers

Clock Domain Crossing

To support reset functionality across clock domains we may need the following synchronisers structures.

- Asynchronous Reset Synchroniser – enables asynchronous assertion and synchronous de-assertion.
- Synchronous Reset Synchroniser – synchronises a synchronous reset to another clock domain.

How many synchronisers do I need ?

Standard two stage synchroniser assumes the first flip flop, will recover from metastability before the signal is clocked into the second flip flop.

How do we know this is the case and what if we need more stages

$$MTBF = \frac{e^{t_r/\tau}}{T_o \times f \times a} \times k\left(\frac{e^{t_r/\tau}}{T_o \times f}\right)$$

t_r = The speed at which the event is being resolved (clock period – setup time)

T_o = Time window during which the register is susceptible to going metastable.

τ = Settling time

a = Frequency of the asynchronous signal.

f = Frequency of the clock (as stated in Part 1, this should be faster than "a")

k = The number of additional stages (so k=1 for a two-stage synchroniser)

Note that τ and T_o are both process constants.

CDC in Xilinx

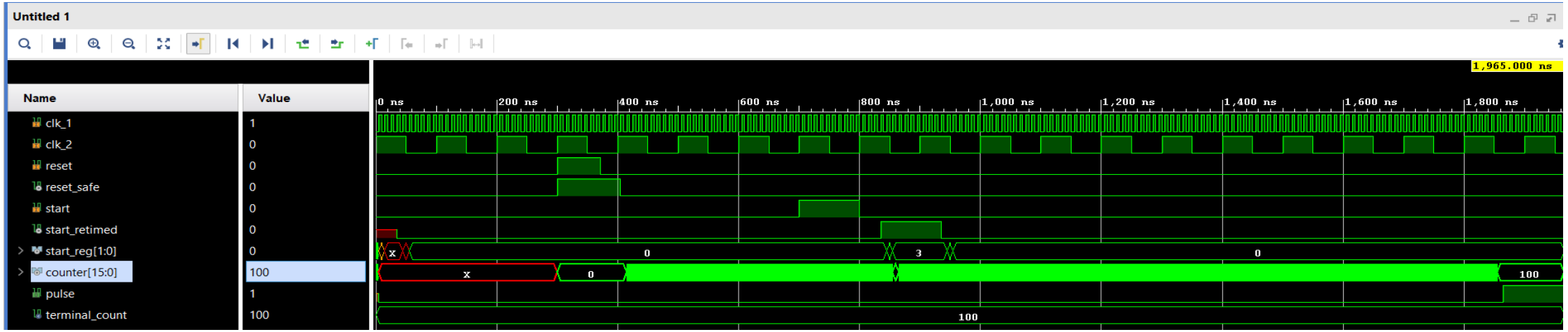
Xilinx Parameterised Macros (XPM) – provide CDC structures

Use registers optimised for CDC in the fabric

- » Registers located close together and have small set up and hold windows

Described within UG953 Libraries Guide

Described within UG 974 UltraScale Architecture Libraries



CDC Design Analysis tools

Detecting all CDC issues can be a challenge in large designs

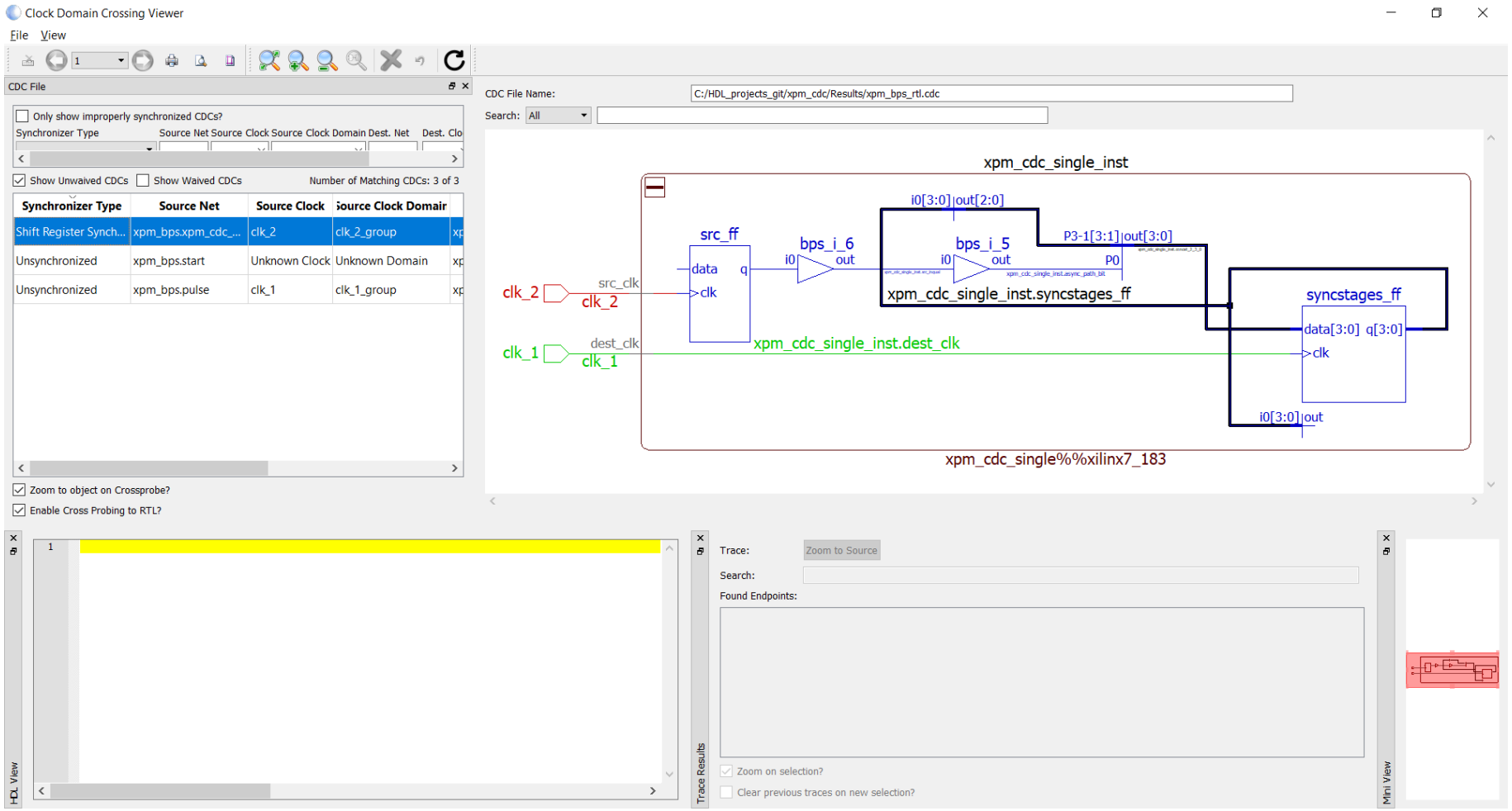
- Are all IP IO on the correct clock domain
- Very easy to associate signal with wrong domain e.g. FIFO empty and WR clock

CDC issues can be very difficult to find changing on each start up and may be intermittent

Can be hard to find in simulation – Timing simulation required, takes a long time simulate the system

Static analysis tools are better suited to find the CDC issues.

Blue Pearl Visual Verification Suite – CDC



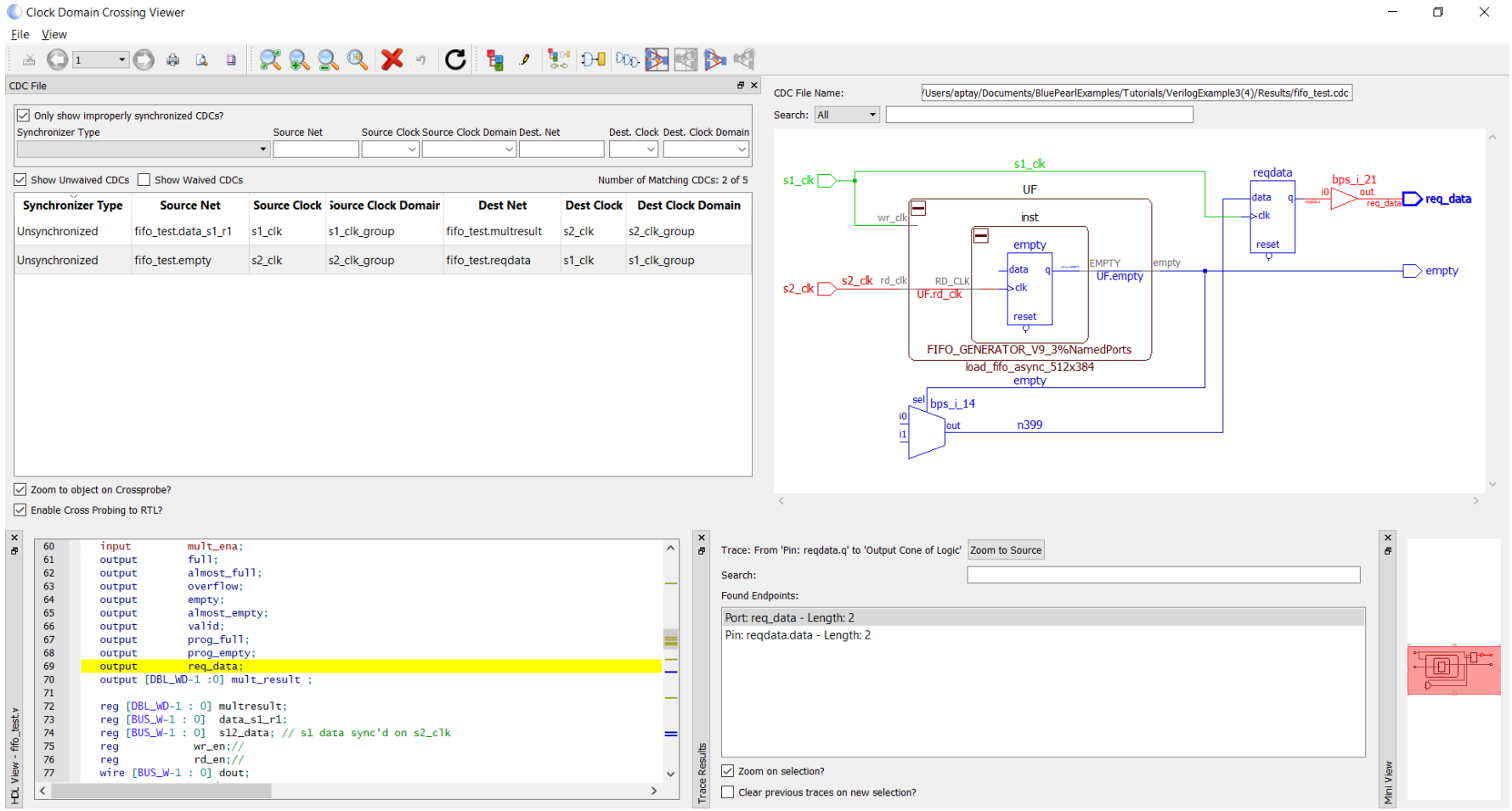
Blue Pearl FIFO Example

```
always @ (posedge s1_clk or negedge reset_n )
  if (~reset_n)
    begin
   wr_en    <= 1'b0;
   reqdata <= 1'b0;
    end
  else
    begin
   sync_clk_r1 <= sync_clk;
   data_s1_r1    <= data_s1;
   wr_en  <= s1_ena;

   if (empty)
     reqdata <= 1'b1;
   else
     reqdata <= reqdata;
    end

assign req_data = reqdata;

always @ (posedge s2_clk or negedge reset2_n )
  if (~reset2_n)
    begin
   rd_en  <= 1'b0;
   sync_clk_r1_s2 <= 1'b0; // Added to remove value hold on reset.
   s2_mux_sel     <= 1'b0; // Added to remove value hold on reset.
   s12_data       <= 1'b0; // Added to remove value hold on reset
    end
  else
    begin
   rd_en          <= s2_ena;
   sync_clk_r1_s2 <= sync_clk_r1;
   s2_mux_sel     <= sync_clk_r1_s2;
   s12_data       <= mux_reg_1_2;
   // a_2    <= synch_a_2;
    end
```

```
module fifo_test(
          reset_n,
          reset2_n,
          s1_clk,  // input slower clock
          s2_clk,  // input slower clock
          sync_clk,  // input rate to sync s1 to s2
          data_s1,  // input data1
          s1_ena,  // input data on s2 valid
          s2_ena,  // input data on s2 valid
          mult_ena,  // input enable the mult stage
          full,
          almost_full,
          overflow,
          empty,
          almost_empty,
          valid,
          prog_full,
          prog_empty,
          req_data,  // Output slower clock
           mult_result // output slower clock
          );
```

Can we detect and error from a simple code review ?

Blue Pearl FIFO Example




```
60    input         mult_ena;
61    output        full;
62    output        almost_full;
63    output        overflow;
64    output        empty;
65    output        almost_empty;
66    output        valid;
67    output        prog_full;
68    output        prog_empty;
69    output        req_data;
70    output [DBL_WD-1 :0] mult_result ;
71
72    reg [DBL_WD-1 : 0] multresult;
73    reg [BUS_W-1 : 0]  data_s1_r1;
74    reg [BUS_W-1 : 0]  s12_data; // s1 data sync'd on s2_clk
75    reg             wr_en;//
76    reg             rd_en;//
77    wire [BUS_W-1 : 0] dout;
```

IO Considerations

ADIUVO
ENGINEERING AND TRAINING, LTD.

IO Start Up

There may be a start up time associated with the FPGA and IO following the truth table.

Ensure the system design addresses this to prevent inadvertent IO behaviour at start up

The design should never rely on the default value of the device output during power-up

IO Handling

Unused pins

- Leave unconnected on the board, internally to the FPGA design configure them as output driving low.
- Do not connect to unused pins to the power or ground.

Consider “special” pins for example

- No Connect Pins – leave unconnected.
- JTAG
  - TDI, TMS, TCK should be pulled down to ground via a resistor.
  - On flight devices if provided TRST should be tied to ground.
- PLL inputs if not used should be tied off in accordance with the data sheet
- Be guided by the data sheet
  - But ensure you have the latest and any errata’s, product change notes / warnings

IO Busses

Occurs when different devices try to drive bus with different logic values

- » Can result in catastrophic failures

Consider causes of contention

- » During power up
- » Reset

During reset drive busses to high impedance

Use asynchronous reset assertion

- » Ensures there is no contention – can occur in synchronous reset between reset assertion and synchronous output

Might want to consider power effects of leaving tristate long term

Radiation Effects

ADIUVO
ENGINEERING AND TRAINING, LTD.

FPGA Mitigation Techniques

Within an FPGA the engineer must consider several factors

- Parametric changes in timing as the device gets slower
- Single Event Upsets – This is what people generally think about when they consider single event effects, a SEU flips the state of a register or memory. This could occur within the Block or Distributed RAM, User Flip Flops or FPGA configuration memory. These events can be corrected using detection and error correction schemes if necessary or if on data paths e.g. filters they will clear on the next sample.
- Single Event Functional Interrupt – This is similar to a SEU in that a bit is flipped however normal operation of the device can only be recovered by power cycling the device. In a FPGA the most common type of SEFI is power on reset receiving a SEE this will result in the FPGA requiring reconfiguration.
- Multi Bit Upset – This is when a SEE effects more than one register or memory element, many mitigation techniques can detect and correct MBU's

FPGA Mitigation Techniques

FPGA are mainly affected by TID as opposed to TNID, TID will gradually change the parameters of the FPGA such as timing performance and power required.

Usually the development tools include the data to include these parametric changes.

For example timing analysis should take into account the effects of the parametric changes.

This is where your radiation hardness assurance comes into play ensuring the component selected are suitable for the mission.

However many companies also ensure derating is included within the timing constraints. E.g. if you are designing to operate at 50 MHz your constraints should be set for 55 MHz providing a margin

FPGA Mitigation Techniques

The major worry of the FPGA designer is the single event upset.

Depending upon where in the device this happens the effect may or may not be noticed, for example

- Flipping the least significant bit of a ADC digitised output, this will quickly clear as other samples are clocked through and will have a similar effect as a rounding error when further processed.

- Flipping a state bit on a state machine and causing the state machine to enter a un used sate and hence locking up the machine. This could result in catastrophic loss of the mission.

But it is not just state machines, counter, RAMs any stored value can be flipped

FPGA Module Considerations

ADIUVO
ENGINEERING AND TRAINING, LTD.

FPGA Module Considerations

Design FPGA as a number of modules

- » If designing several identify the common IP blocks so you can maximise reuse

Where possible use generics to customise between similar modules

Communication between modules – consider what happens if modules experience failures on the input e.g.

- » Stuck at 1
- » Stuck at 0

Analyse every model for this

Active signals e.g. should use edges not levels to prevent a stuck signal being incorrectly translated as a “go” signal

If levels are un-avoidable the use a dual signal vector approach

- » If one of them becomes stuck at a level then it will not function
- » Need to be careful with this one to prevent optimisation commoning the 2 signals

FPGA Module

Ideal world – Document each module created

- Module Description – Inputs, Outputs, reset states, behaviour under fault conditions, algorithm description and behaviour – quantisation etc.
- Test Plan – Written by independent person to module developer. Outlines the test cases to be applied to the module, pass and fail conditions, corner cases and boundary positions tested etc.
- Use a Continuous Integration flow – Can also use constrained random testing on the module

State Machines

ADIUVO
ENGINEERING AND TRAINING, LTD.

ADIUVO
ENGINEERING AND TRAINING, LTD.

State Machine

State machines are logical constructs that transition between a finite number of states. They are often at the heart of many of our FPGA designs.

However, there are several issues which can arise if not designed correctly

Terminal states – State from which once entered there is no path to leave

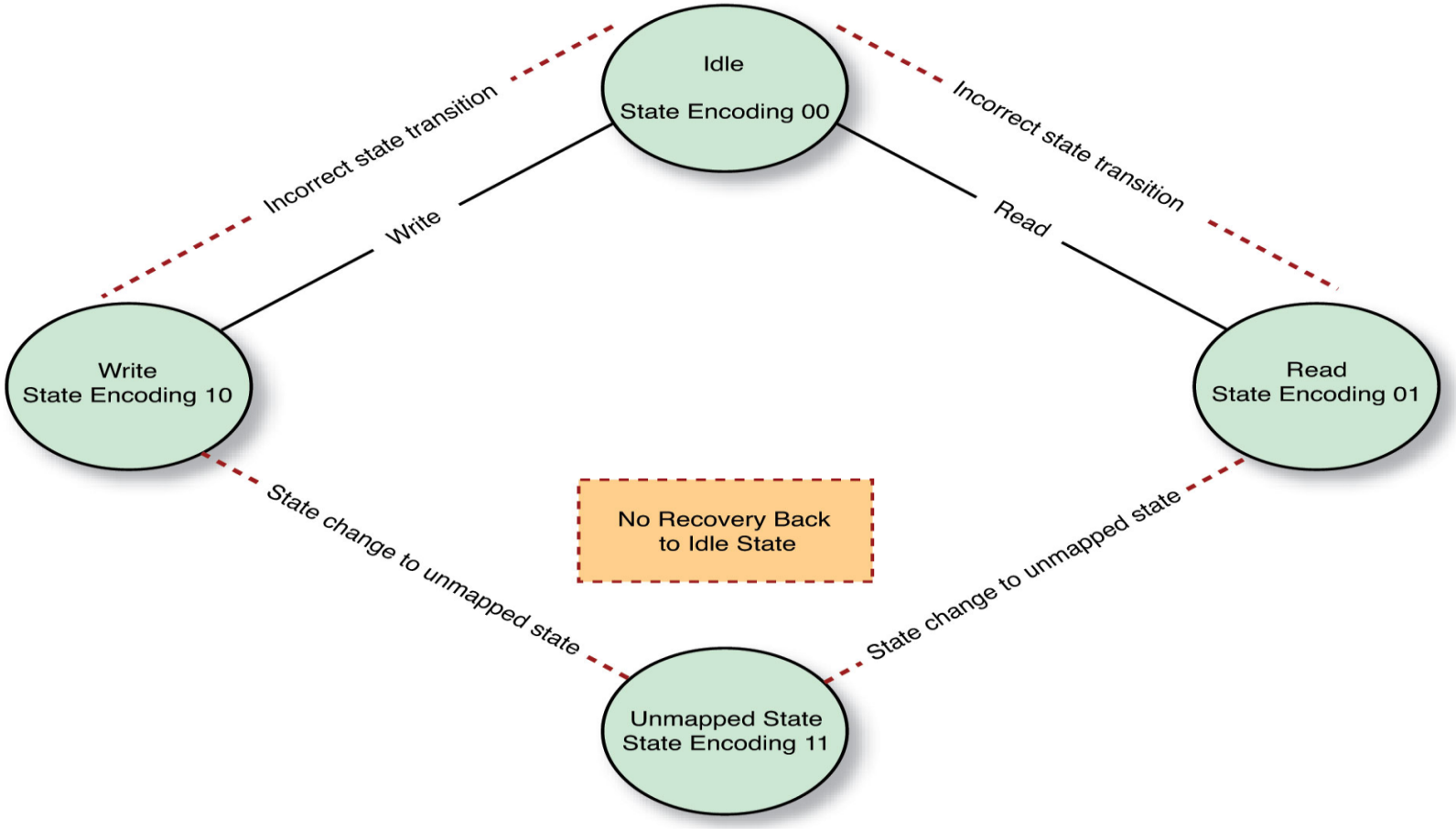
Unused states – State which are unsed in the design, transition in these states also results in a terminal state.

Safe state machine – Adds in additional logic, which could be effected and cause un-expected behaviour

Encoding – Is the state machine encoding correct for the environmental challenges

State Machine

The Unmapped State Machine



State Machine

ADIUVO
ENGINEERING AND TRAINING, LTD.

How can we harden state machines?

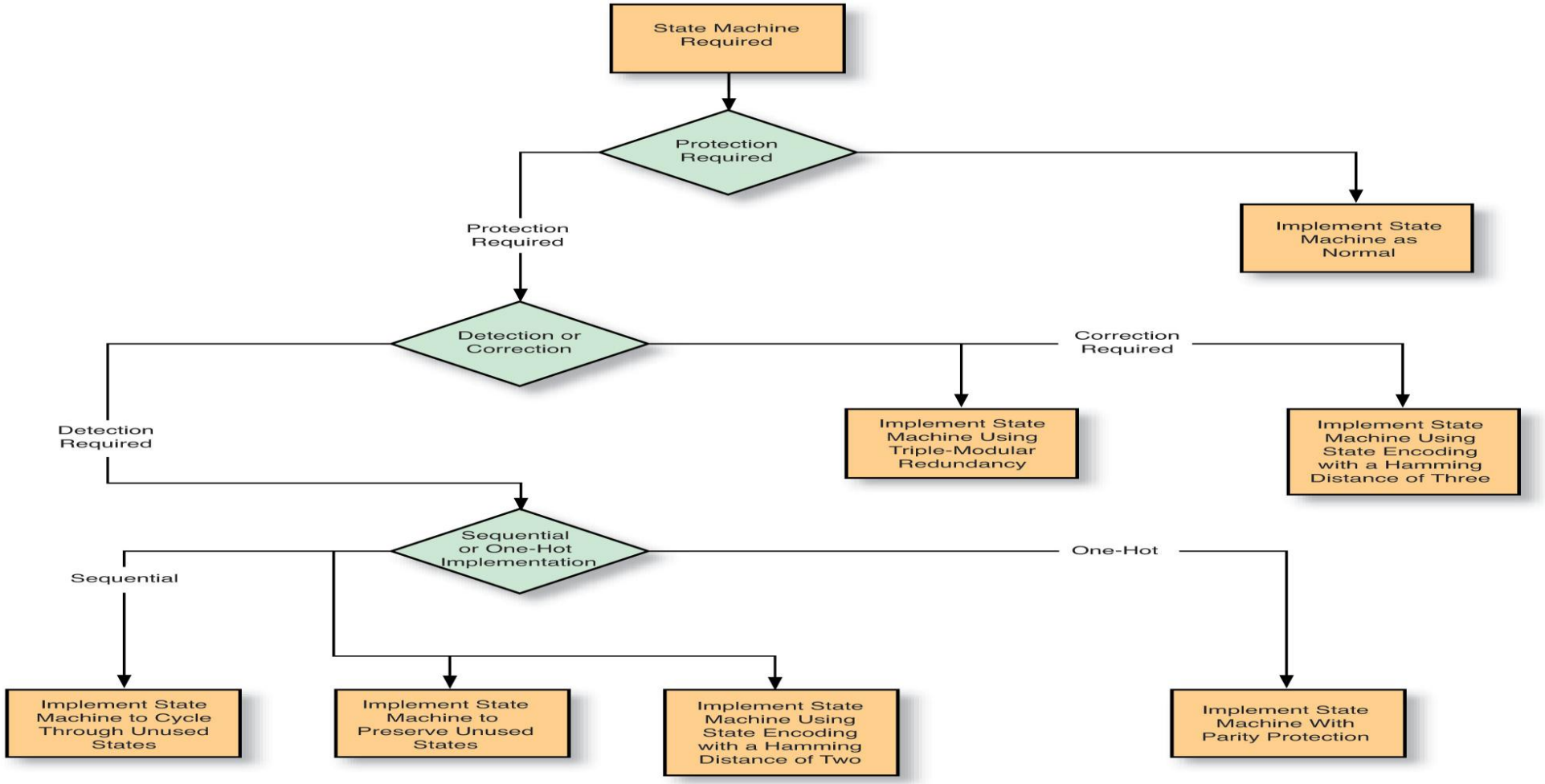
Triple Modular Redundancy – Instantiate three implementations of the state machine and majority vote on the output. This obviously has an area penalty and can reduce the operating frequency. There are tools like XMR from Xilinx which will assist with inserting TMR.

TMR also requires that the instantiations are separated physically from each other to ensure more than one instantiation cannot be effected by a SEU and create a MBU.

What other options are available ?

State Machine

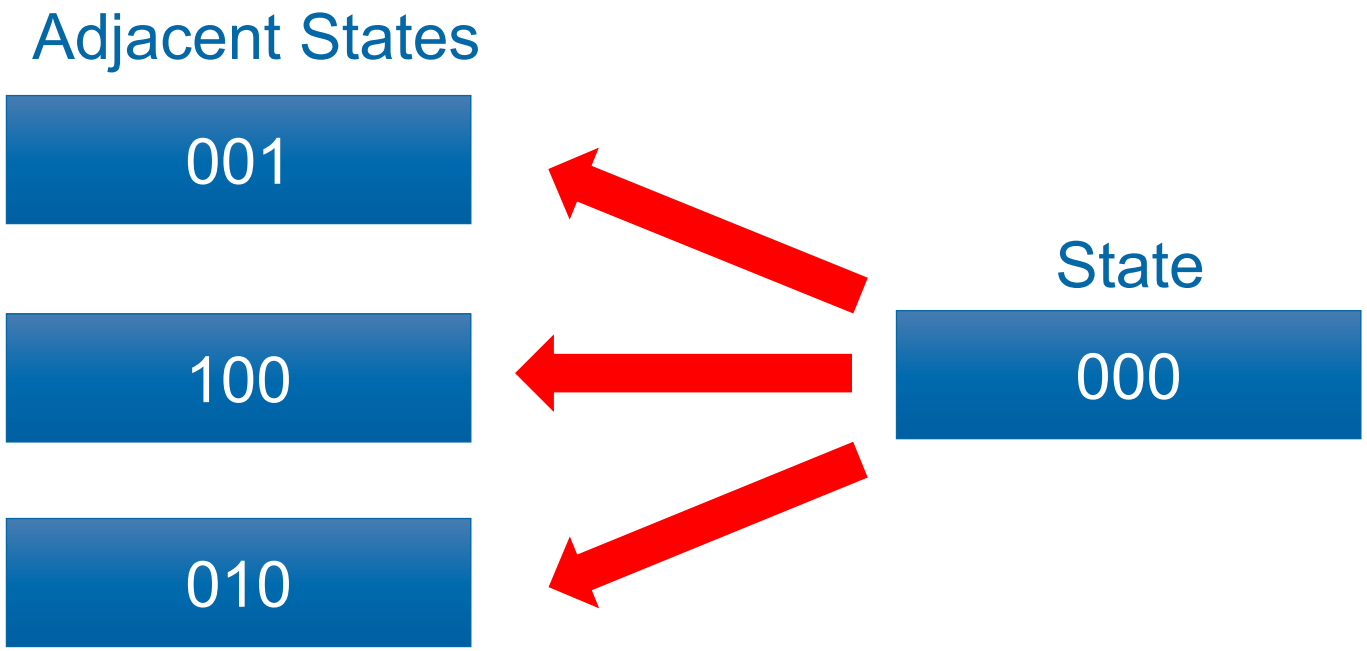
State machine choices other than TMR



State Machines

What does a Hamming code of three mean?

For each state with n number of bits there are also n adjacent states



ADIUVO
ENGINEERING AND TRAINING, LTD.

State Machines

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
PACKAGE sc_fsm IS
CONSTANT state_1 : std_logic_vector(5 DOWNTO 0) := "000000";
CONSTANT state_2 : std_logic_vector(5 DOWNTO 0) := "000111";
CONSTANT state_3 : std_logic_vector(5 DOWNTO 0) := "011001";
CONSTANT state_4 : std_logic_vector(5 DOWNTO 0) := "011110";
CONSTANT state_5 : std_logic_vector(5 DOWNTO 0) := "101010";
CONSTANT state_6 : std_logic_vector(5 DOWNTO 0) := "101101";
CONSTANT state_7 : std_logic_vector(5 DOWNTO 0) := "110011";
CONSTANT state_8 : std_logic_vector(5 DOWNTO 0) := "110100";
CONSTANT state_1_1 : std_logic_vector(5 DOWNTO 0) := "000001";
CONSTANT state_1_2 : std_logic_vector(5 DOWNTO 0) := "000010";
CONSTANT state_1_3 : std_logic_vector(5 DOWNTO 0) := "000100";
CONSTANT state_1_4 : std_logic_vector(5 DOWNTO 0) := "001000";
CONSTANT state_1_5 : std_logic_vector(5 DOWNTO 0) := "010000";
CONSTANT state_1_6 : std_logic_vector(5 DOWNTO 0) := "100000";
```

What it looks like in VHDL, when we automatically generate a package to support Hamming three states.

State Machines

What it looks like in RTL – But there are easier ways to implement

```
CASE current_state IS
  WHEN state_1 | state_1_1 | state_1_2 | state_1_3 | state_1_4 | state_1_5 | state_1_6 =>
    IF re_ip = "01" THEN
      current_state <= state_2;
    ELSIF re_ip ="10" THEN
      current_state <= state_3;
    ELSE
      current_state <= state_1;
    END IF;
```

Hamming Three – Overhead

One Hot Implementation
3 DFF used for state encoding
64 DFF used for two 32-bit registers
LUT next state logic

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	3	53200	0.01
FF	67	106400	0.06
IO	68	200	34.00
BUFG	1	32	3.13

Hamming Three Implementation
5 DFF used for state encoding up 66%
64 DFF used for two 32-bit registers
LUT next state logic up 100%

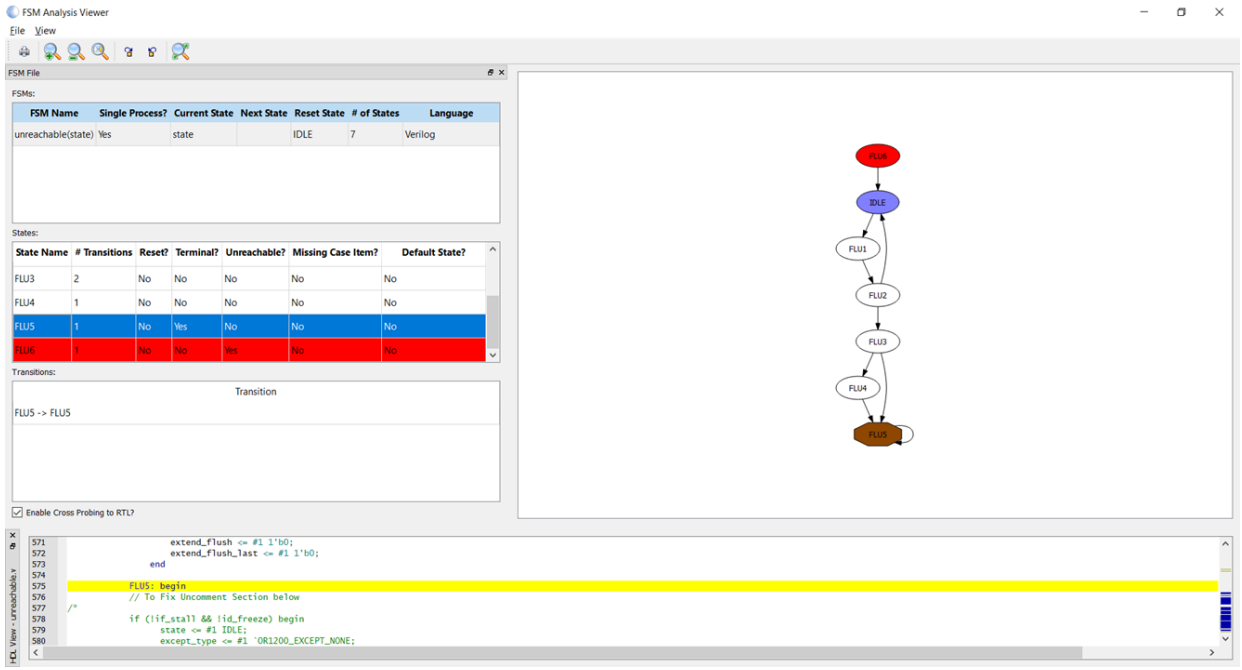
Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	6	53200	0.01
FF	69	106400	0.06
IO	68	200	34.00
BUFG	1	32	3.13

On a small implementation, overhead impact is minimal – but consider the impact when deployed large scale across a device.

State Machine Analysis

- Complex state machines can be hard to analyse
- Are all states mapped
- Is there terminal states
- Static FSM Analysis is important in providing understanding & correct behaviour
- Can be used for documentation as well
- Blue Pearl VVS ideal



State Machine – One or Two Process

- Two-process state machine implementation the main functionality will be contained in a combinatorial process. If we fail to fully define all the combinatorial conditions within this process we will implement latches during synthesis. The combinatorial process may also generate glitches on its outputs as the state and inputs change.
- By contrast a one process state machine enables much easier use of conditional definition and removes the ability for latches to be created. While preventing glitches as all outputs signals are registered

State Machine Tips

Decouple functionality, only allow single bit input and outputs to your state machine.

For example, as shown in the code snippet below

```
when wait_cnt =>
   if cnt >= std_logic_vector(to_unsigned(128,16)) then
      current_state <= read_fifo;
      cnt <= (others =>'0');
   else
      cnt <= cnt + 1;
   end if;
```

Implementing the state machine in this manner includes additional logic within the state machine for both the comparator and the counter. This will impact the performance of the state machine within the implemented FPGA as it requires more resources and consequently more routing.

State Machine Tips

A better method is to use an external process for the counter or other external functions which pass in a single control signal. This leaves the bulk of the logic decoupled from the state machine.

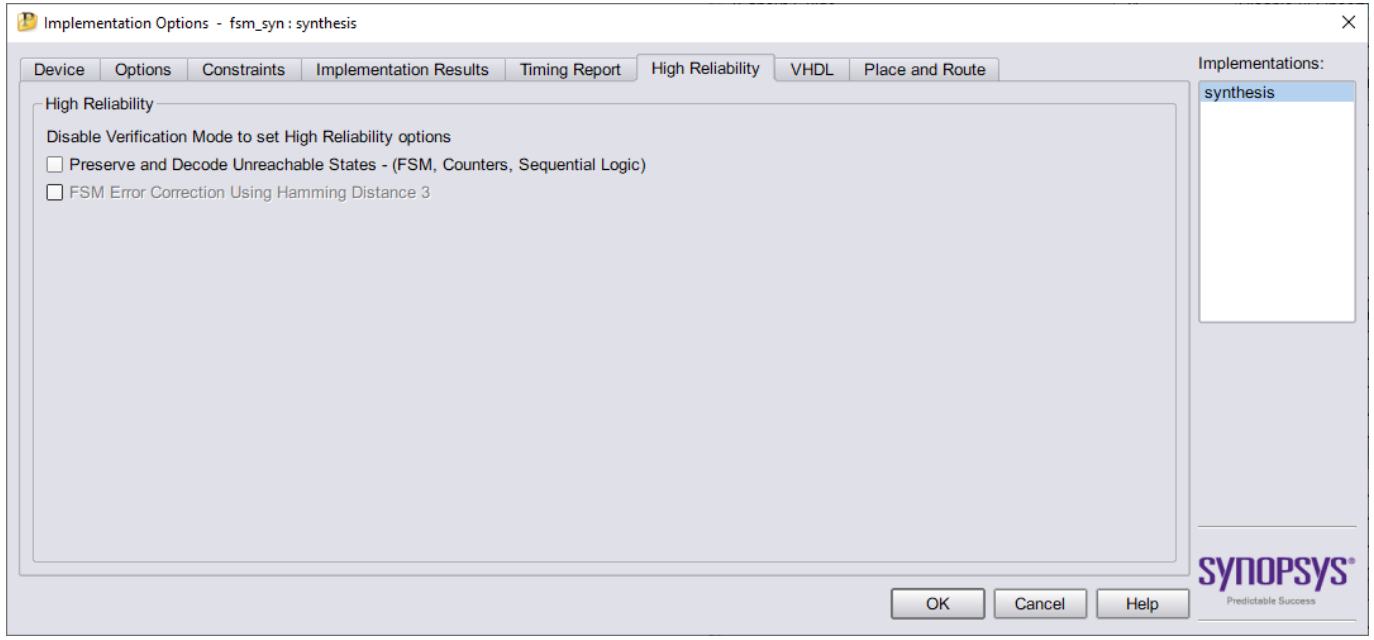
```
when wait_cnt =>
    if cnt_terminal = '1' then
        current_state <= read_fifo;
        cnt_reset <= '1';
    end if;
```

Synplify State Machine Synthesis

Global – preserving of unreachable states – Via High Reliability tab

Individual FSM – Using attributes in source code

Critically need to define – when other state



Synplify Pro Syn_safe_case

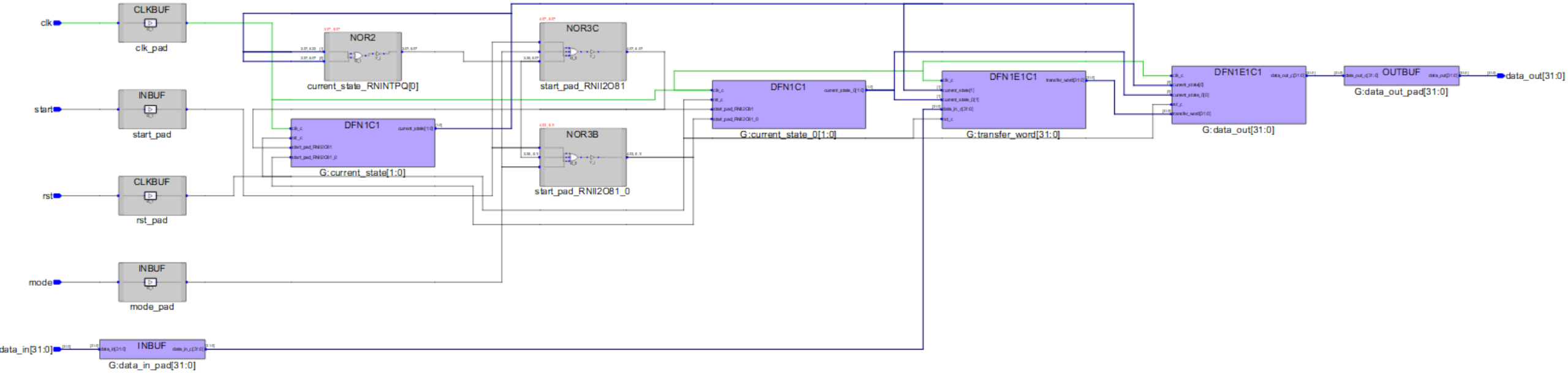
Not enabled

```
Target Part: RT3PE3000L_LGA484_STD
Report for cell fsm.rtl
  Core Cell usage:
              cell count     area count*area
               GND     1      0.0        0.0
              NOR2     1      1.0        1.0
             NOR3B     1      1.0        1.0
             NOR3C     1      1.0        1.0
               VCC     1      0.0        0.0


            DFN1C1     4      1.0        4.0
          DFN1E1C1    64      1.0       64.0
                     -----          ----------
             TOTAL    73                 71.0


  IO Cell usage:
              cell count
            CLKBUF     2
             INBUF    34
            OUTBUF    32
                     -----
             TOTAL    68


Core Cells         : 71 of 75264 (0%)
IO Cells           : 68

  RAM/ROM Usage Summary
Block Rams : 0 of 112 (0%)
```

Enabled

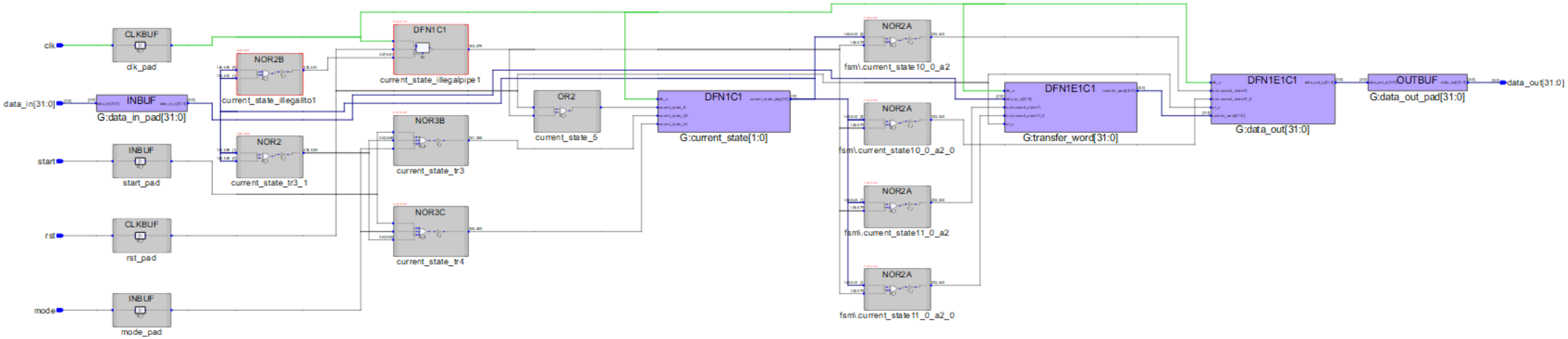
```
Target Part: RT3PE3000L_LGA484_STD
Report for cell fsm.rtl
  Core Cell usage:
              cell count     area count*area
               GND     1      0.0        0.0
              NOR2     1      1.0        1.0
             NOR2A     4      1.0        4.0
             NOR2B     1      1.0        1.0
             NOR3B     1      1.0        1.0
             NOR3C     1      1.0        1.0
               OR2     1      1.0        1.0
               VCC     1      0.0        0.0


            DFN1C1     3      1.0        3.0
          DFN1E1C1    64      1.0       64.0
                     -----          ----------
             TOTAL    78                 76.0


  IO Cell usage:
              cell count
            CLKBUF     2
             INBUF    34
            OUTBUF    32
                     -----
             TOTAL    68


Core Cells         : 76 of 75264 (0%)
IO Cells           : 68

  RAM/ROM Usage Summary
Block Rams : 0 of 112 (0%)
```

Technology View



Technology view – syn_safe_case

Note added elements



Xilinx Synthesis

Safe State Machines can be created in Vivado using the Attribute FSM_SAFE_STATE

- auto_safe_state – Hamming three encoded to ensure tolerance of SEU.
- reset_state – Hamming two encoding to force the state machine to its reset state on error.
- power_on_state – Hamming two encoding to force the state machine to its power-on state on error.
- default – Hamming two forces the state machine to the default / when others case define in the HDL.

Can be implemented in XDC or Source code

Counters

ADIUVO
ENGINEERING AND TRAINING, LTD.

Counters

Many Engineers look just for the terminal count of the counter for example

```
IF count = 9 THEN
```

However SEU can result in the value of the counter being >= 9

In such a case the counter would fail, and the effects may propagate

Instead use

```
IF count >= 9 THEN
```

In the worst case the timer will action early (which can be mitigated at FPGA level) rather than lock up the counter

Block RAM

ADIUVO
ENGINEERING AND TRAINING, LTD.

Block RAM

RAMS can be protected by error protection codes. Depending upon the device these are hard coded into the RAMS and transparent to the user or require user implementation.

Regardless of how they are implemented it is a good idea to use a scrubbing algorithm which will read back contents out of the memory periodically and ensure correction of any errors to prevent a build up of errors.

If these RAMS are being used to store the configuration of the system it is a good idea to have a copy of this configuration on the ground payload control system to ease recovery in the worse case

Block RAM

Consider Storage time static data has a longer window for corruption

Similarly data stored in FIFO etc is there for a shorter time and less likely to be corrupted.

If we are concerned about the storage we could

- Implement a structure which reads out data stored and refreshes it
- Requires use of EDAC code to correct for errors

Often called Scrubbing

Libero BRAM

ADIUVO
ENGINEERING AND TRAINING, LTD.

BRAM In Libero

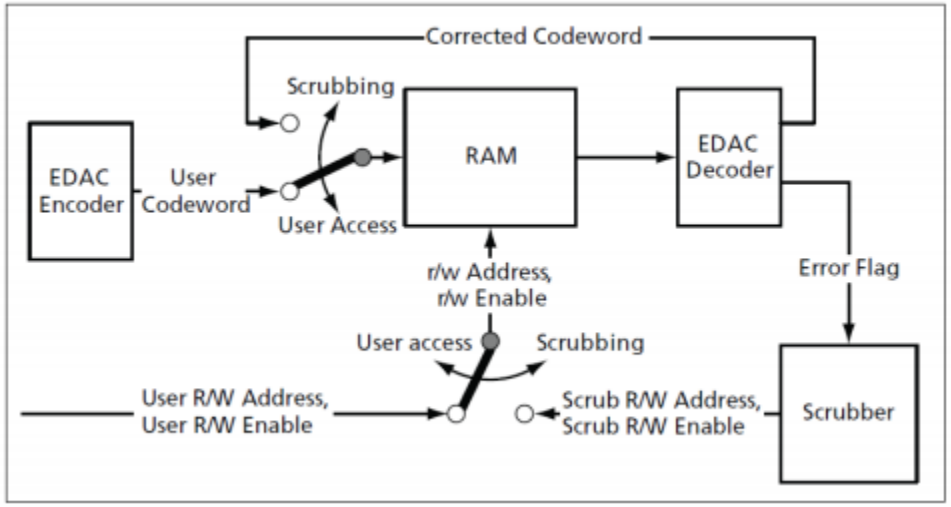
Libero provides an EDAC block which provides EDAC correction using Hamming Three. The EDAC block itself is capable of supporting the following widths

RAM Bit Width	Maximum User Data Bit Width
9	4
18	12
27	21
36	29
54	47
72	64

BRAM in Libero

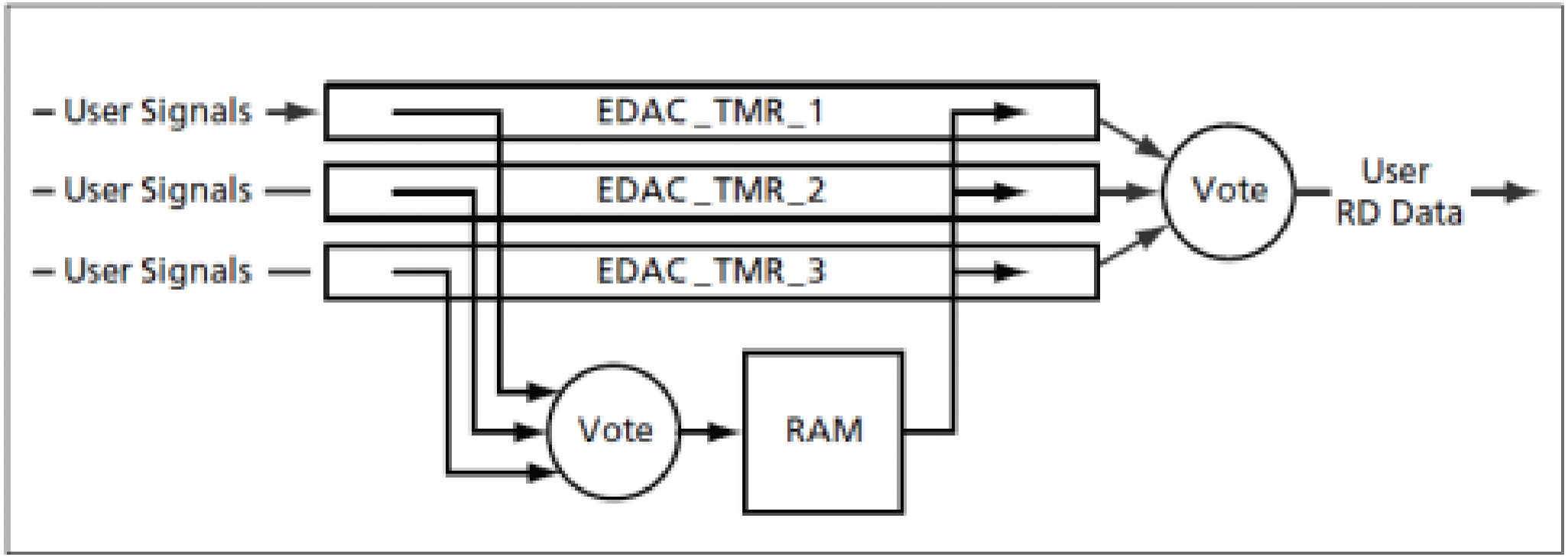
EDAC module is capable of

1) Generating the codeword
2) Decoding the code word to correct for a single error
3) Scrubbing the RAM to correct for errors accumulating in the RAM



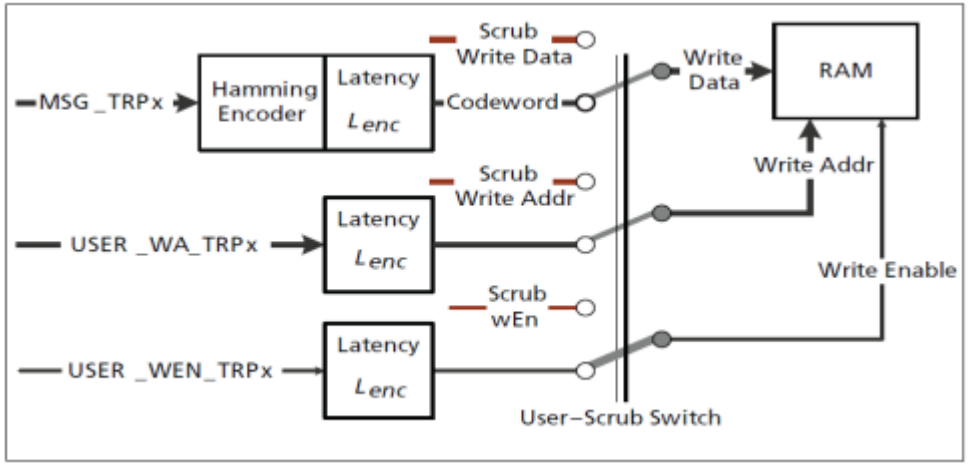
BRAM in Libero

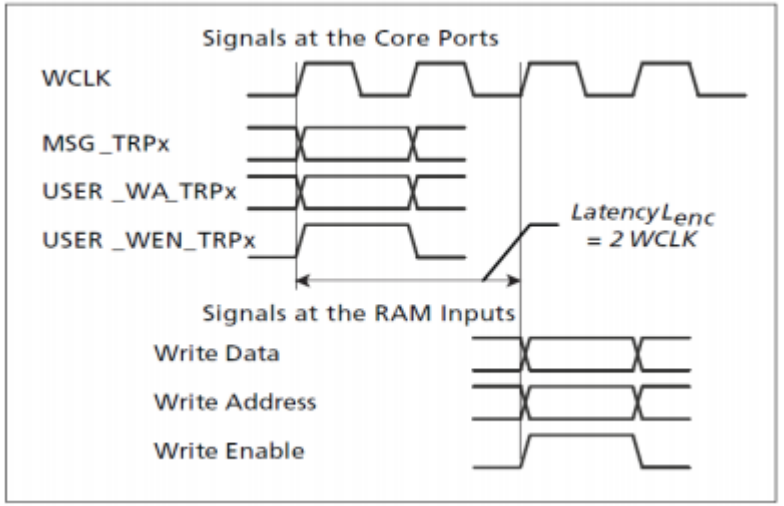
If desired local TMR can be implemented



BRAM In Libero

When writing data into the BRAM – the EDAC delays the control signals to take into account the latency



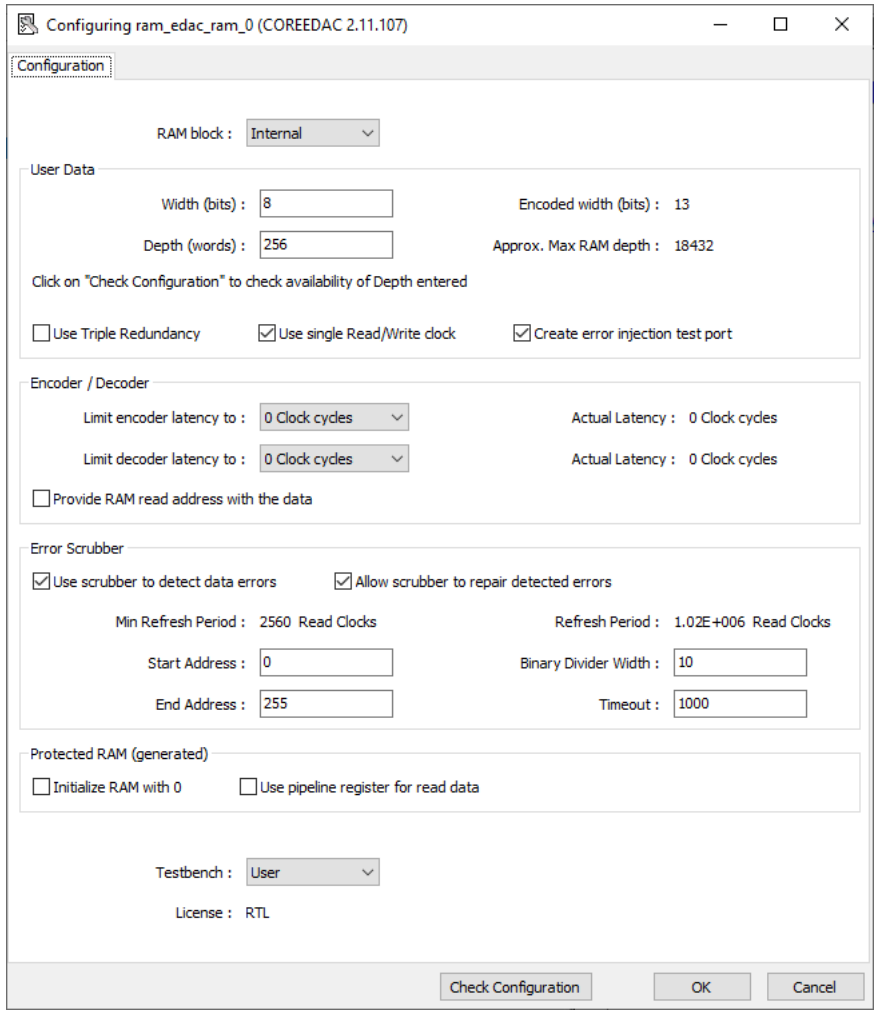


BRAM in Libero

Limitations to the BRAM EDAC

- Single port only - does not support true dual port access

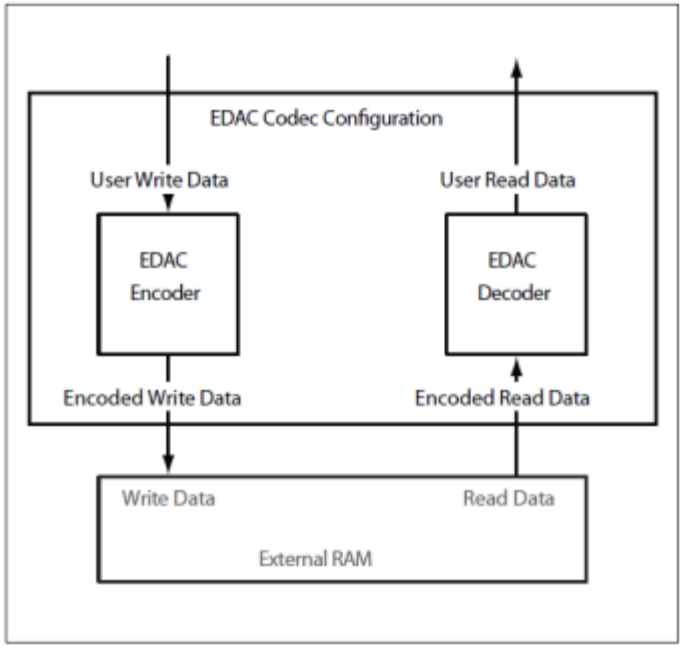
- But could configure as external and wrap Dual Port BRAM



EDAC In Libero

EDAC module is also capable of working with external RAMS e.g. external SRAM

This provides protection against corruption in larger memories.



Xilinx BRAM

ADIUVO
ENGINEERING AND TRAINING, LTD.

BRAM in Xilinx

Inherent support for ECC

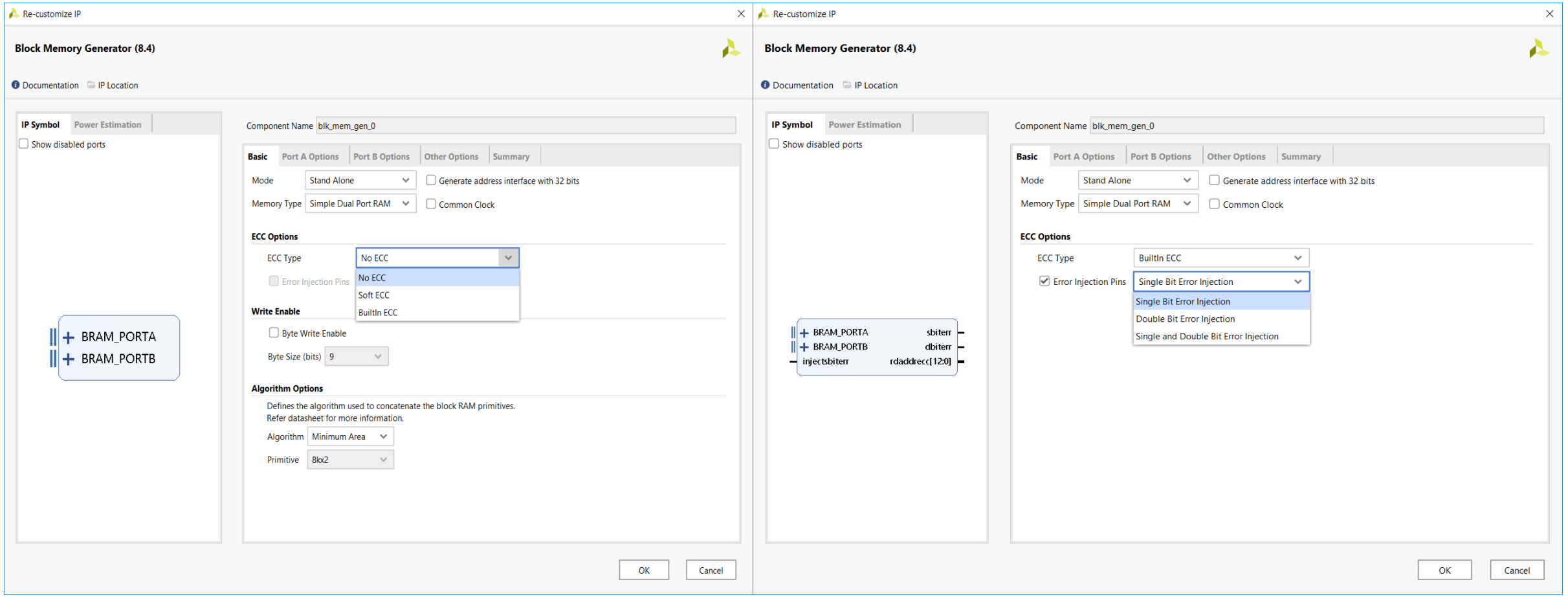
Cannot initialise BRAM with a COE file it ECC is used

BRAM 64 bit data width and greater Hard Hamming Implementation

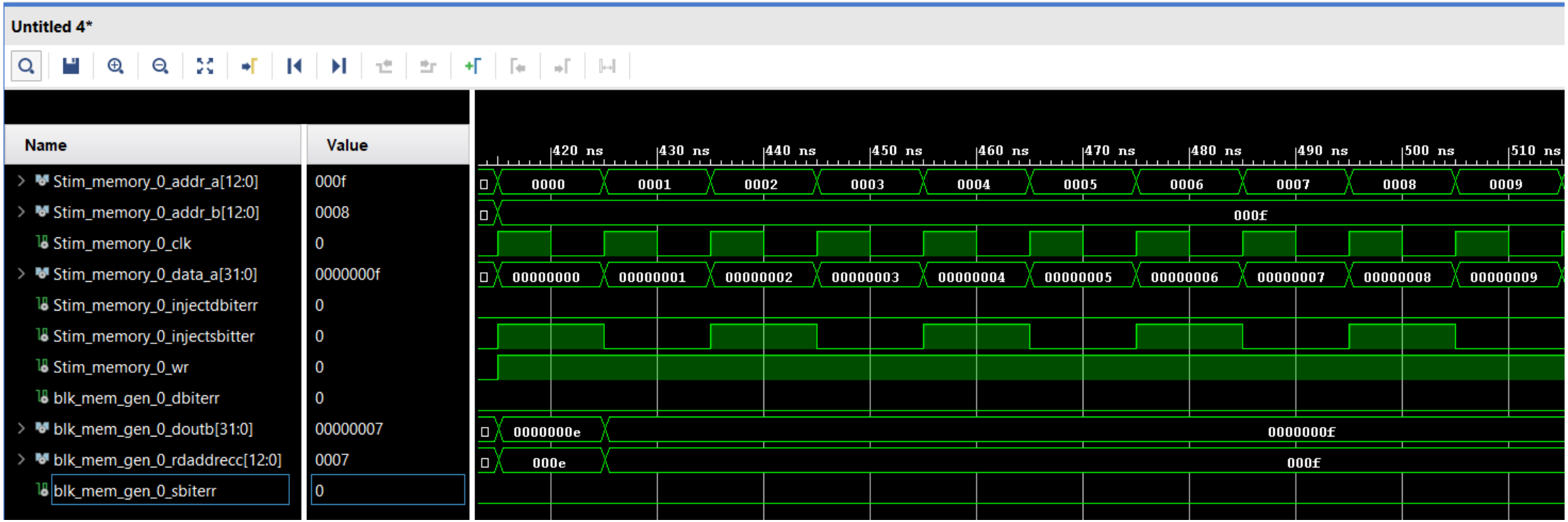
BRAM less than 64 bit data width soft hamming implementation

Can optimise for performance / power

ECC In Block Memory Generator



Injection of Error in to BRAM



Read out of Error

Watchdogs

ADIUVO
ENGINEERING AND TRAINING, LTD.

Watchdog

Commonly used in embedded systems

Useful in FPGA systems as well to ensure a single event functional interrupt (SEFI) lock up has not occurred.

If watchdog is not petted the device can be reset or reprogramming is triggered.

Can be integrated in the FPGA if desired

SRAM Configuration Corruption

ADIUVO
ENGINEERING AND TRAINING, LTD.

FPGA Mitigation Techniques 9

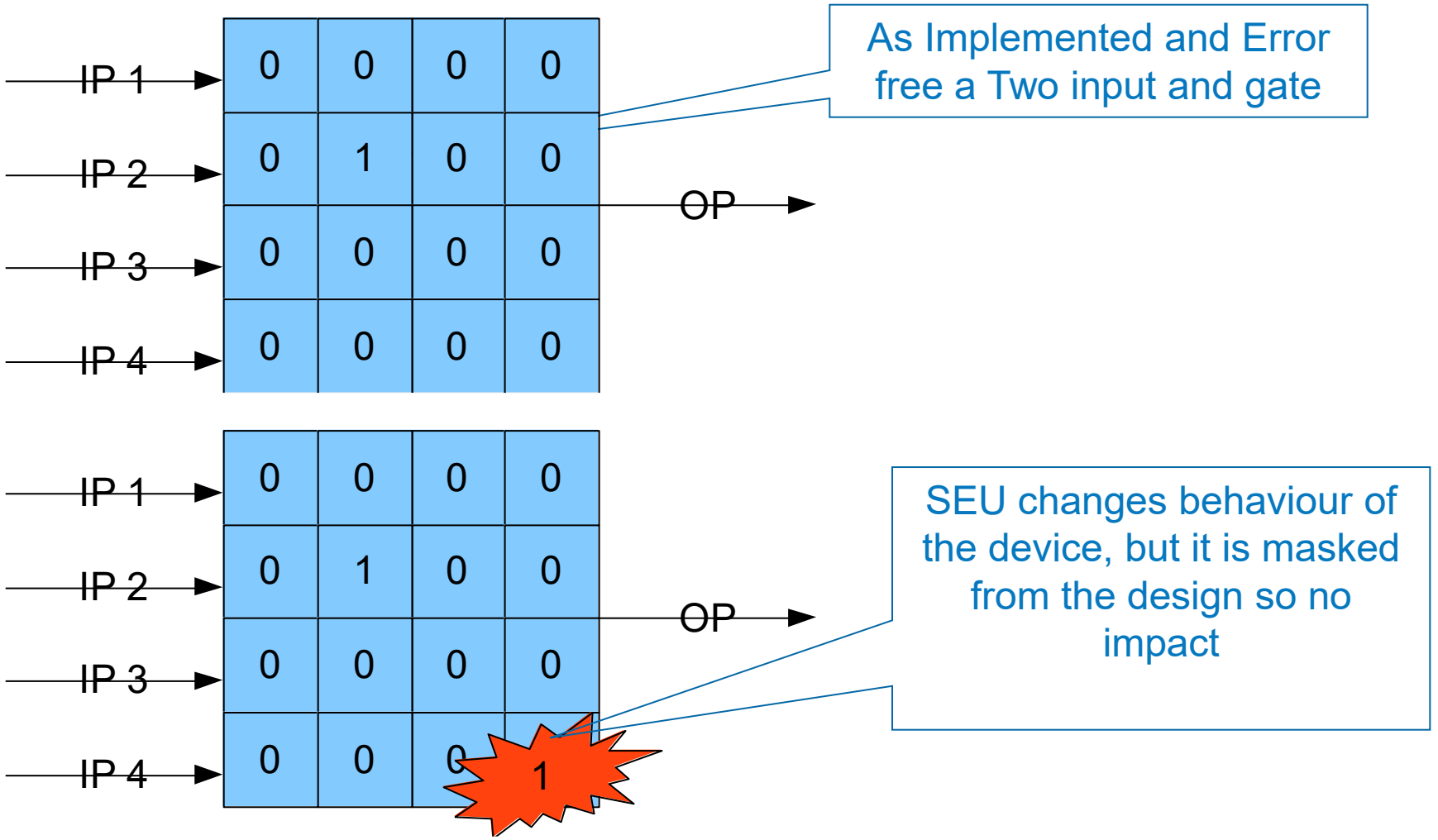
If you are using a SRAM FPGA not only do you have to consider the user registers, Ram, DSP etc but also the configuration memory.

What does the configuration memory do?

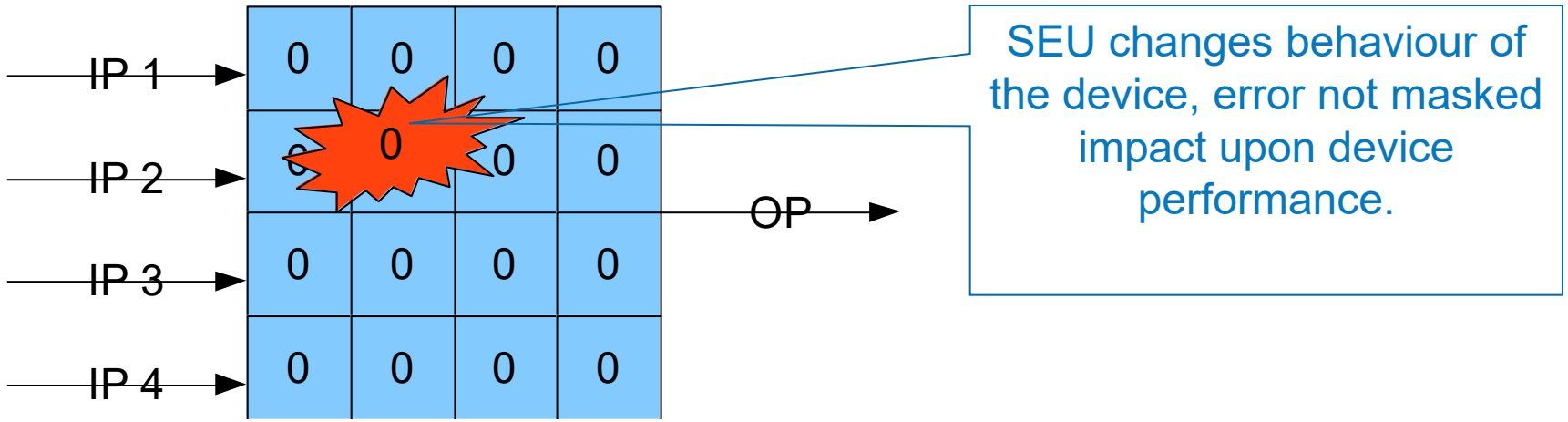
- Defines the equations implemented by Look Up Tables
- Defines the functionality of the LUT and User flip flops i.e. turn LUT to SRL, FF to FFSR
- Defines the routing of the signal – especially global signals
- Defines the constants within the circuit
- Defines the IO standards
- Defines functionality of advanced macros – DSP48 etc

Configuration

Equations implemented within LUT



Configuration



Configuration

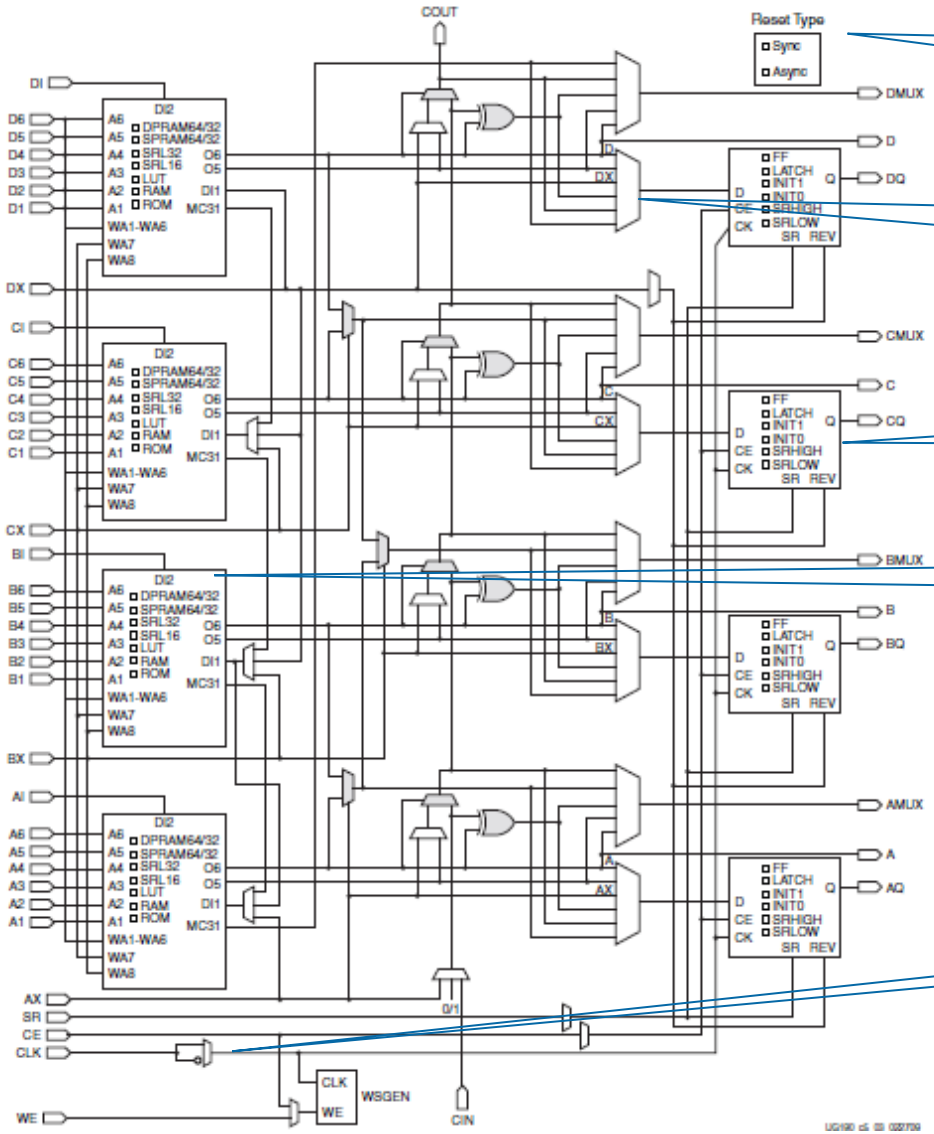
FPGA functions are very customisable hence a SEU in the configuration memory can have interesting affects.

For instance a LUT can be used as LUT, SRL, RAM or ROM depending upon how it is configured by the configuration memory.

A Flip Flop can be implemented as a Flip Flop, Flip Flop with Set, Flip Flop with Reset, Flip Flop with Enable and so on

Inputs in to the LUT or flip flop can be inverted

Configuration



Reset – ASync or Sync Selection

Op selection & Carry Chain Logic

Flip Flop Configuration

LUT Configuration

Clock Polarity

Soft Error Mitigation

Xilinx Soft Error Mitigation IP – Not supported in Vivado IP integrator

Xilinx TMR Soft Error IP – Supported in Vivado IP integrator

Need to use Essential Bits in the bit stream generation

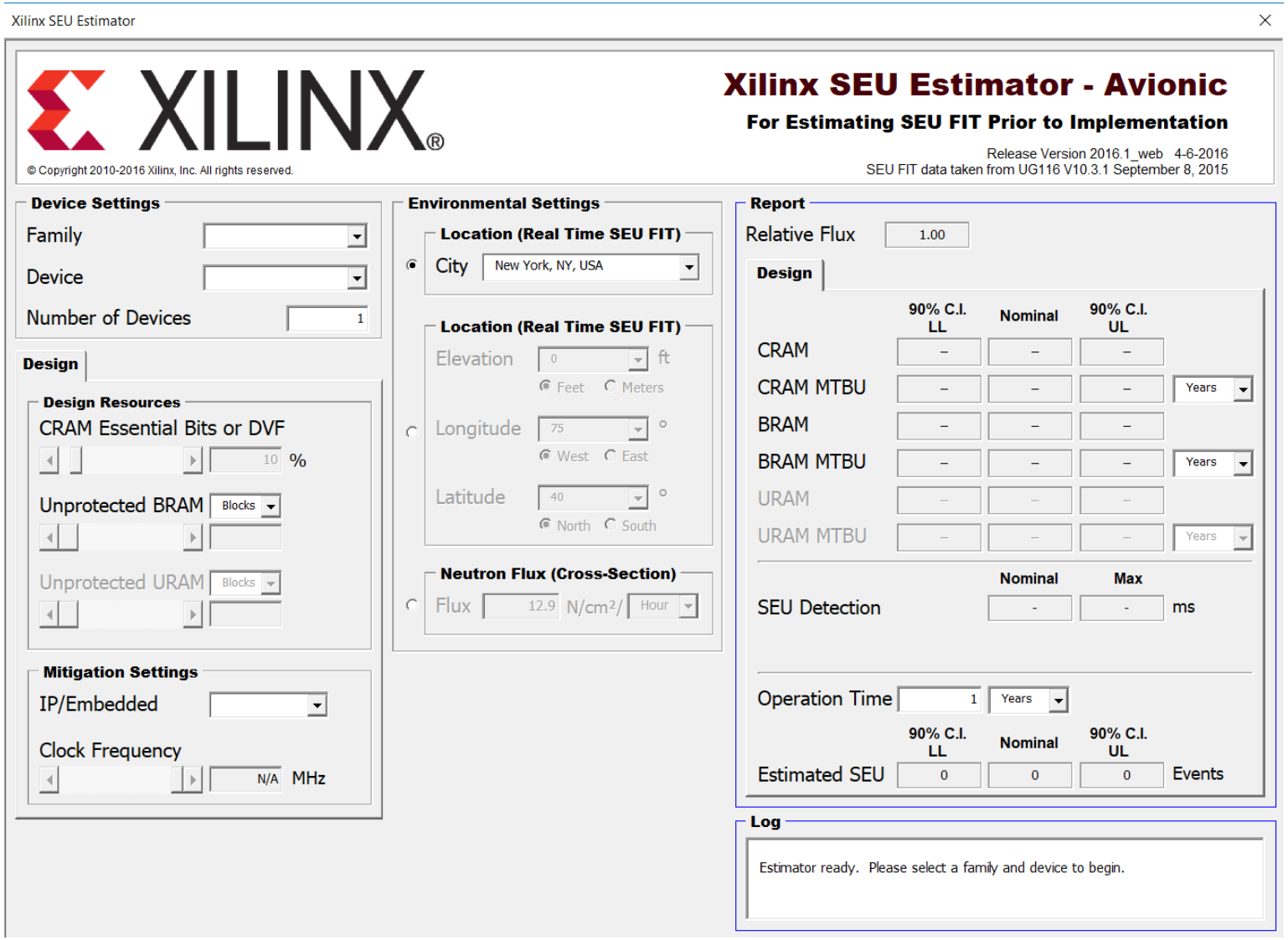
set_property bitstream.seu.essentialbits yes [current_design]

```
936 | Creating essential bits data...
937 | This design has 752055 essential bits out of 25697632 total (2.93%).
    Creating essential bits data...
    This design has 705781 essential bits out of 25697632 total (2.75%).
```

FPGA FIT/Mb rate

Tech Node	Product Family	LANSCE Neutron Cross-section per Bit [2]		FIT/Mb (Thermal Neutrons)		FIT/Mb (Alpha Particle) [3]		FIT/Mb [4] (Real-Time Soft Error Rate Per Event)[5][7]	
		CRAM	Error	CRAM	Error[6]	CRAM	Error[6]	CRAM	Error[6]
28 nm	Artix-7, Spartan-7, and Zynq-7000	6.99×10^{-15}	±18%	29	−10% +10%	43	−41% +80%	76	−8% +9%
28 nm	Kintex-7 and Virtex-7	5.69×10^{-15}	±18%	1.1	−15% +18%	43	−41% +80%	50	-20% +26%
20 nm	UltraScale	2.55×10^{-15}	±18%	0.5	−13% +16%	9	−64% +374%	31	-14% +17%
16 nm	UltraScale+	2.67×10^{-16}	±18%	0.35	−16% +20%	0.1	−20% +20%	5	-27% +39%

Using the SEU Estimator



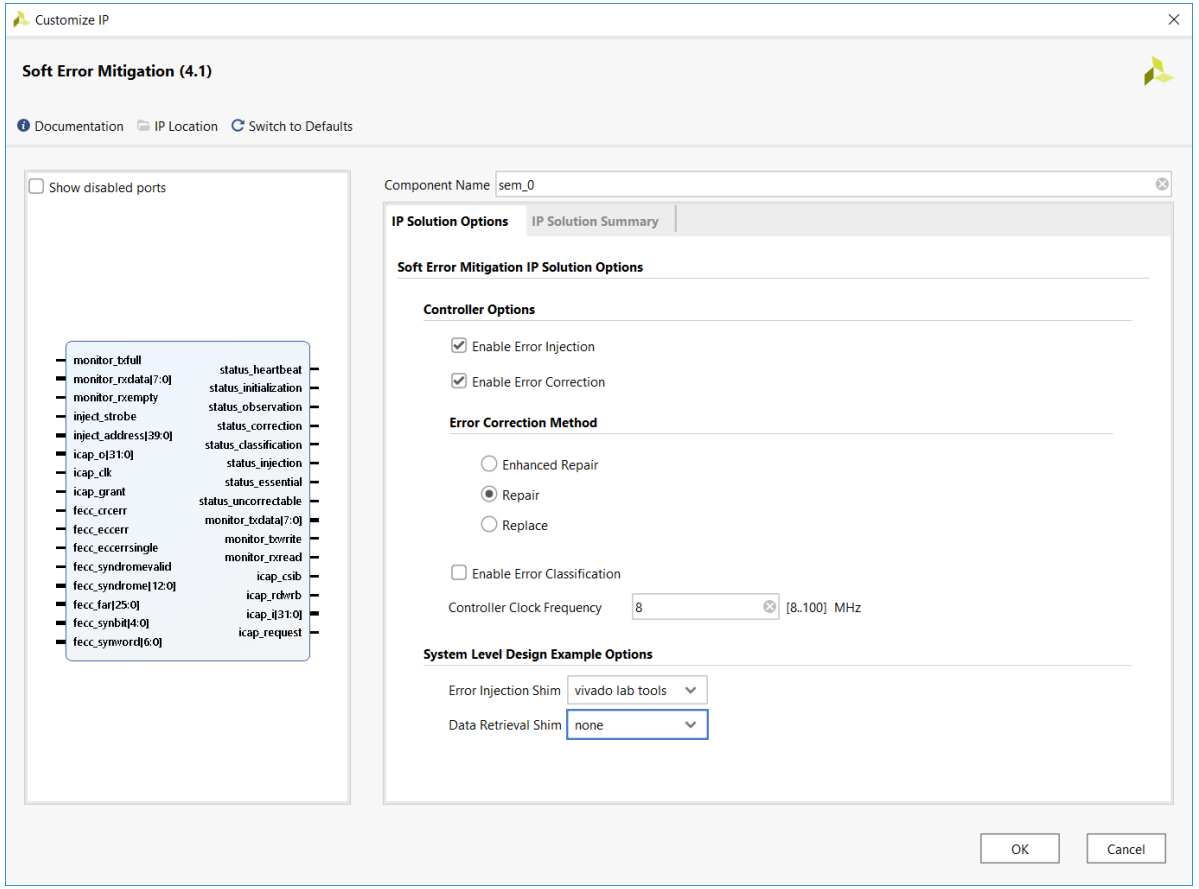
Results provides

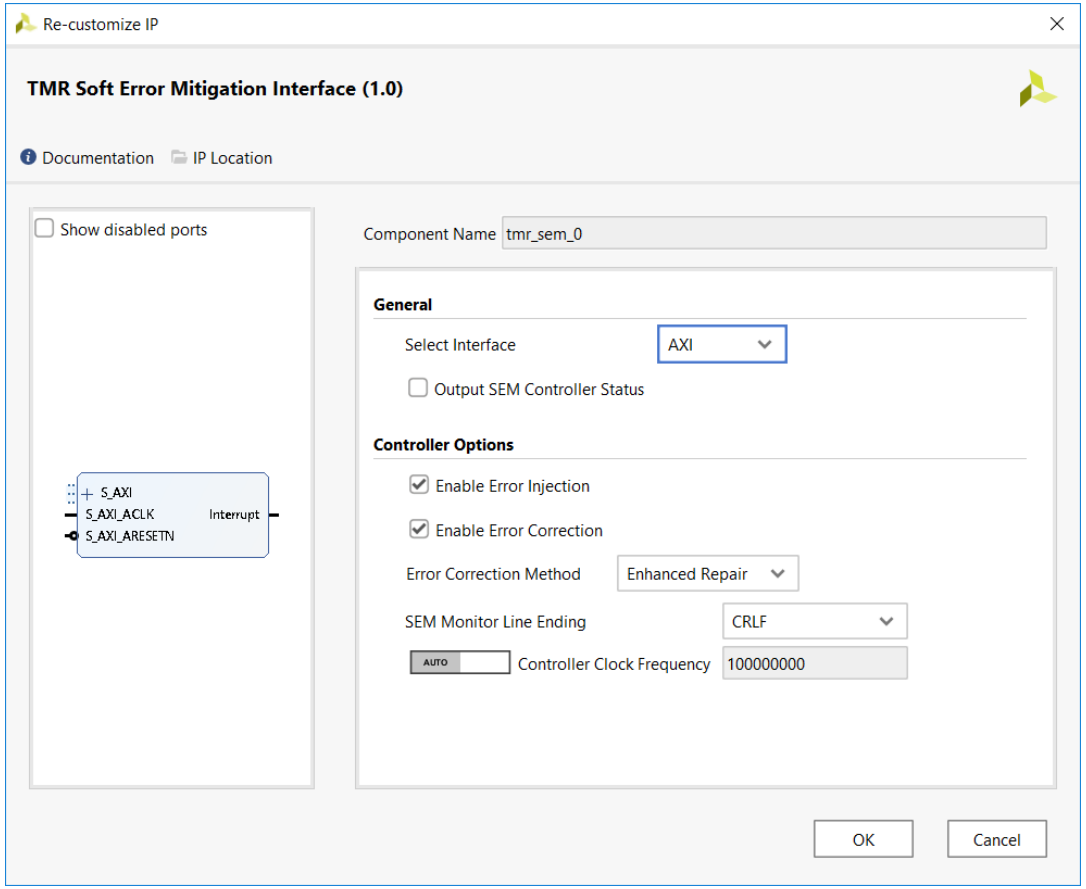
Detailed analysis of performance

Design Resources		
CRAM Essential Bits or DVF	3%	10%
Unprotected BRAM	0 / 50 Blocks	0 / 715 Blocks
Unprotected URAM		
Mitigation Settings		
IP/Embedded	SEM IP	SEM IP
Clock Frequency	100 MHz	100 MHz
Environmental Settings		
Location (Real Time SEU FIT)		
Elevation	25000 ft	
Longitude	0° W	
Latitude	40° S	
Neutron Flux (Cross-Section)		
Flux		
Location (Real Time SEU FIT)		
City		New York, NY, USA
Report		
Relative Flux	63.06	1
CRAM		
90% C.I. LL	2,028 SEU FIT	641 SEU FIT
Nominal	2,278 SEU FIT	720 SEU FIT
90% C.I. UL	2,552 SEU FIT	807 SEU FIT
CRAM MTBU		
90% C.I. LL	56.3 Years	178 Years
Nominal	50.1 Years	159 Years
90% C.I. UL	44.7 Years	142 Years

SEU Detection		
Nominal	2.29 ms	13.2 ms
Max	4.58 ms	26.5 ms

SEM Configuration





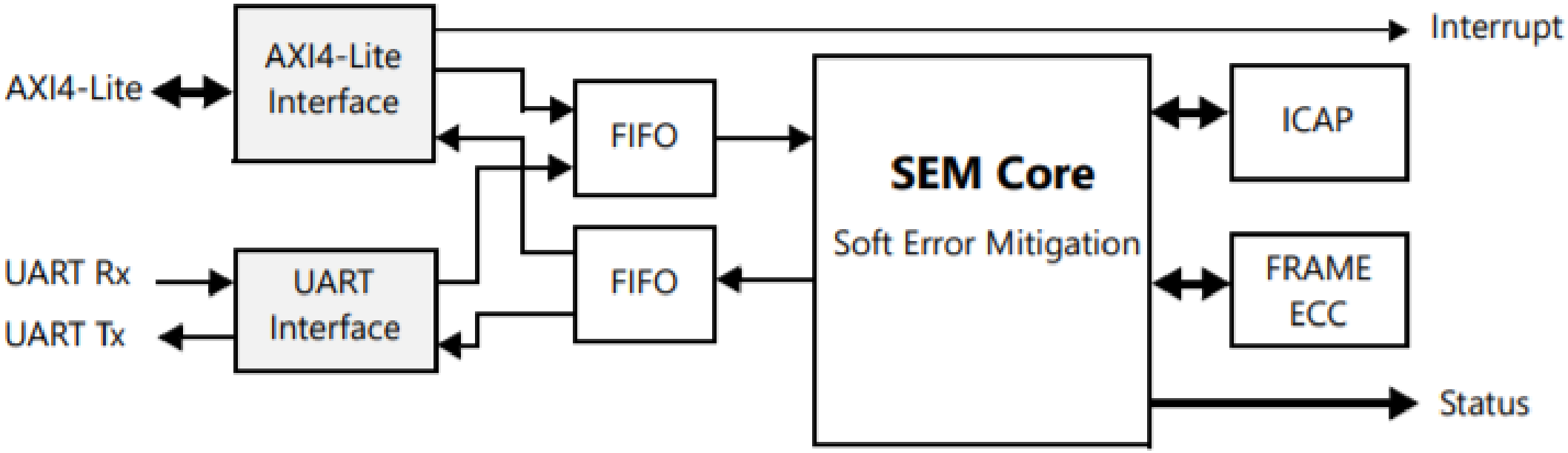
Without Enabling Classification – cannot tell essential / non essential split

SEM Repair Options

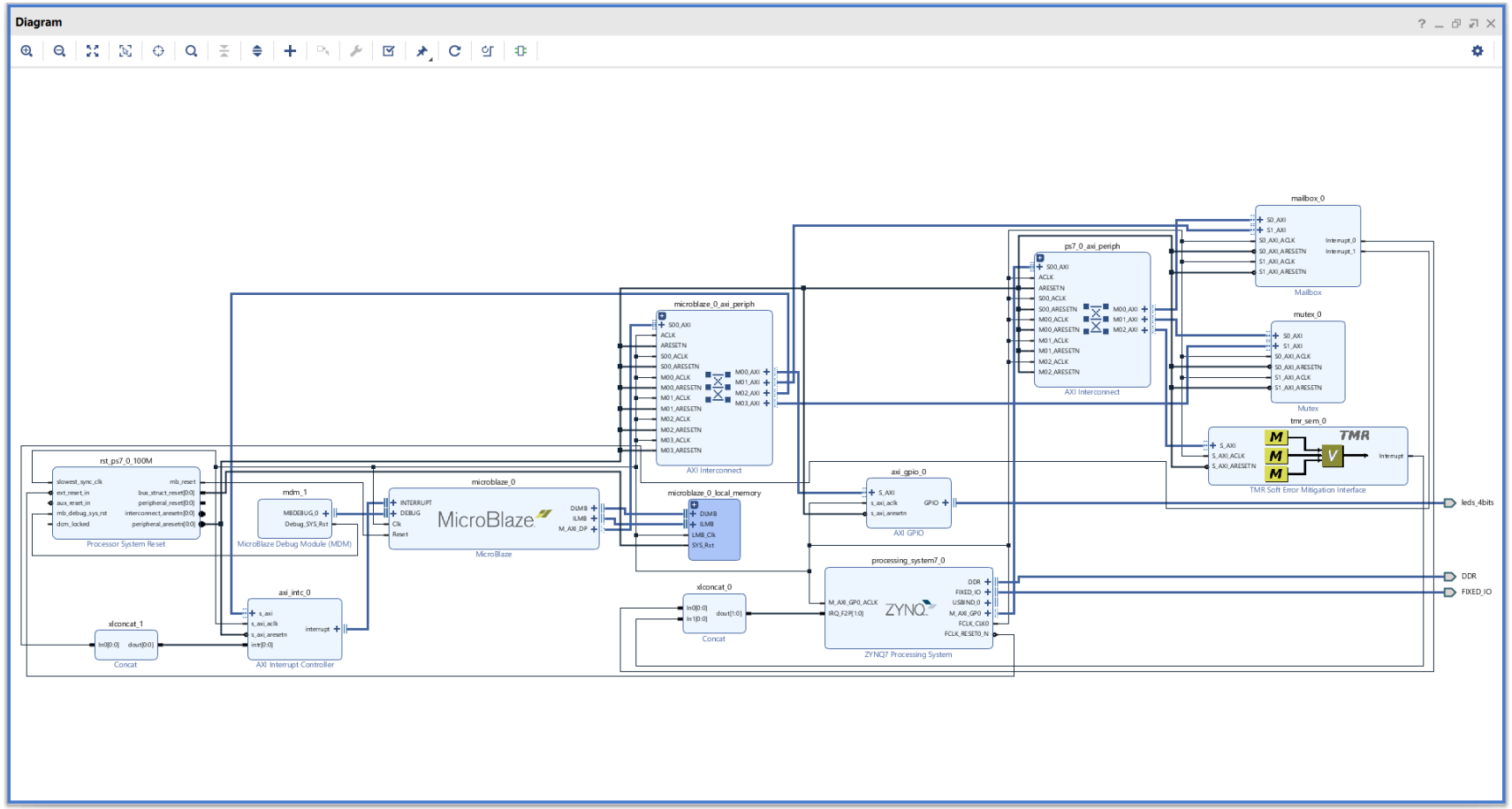
Repair	• ECC algorithm-based correction • Supports correction of configuration memory frames with single-bit errors • Covers correction of all single-bit upset events • Covers correction of multi-bit upset events when errors are distributed one per frame as a result of configuration memory interleaving
Enhanced Repair	• ECC algorithm-based correction • supports correction of configuration memory frames with single-bit errors. • covers correction of all single-bit upset events. • covers correction of multi-bit upset events when errors are distributed one per frame as a result of configuration memory interleaving
Replace	• Data reload based correction • Supports correction of configuration memory frames with arbitrary errors • Covers correction of any upset event that can be resolved to specific configuration memory frames, even if the exact bit locations in the frames cannot be determine

TMR SEM IP Core

Wrapped version of the most commonly used SEM IP core versions, it is controlled using the UART protocol which is provided internally or externally.



TMR SEM IP Example



Questions

1. What is the difference between the TMR SEM and SEM IP?
2. What are essential bits technology

FPGA Mitigation Technique

One mitigation techniques which addresses both configuration and user SEU is triple modular redundancy

Under ideal implementation, TMR will mask all faults provided there is only one error within the system.

What is ideal TMR

- Input and outputs triplicated
- All global signals are triplicated – Clocks, Reset, Enables etc
- All logic triplicated

Of course it can be difficult to implement the ideal TMR implementation

FPGA Mitigation Techniques 15

TMR will only work if there is one error within the system, this could be as a result as either a configuration or data upset. However to ensure continued correct functionality you need to do the following

- Scrubbing of the configuration memory to prevent a build up of errors within a configuration memory
- Correction of the failing data path in the TMR chain, his will very between data and control path. Data path corruptions will clear themselves as they are clocked through the system. Control path such as counters and state machines will need re-synchronisation to ensure the error is cleared.

FPGA Mitigation Techniques 16

Adding TMR can be complicated – by designer at RTL level requires determining the synthesis tool has not removed what it see's as redundant logic.

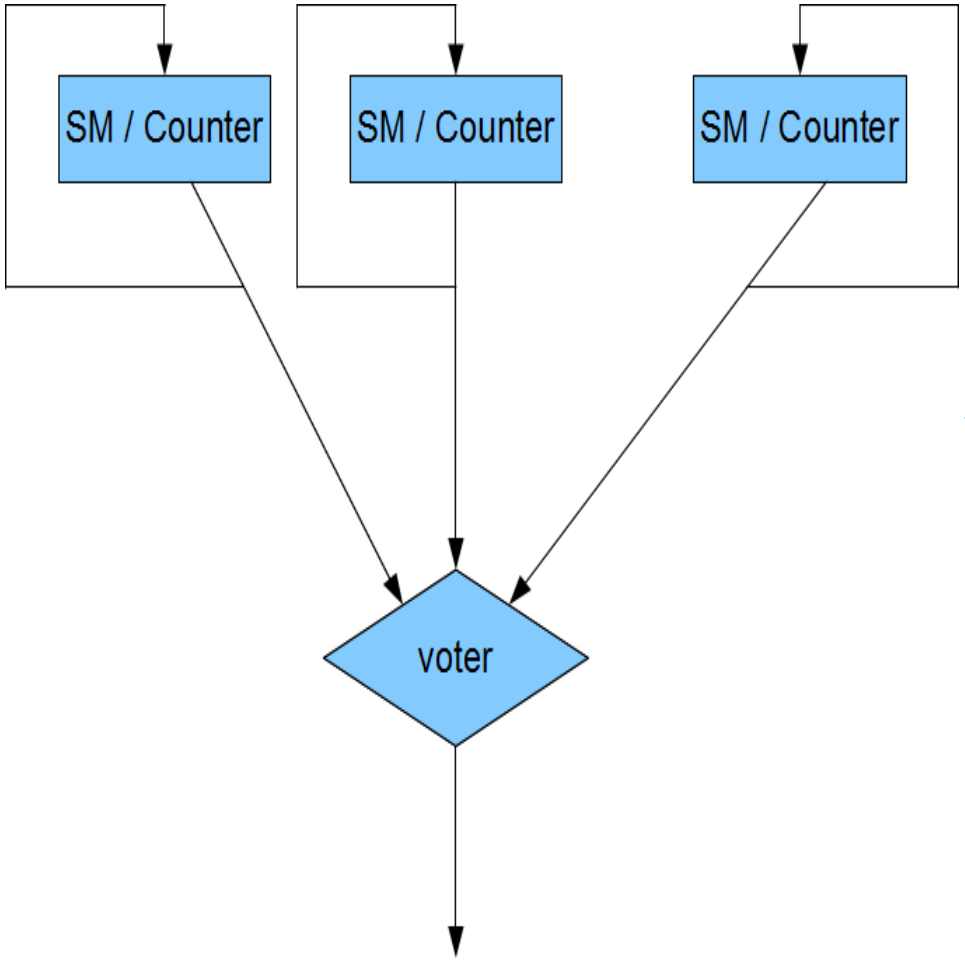
Easier to add it device level using a tool like

- XMTR – ITAR Controlled
- BL-TMR - Not ITAR Controlled

Implementing TMR will of course reduce the available logic resources within your FPGA

Mitigation Techniques 17

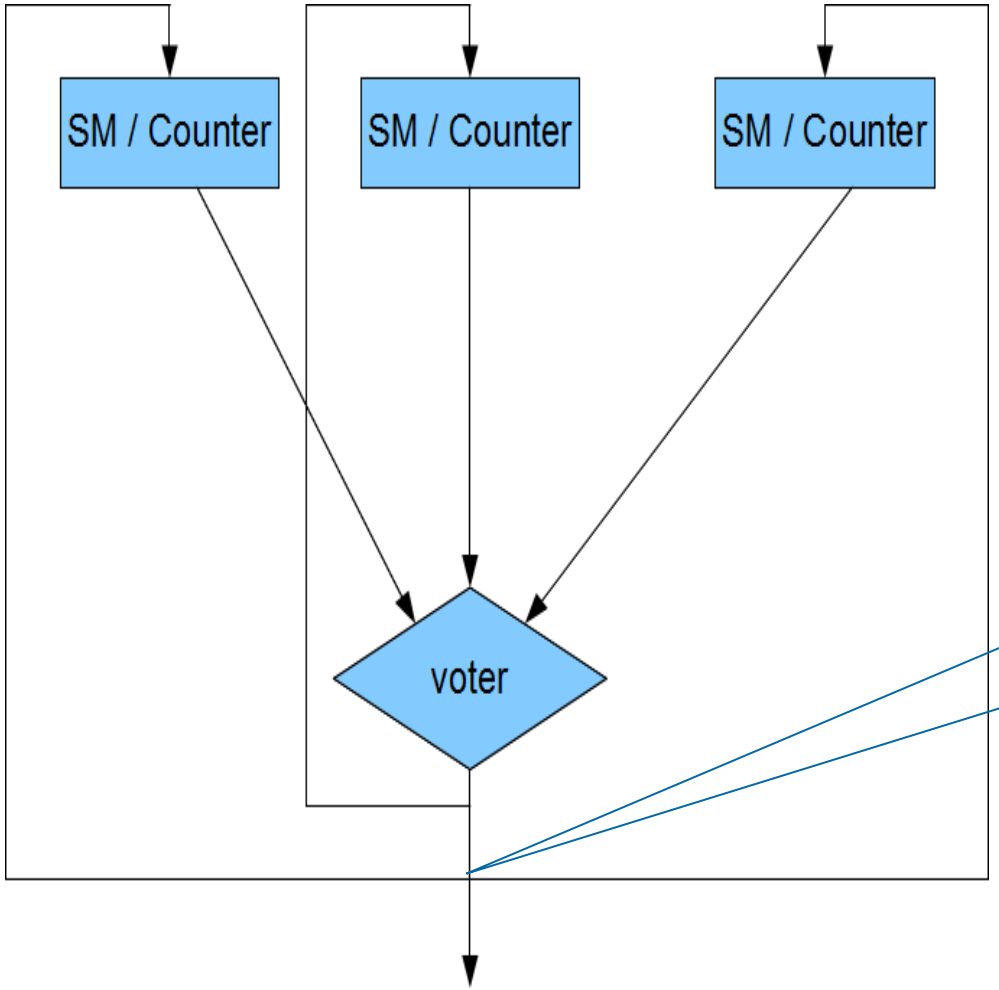
Without Synchronisation



Error in one State Machine is masked but it is never corrected, hence next error results in issues.

FPGA Mitigation Techniques 18

With synchronisation



Voted result is fed back to the next state logic, count value hence an error in one voter is detected and corrected

Global TMR

Triplicates all resources in the design including IOB and Clocks, reset trees

Protects entire design from SEU and SET

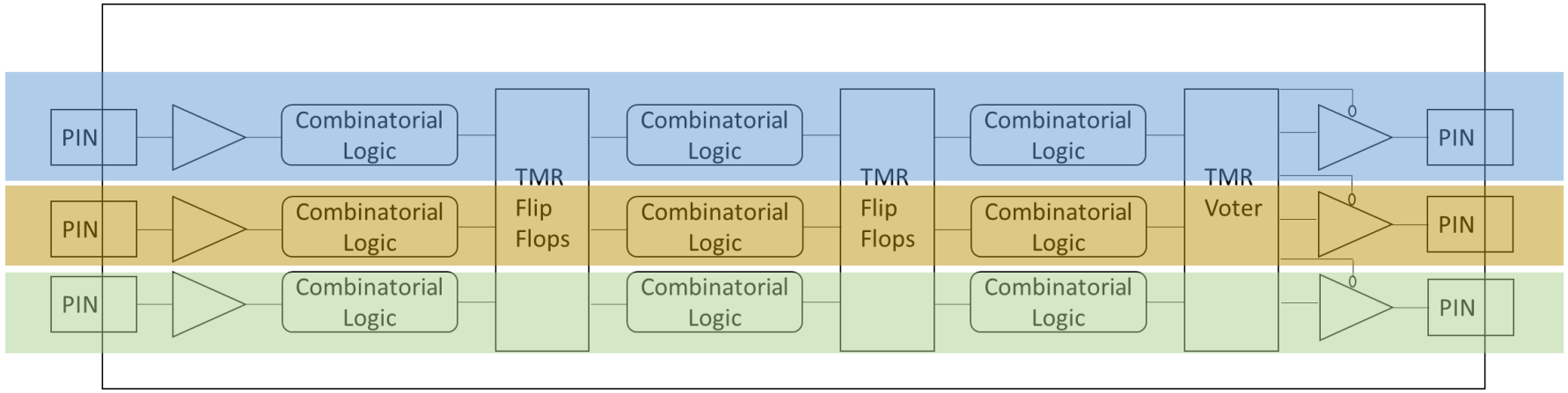
Protects from errors in configuration memory – but it does not correct them

More complicated to implement

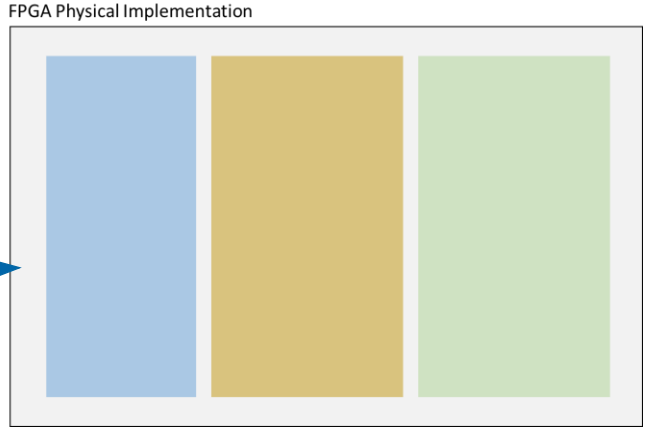
- Areas Penalty reduces size of design to be implemented
- Validation is increased need to ensure final bit file is implemented as desired
- Will have an impact upon power of the design
- Need to manage clock skew carefully

TMR needs to re synchronise

Global TMR

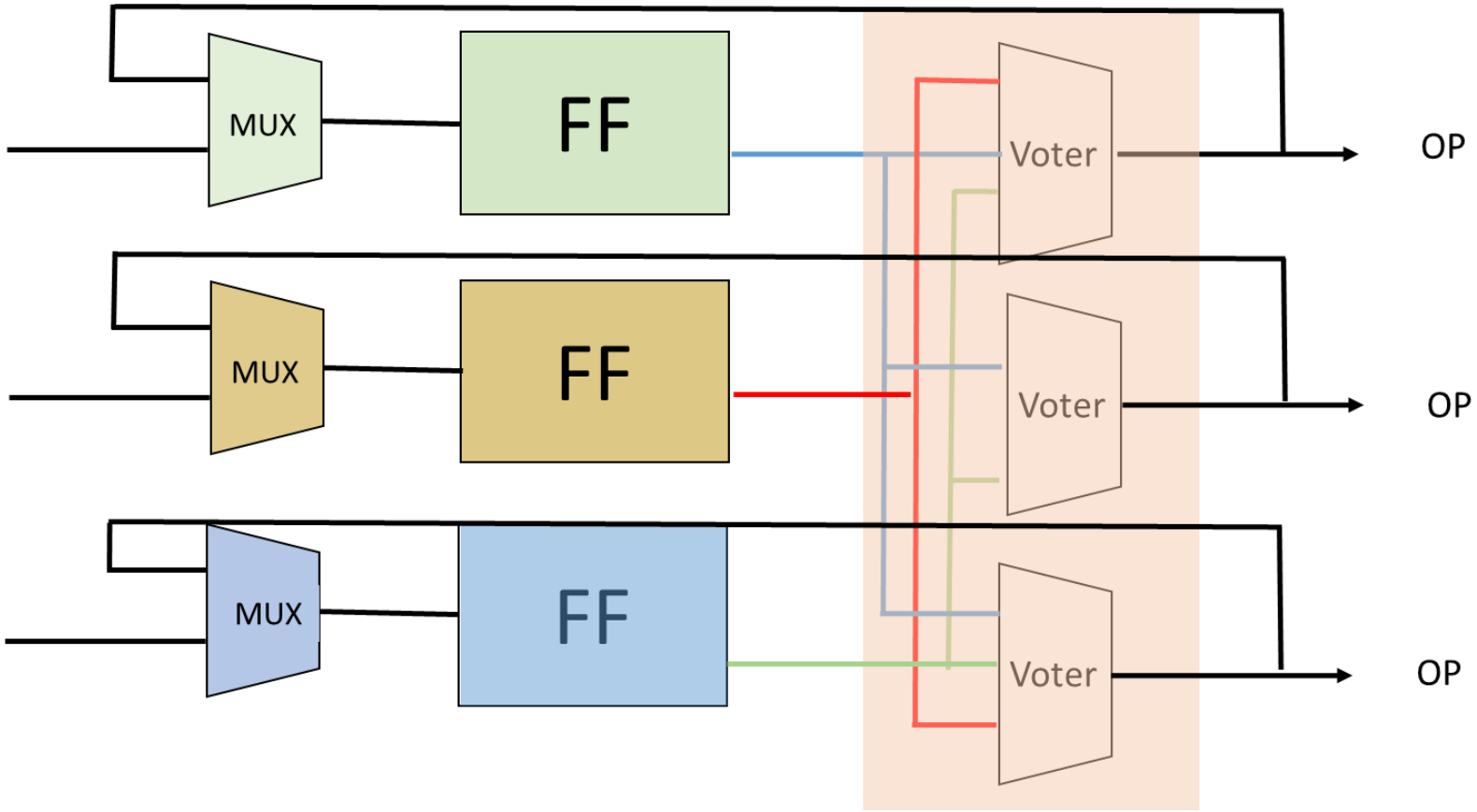


Need to ensure spatial separation in the implemented device



ADIUVO
ENGINEERING AND TRAINING, LTD.

Synchronisation of Flip Flops



Large Grain TMR

Triplicates all resources in the design including IOB and Clocks, reset trees, HOWEVER Flip Flops are not voted upon

Uses one voter prior to the output of the three modules.

Unlike Global TMR the FF are not resynchronised

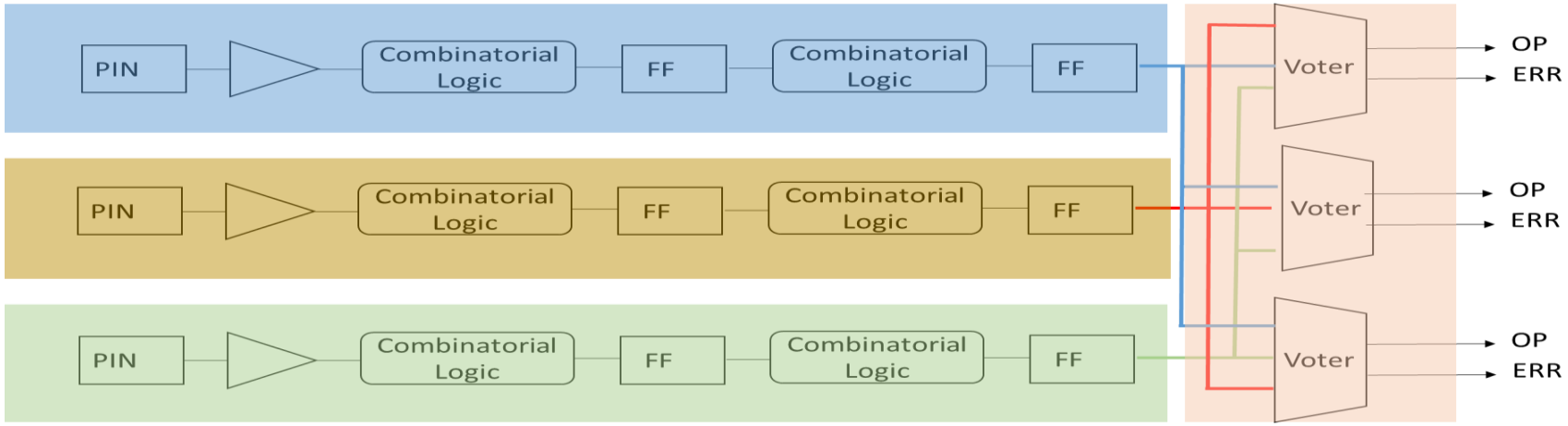
- Can be used with partial reconfiguration to reconfigure a incorrect chain if required.

It has minimal domain cross points unlike global TMR.

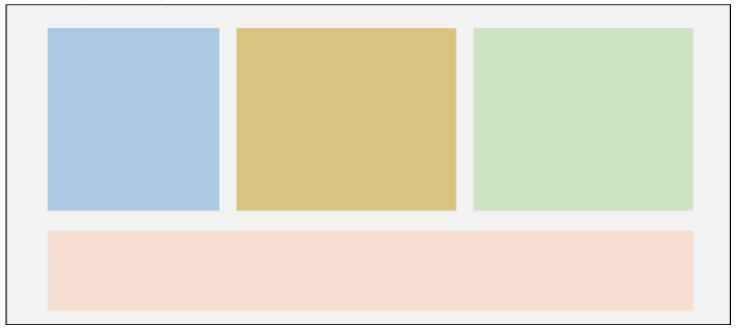
Mitigates both configuration and user logic errors

Like global TMR it has area and power penalties

Large Grain TMR



FPGA Physical Implementation



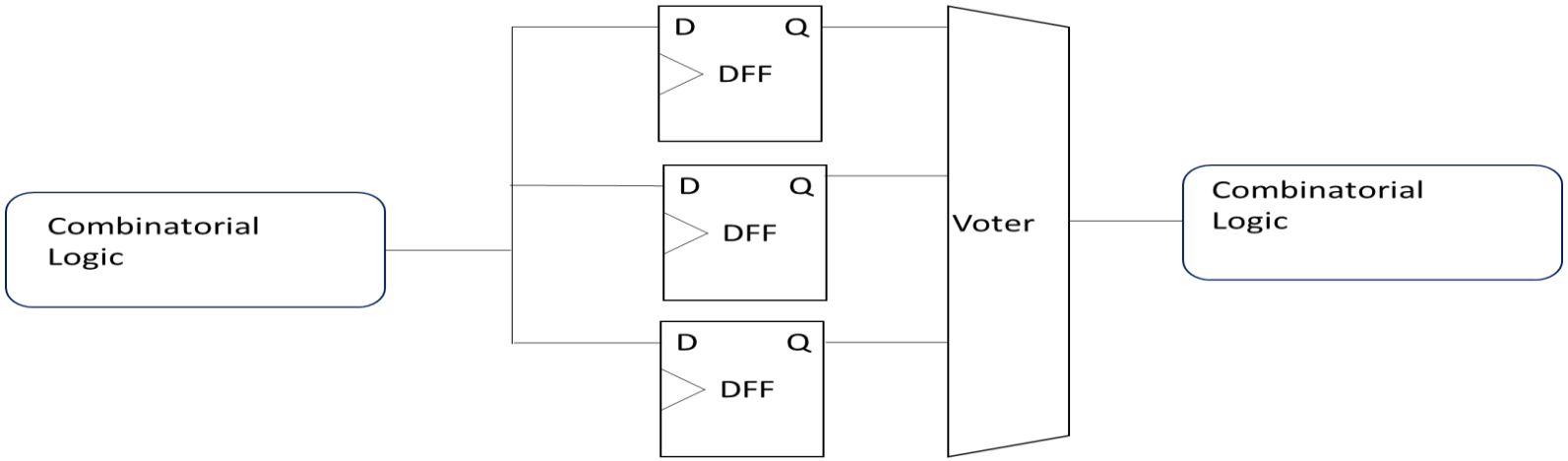
Local TMR

Triplicates Flip Flops and votes on the output

Best used in slow deigns to stop SET being clocked in

Offers area advantage as combinatorial logic is not replicated

Mitigates both user and configuration memory



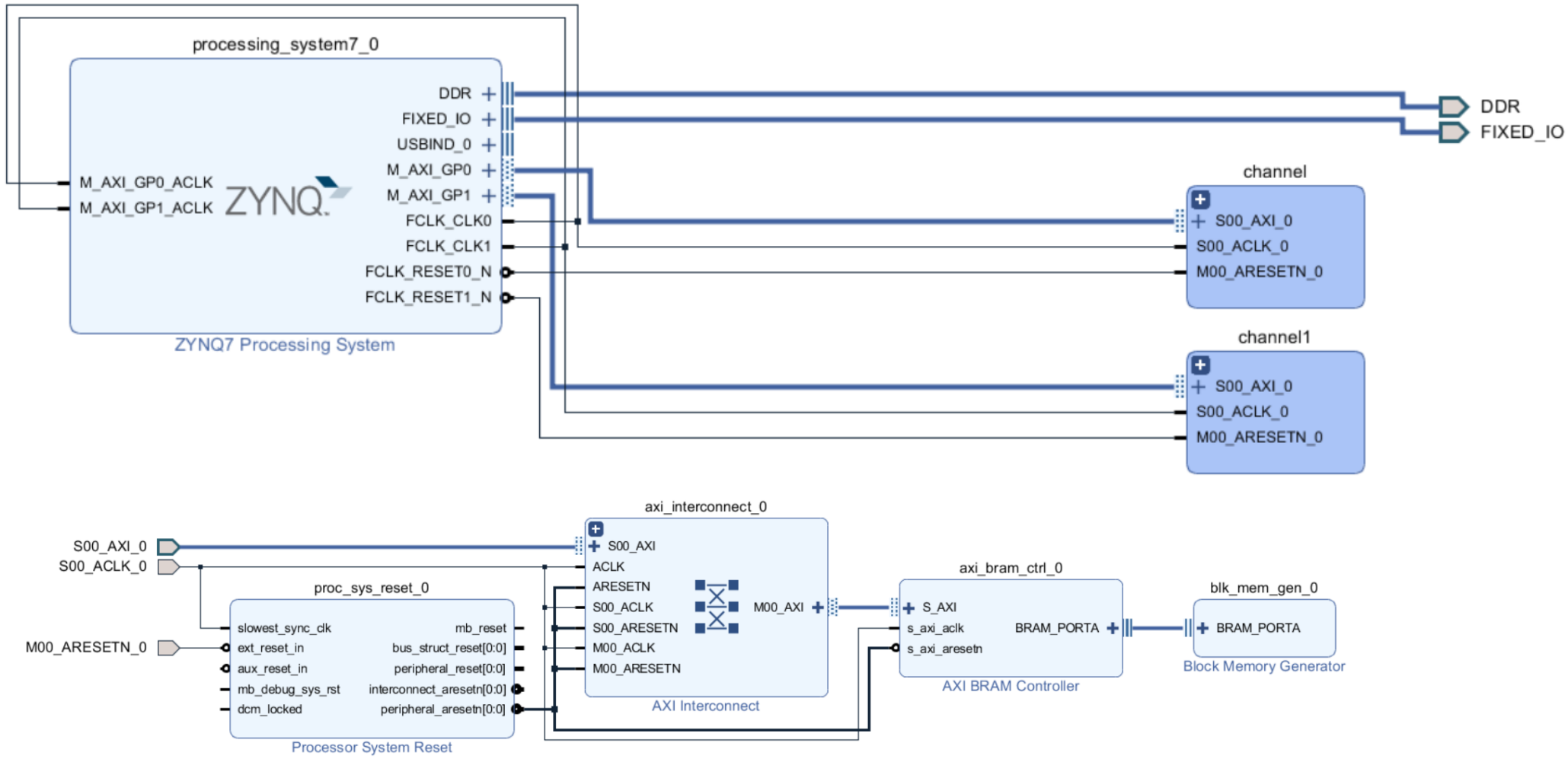
Isolation Flow

ADIUVO
ENGINEERING AND TRAINING, LTD.

Isolation Flow

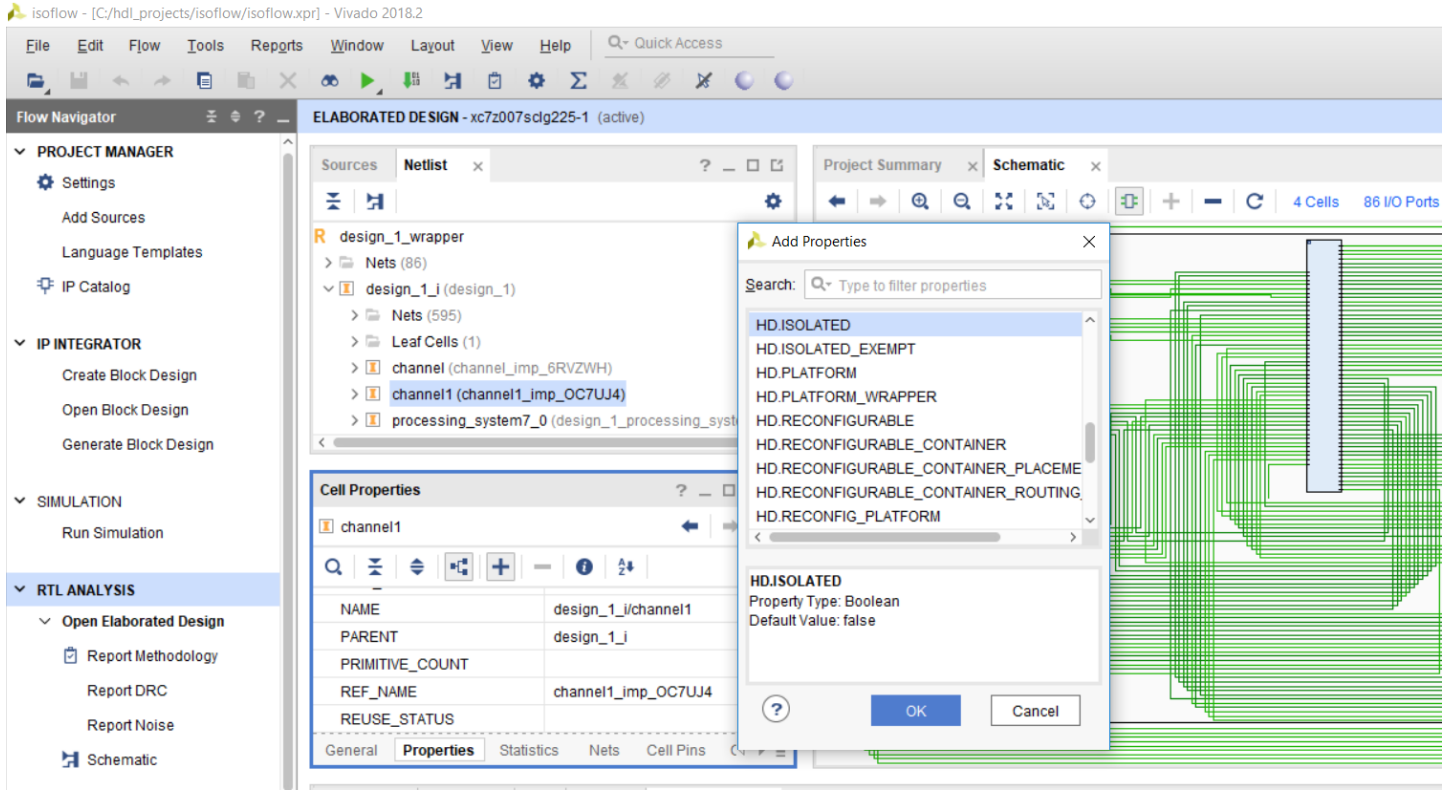
- Provides physical isolation of functions within the device itself
- This is very useful for when we want to ensure redundant functions cannot affect the other, e.g. to provide physical separation between TMR channels
- Isolation is also useful when it comes to information security and ensuring encrypted data (black) cannot be leak out clear text data (red)
- The first and most important is that the top level of the design should contain minimal logic, all functionality should be contained within hierarchical blocks

Isolation Flow

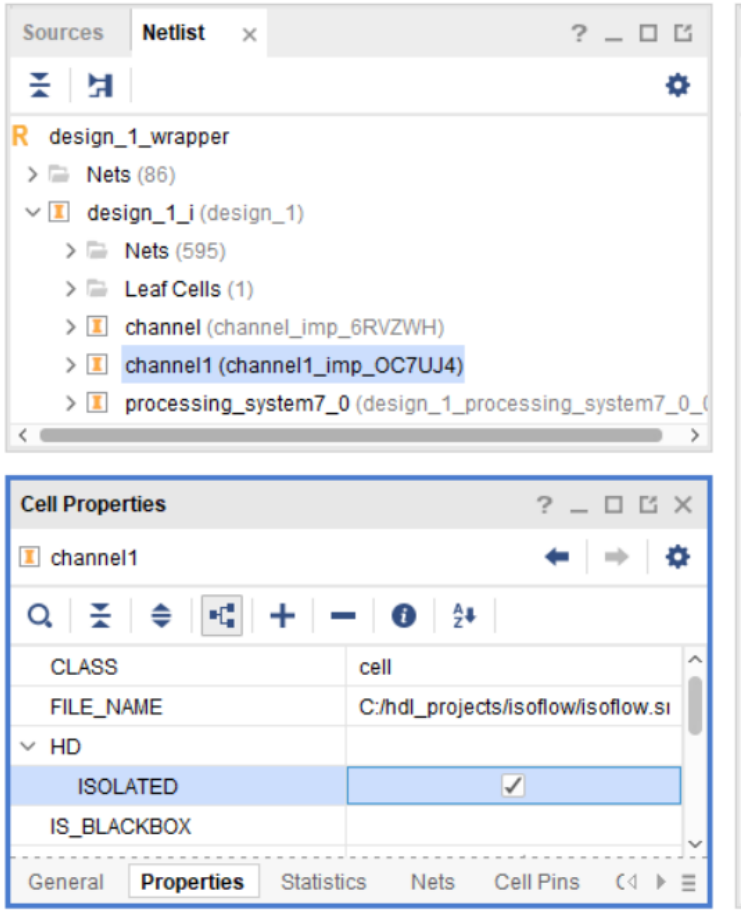


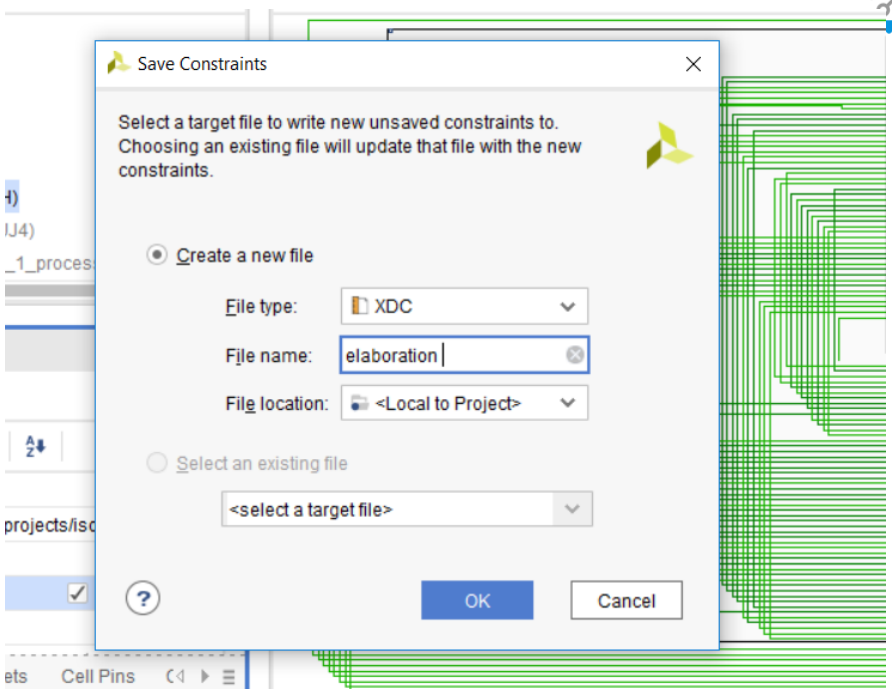
Isolation Flow

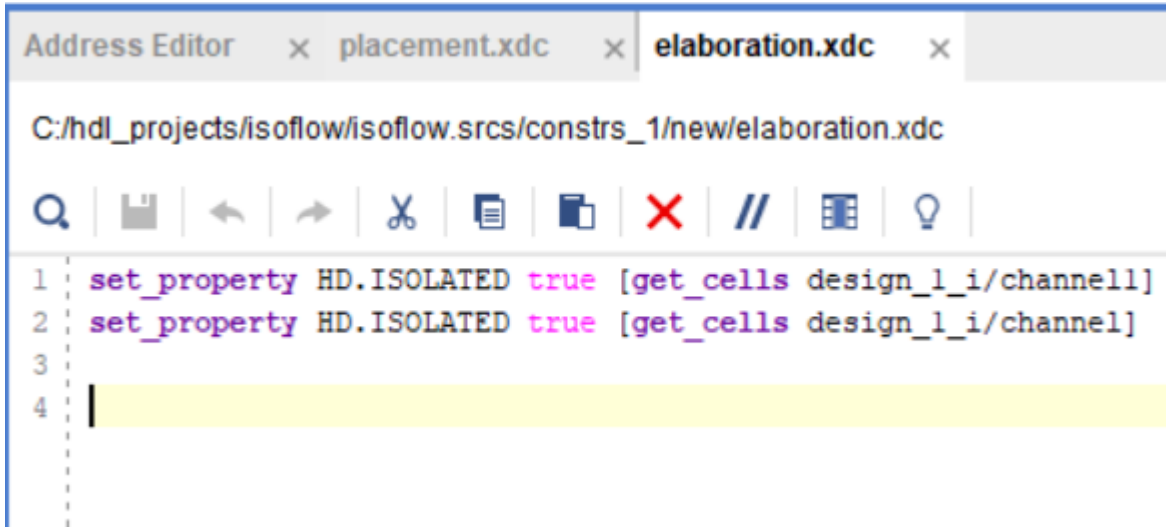
- First step in implementing isolation between elements is to open the elaborated design
- Apply HD.ISOLATED property to elements we wish to isolate



Isolation Flow

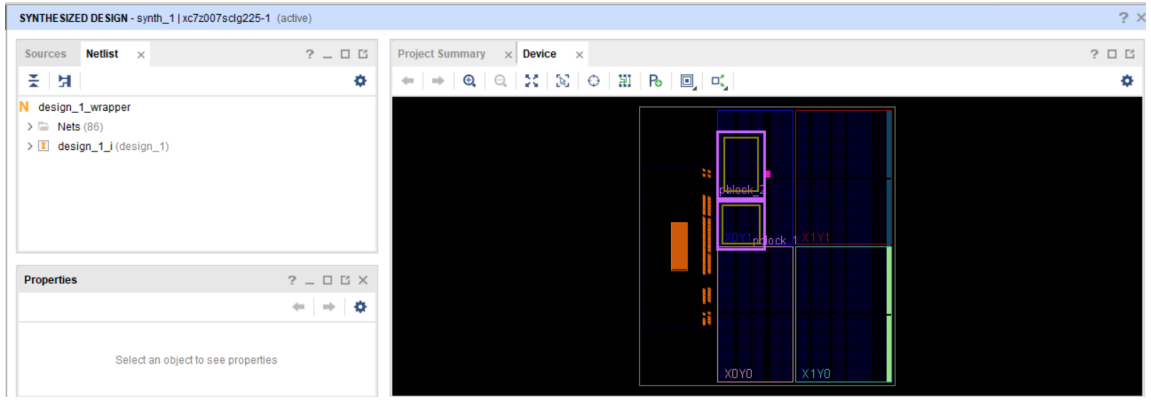






Isolation Flow

- Pblocks contain the resources needed to implement the isolated function
- Communicates with another isolated function, uses a trusted route
- Trusted routes are only allowed between isolated functions and cannot be used to communicate with non-isolated functions
- Could present issues with global routes such as clocks, inside a isolated function
- To address this issue, use the ISOLATE EXEMPT constraint, which can be applied to global / regional clocks and other exempted signals



ADIUVO
ENGINEERING AND TRAINING, LTD.

Isolation Flow

- When creating the Pblocks, keep in mind the rules regarding fencing
- Fencing is the insertion of at least one unused tile / function between isolated functions, this ensures a single error cannot break the isolation
- Placement Constraints files defines the Pblock and associated locations for Slices, DSPs & RAMs

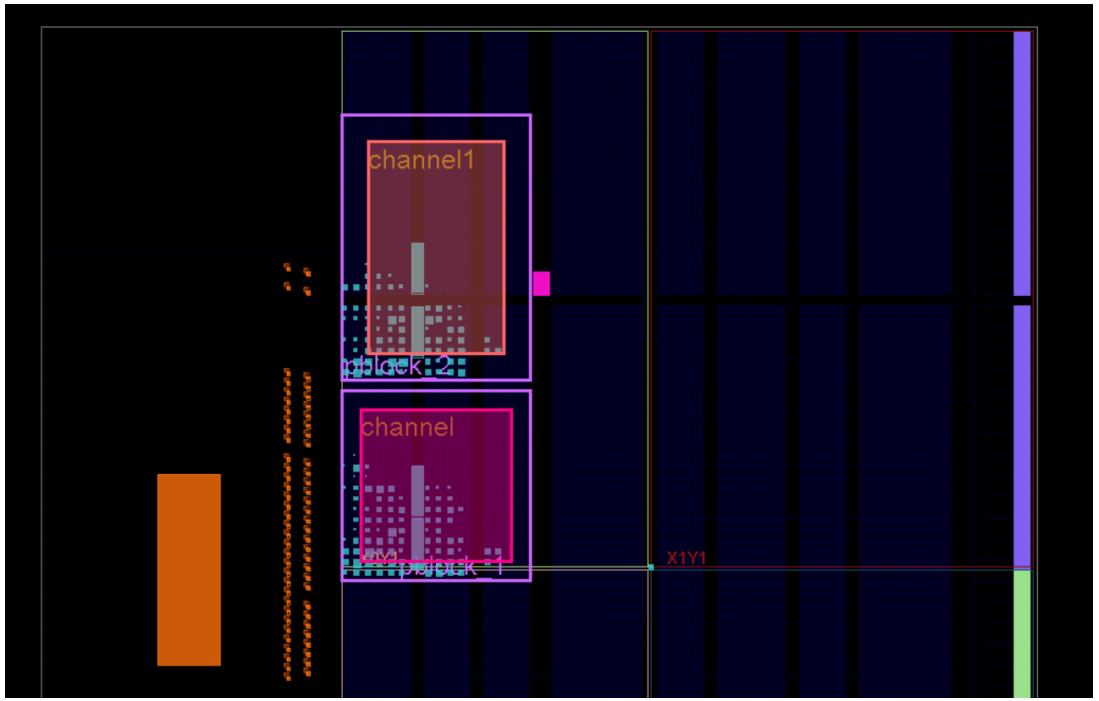
Project Summary | placement.xdc

C:/hdl_projects/isoflow/isoflow.srcs/constrs_1/new/placement.xdc

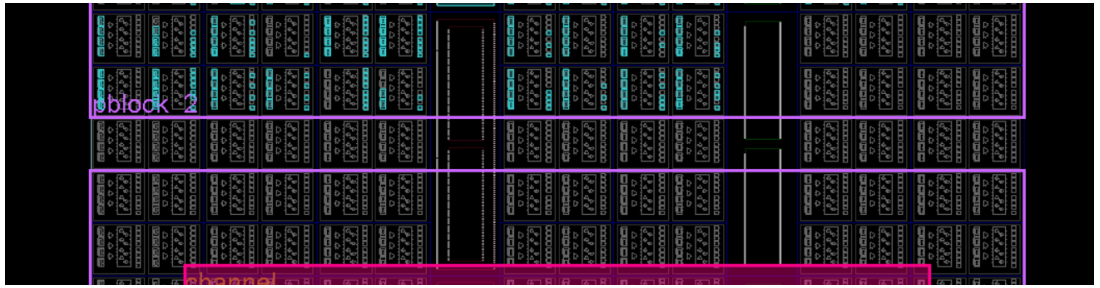
```
7
8  create_pblock pblock_2
9  add_cells_to_pblock [get_pblocks pblock_2] [get_cells -quiet [list design_1_i/channel1]]
10 resize_pblock [get_pblocks pblock_2] -add {SLICE_X0Y68:SLICE_X13Y91}
11 resize_pblock [get_pblocks pblock_2] -add {DSP48_X0Y28:DSP48_X0Y35}
12 resize_pblock [get_pblocks pblock_2] -add {RAMB18_X0Y28:RAMB18_X0Y35}
13 resize_pblock [get_pblocks pblock_2] -add {RAMB36_X0Y14:RAMB36_X0Y17}
14 create_pblock pblock_1
15 add_cells_to_pblock [get_pblocks pblock_1] [get_cells -quiet [list design_1_i/channel]]
16 resize_pblock [get_pblocks pblock_1] -add {SLICE_X0Y49:SLICE_X13Y66}
17 resize_pblock [get_pblocks pblock_1] -add {DSP48_X0Y20:DSP48_X0Y25}
18 resize_pblock [get_pblocks pblock_1] -add {RAMB18_X0Y20:RAMB18_X0Y25}
19 resize_pblock [get_pblocks pblock_1] -add {RAMB36_X0Y10:RAMB36_X0Y12}
20
21
```

Isolation Flow

Pblock 1 and Pblock2 Following Implementation

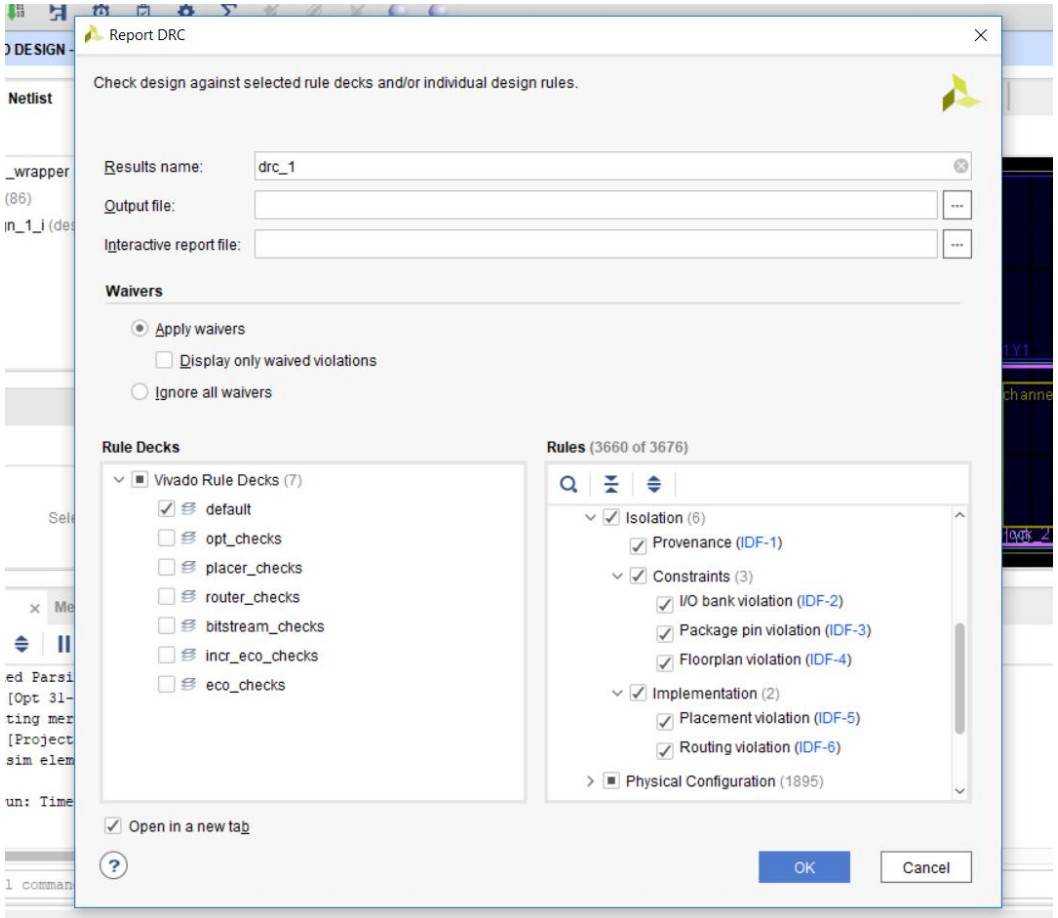


Fencing between Pblock 1 and Pblock2



Verifying the Isolation

- Need the Vivado Isolation Verifier (Pre 2018.2)
- Post 2018.2 integrated with the solution



ADIUVO ENGINEERING AND TRAINING, LTD.

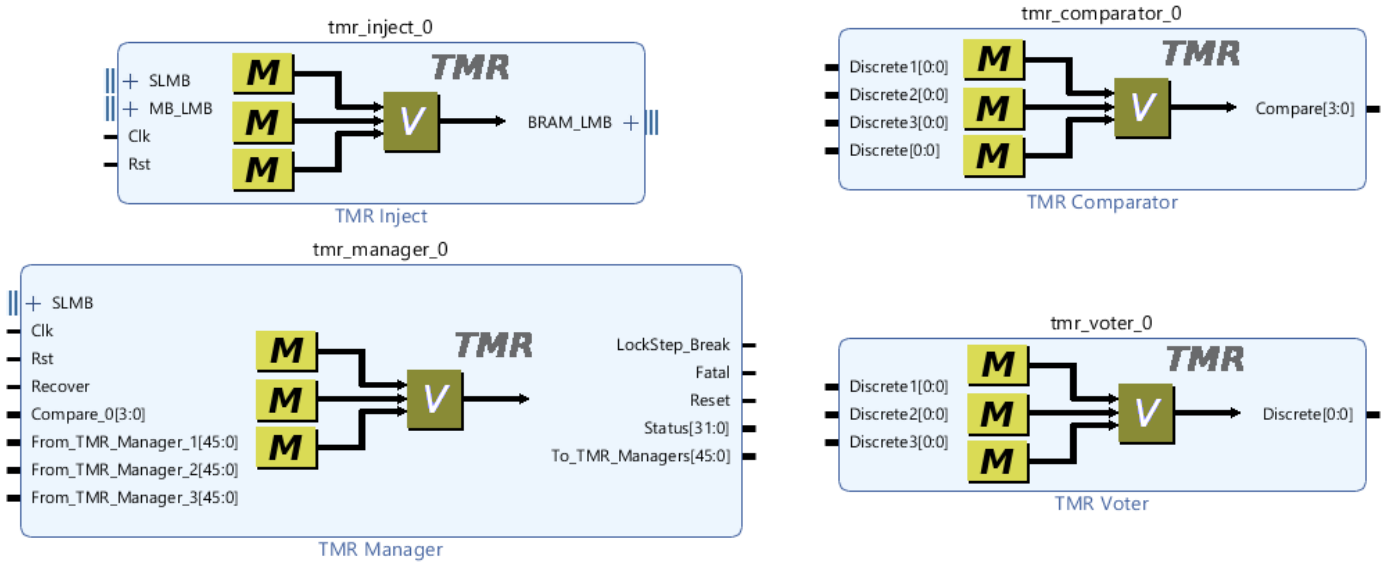
Xilinx TMR Support

ADIUVO
ENGINEERING AND TRAINING, LTD.

Xilinx Support for TMR Operations

Several IP blocks provided to support TMR Operations including

- TMR Comparator – Compares inputs across a range of different standards
- TMR Manager – Controls overall redundancy and SEM
- TMR Voter – Implements a majority voter across a range of different standards
- TMR Injection – implements functional injection in to MicroBlaze RAMS
- TMR SEM IP – SEM IP



Xilinx TMR

TMR Voter—This implements a majority voter, the output value is that which two out of the three inputs agree on.

TMR Comparator—This is used to identify faults on its input bus. It is also possible to use the comparator to check for voter errors. The difference between a TMR Voter and TMR Comparator is that the voter outputs a value which the majority vote for, a comparator outputs the results of the comparison.

TMR Manager—This manages the overall TMR system state, including fault detection and recovery.

TMR SEM—This implements Soft Error Mitigation to ensure soft errors in the device configuration memory cannot impact the design in the logic.

TMR Inject—This provides the ability for fault injection, and to ensure fault detection and fault recovery logic is working correctly.

Verification

ADIUVO
ENGINEERING AND TRAINING, LTD.

Verification

Often more complex than the design

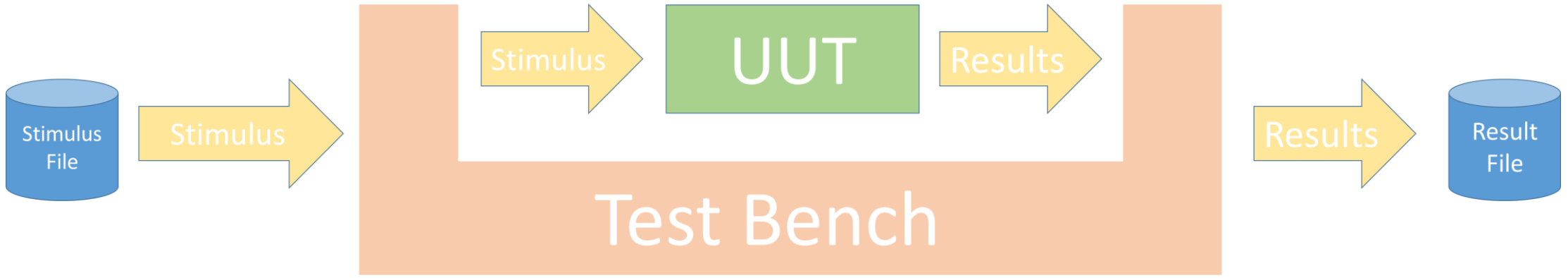
Some basics

- Performed by independent engineer
- Test Plan should align with the
- Test Benches should be self checking and documenting
- Use frameworks wherever possible e.g. OSVVM, UVVM, UVM
- Formal Verification

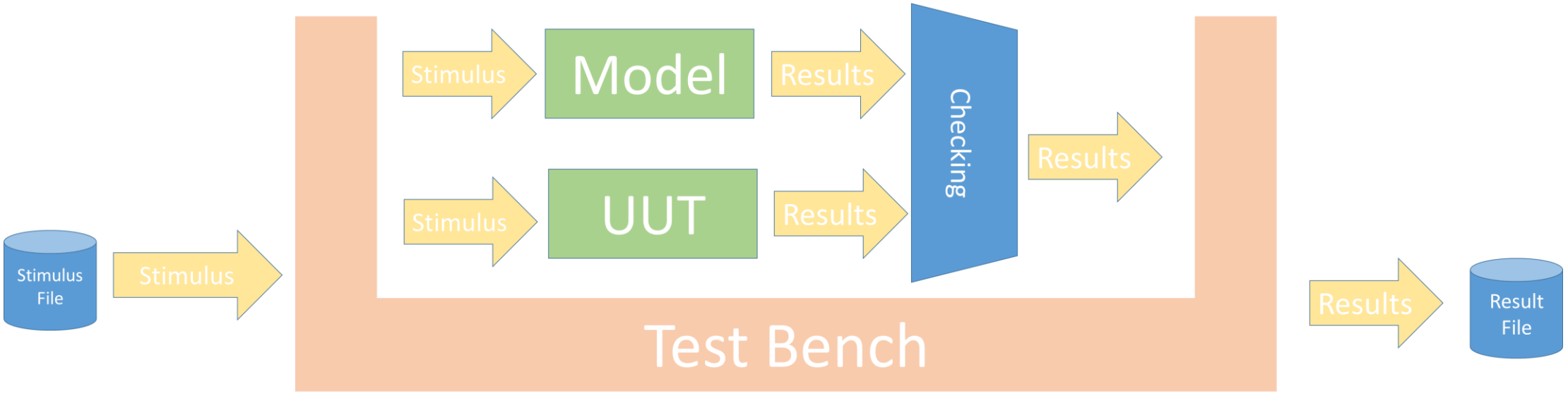
Not all Verification needs a Simulator

- Approximately 60% of issues found in simulation could be found via static methods
- Code / structure analysis – e.g. Blue Pearl Visual Verification Suite

Verification



Verification



Verification

Need to address all the corner cases and boundary conditions

Code coverage is also important to check

- Paths, Toggle, Branch, Triggering, Conditions etc
- Code coverage cannot tell you what is missing from the design!

Formal Methods can be a more useful in hitting the extreme corner cases

What is the difference between simulation and formal

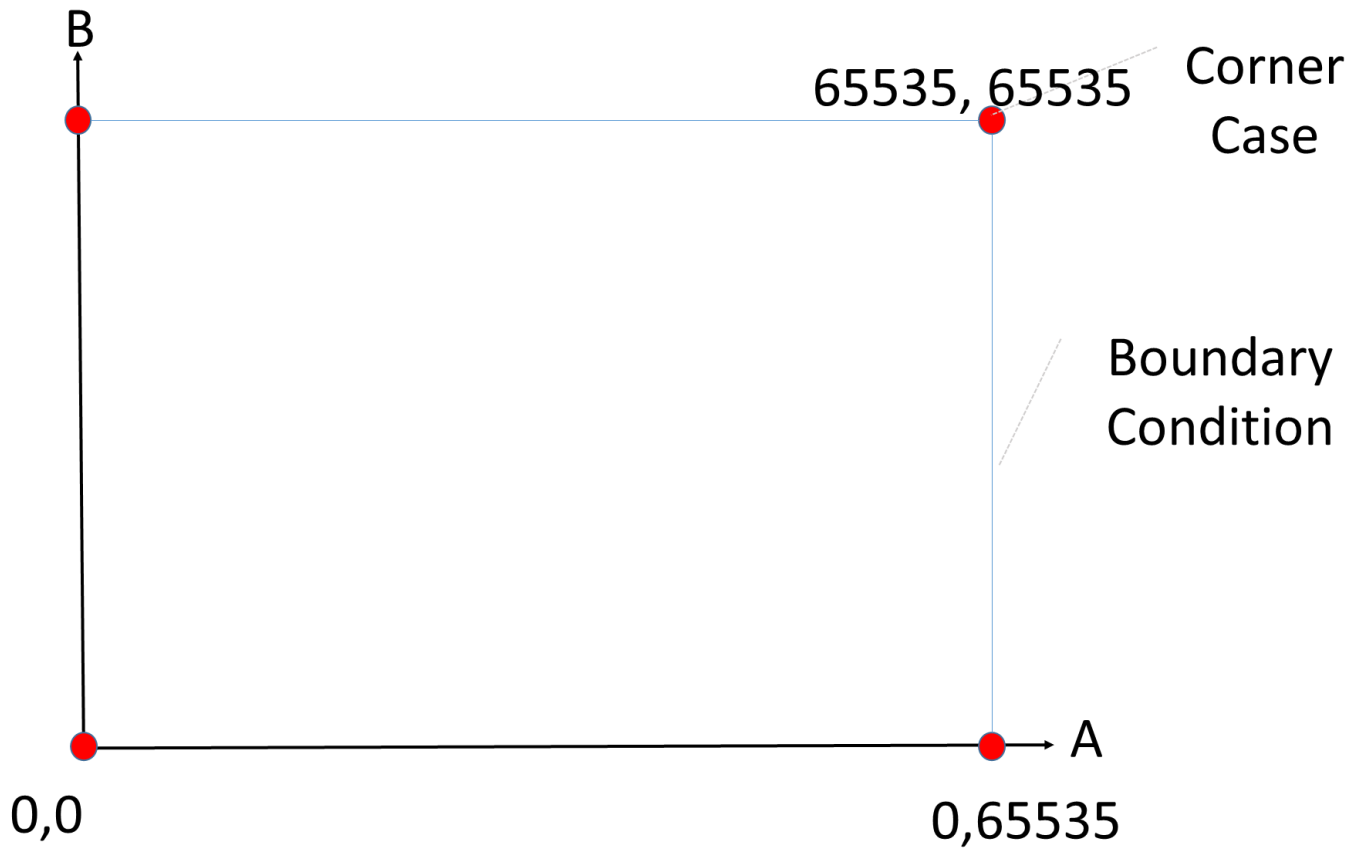
- Simulation – hand created stimulus to expose states to be checked
- Formal – Applies checks to all states and transitions – no stimulus need be created
  - Instead assertions are used express expected behaviour

Verification

Code Coverage Parameters

- Statements - each executable statement is examined to determine how many times is executed.
- Branch, all possible IF, CASE, SELECT, branches are executed
- Conditions and sub conditions within a code branch are tested to see what condition caused it to be true.
- Paths, all paths through the HDL are traversed to identify any untravelled paths.
- Triggering, monitor signals in the sensitivity list of VHDL processes and wait statements.

Corner and Boundary Cases - Simple



Optimization / Synthesis Issues

ADIUVO
ENGINEERING AND TRAINING, LTD.

Synthesis

Synthesis tools are optimised for logic performance in many instances

Introduce structures which might cause issues with mission criticality

Things we need to consider are

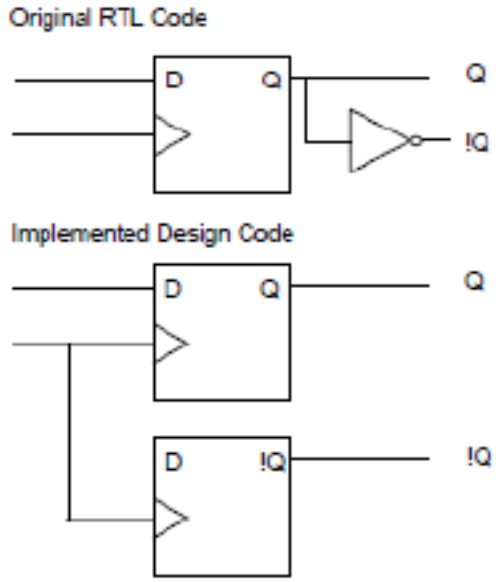
- Fan out
- Replication

May need to use synthesis constraints

- Do you allow them in the RTL source code
- Or constraint only

Can use formal equivalent checking

- Check output of synthesis engines



syn_keep	Only works on nets and combinational logic. It ensures that the wire is kept during synthesis, and that no optimizations cross the wire. This directive is usually used to prevent unwanted optimizations and to ensure that manually created replications are preserved. When applied to a register, the register is preserved and not absorbed into a macro.
syn_preserve	Ensures that registers are not optimized away.
syn_noprune	Ensures that a black box is not optimized away when its outputs are unused (i.e., when its outputs do not drive any logic).

Multiboot FPGA & SOC

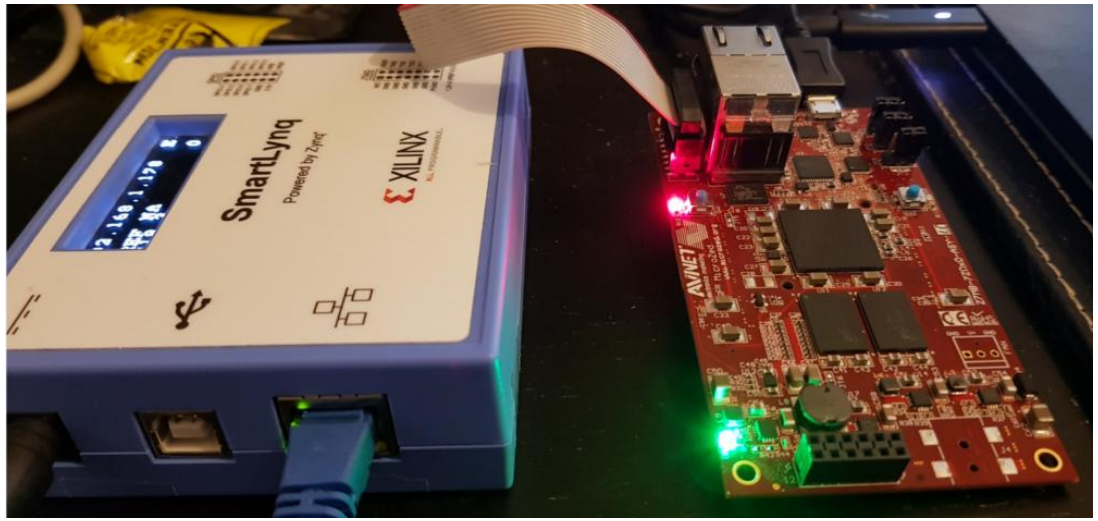
ADIUVO
ENGINEERING AND TRAINING, LTD.

SoC

Zynq, there are three MultiBoot options

- Golden Image Search—If no boot header is found at the bottom of flash memory, the BootROM will search for a valid boot header at 32KB offsets.
- BootROM MultiBoot—Works under control of the FSBL. This allows the FSBL to load an application and then a follow on application. This is useful if you wish to run a self test application before executing the main application.
- Fallback MultiBoot—If an error occurs during loading of the application, the FSBL does a fallback and enables the BootROM to load the next good image.

Use a Golden Image for fall back

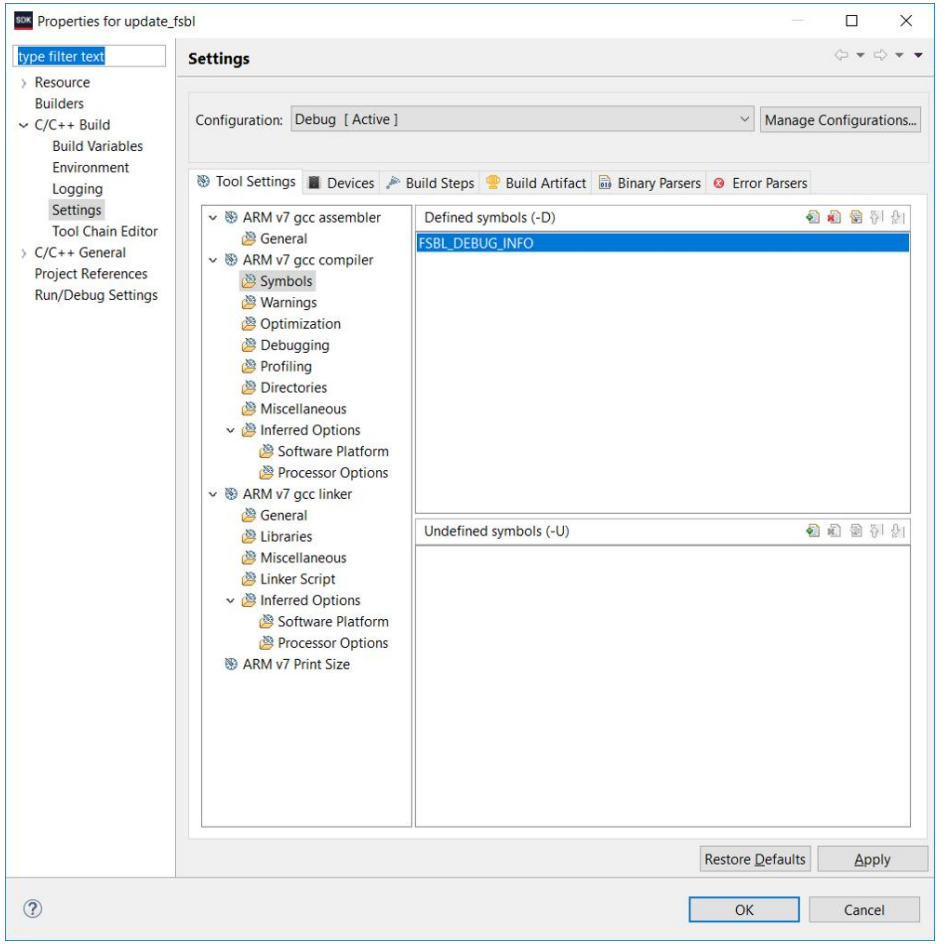


SoC

Software driven flow, need the following applications

- » Golden / Updatable Image Application
- » Golden / Updatable Image Application Board Support Package (BSP)
- » Golden / Updatable Image First Stage Boot Loader
- » Golden / Updatable Image First Stage Boot Loader BSP

- Use the SW symbol FSBL_DEBUG_INFO

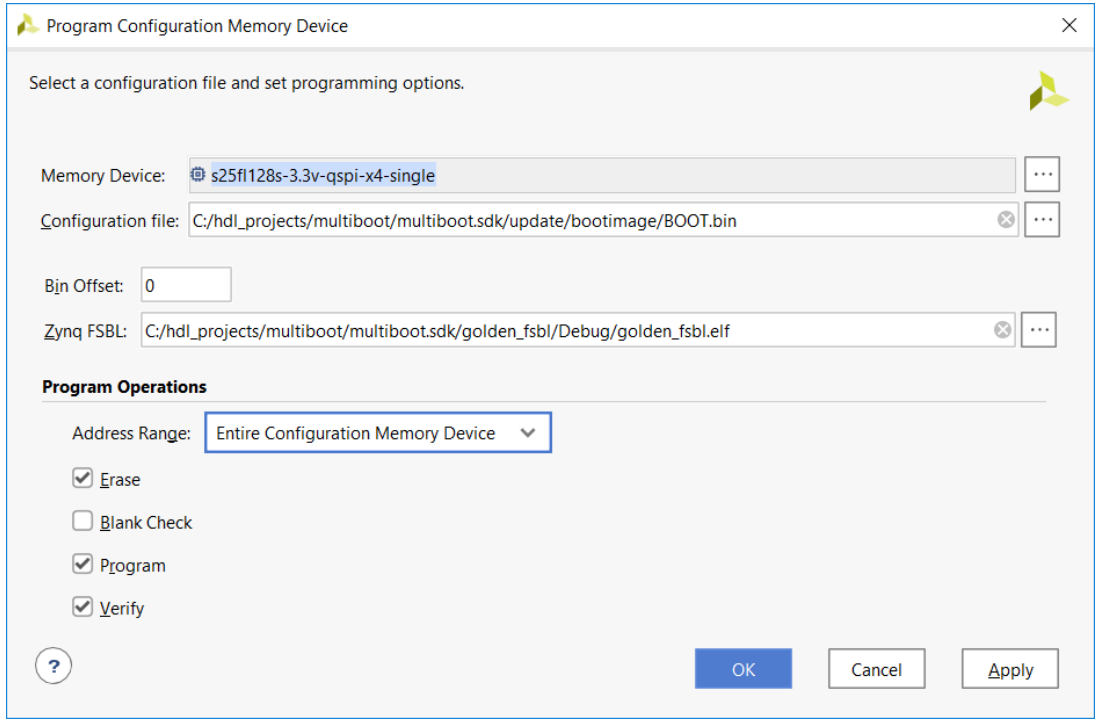


SoC

Output is two bin files, one golden and one updatable

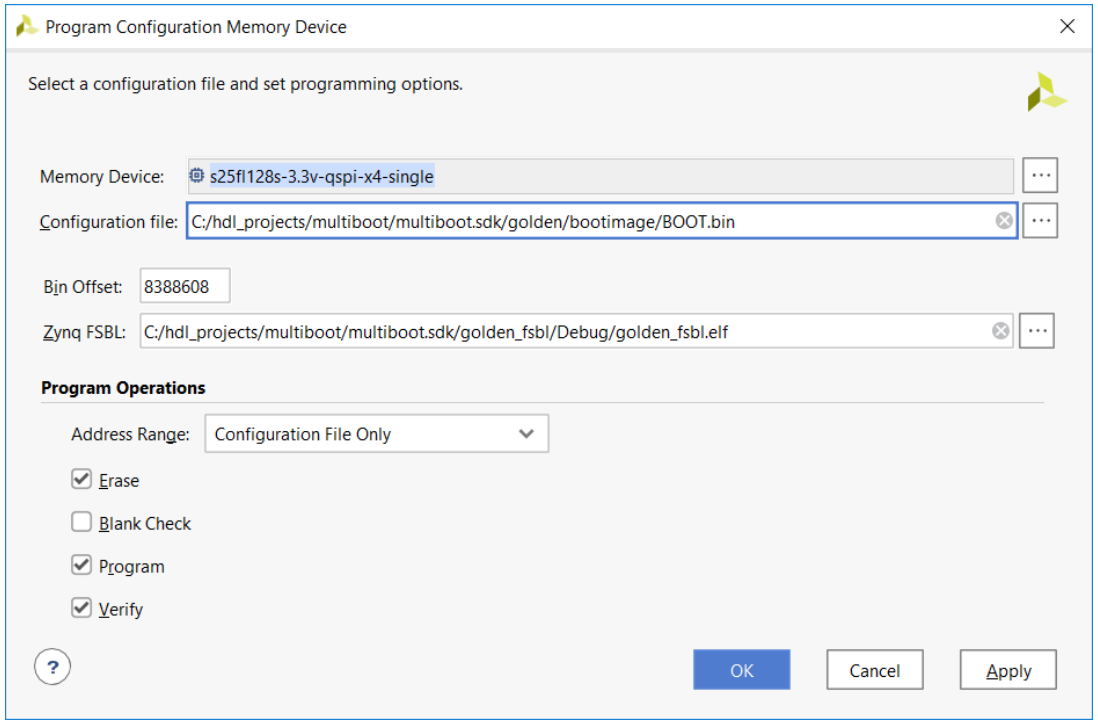
Use Vivado Hardware manager to program

For the updatable image that is written at offset 0 in the memory, we should erase the entire device and then load in the updatable boot.bin file



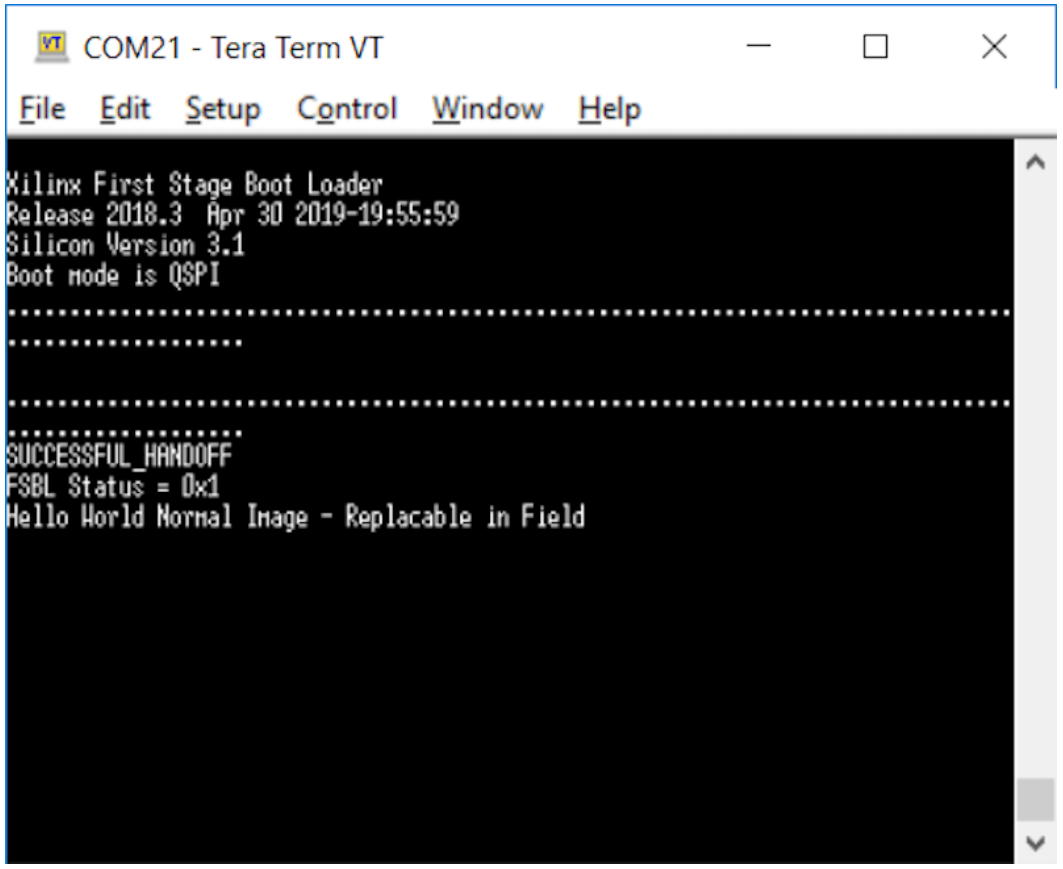
SoC

For loading in the golden image, we use the following settings. This will write in the golden image to the desired fallback location (0x80_0000).



SoC

Normal load

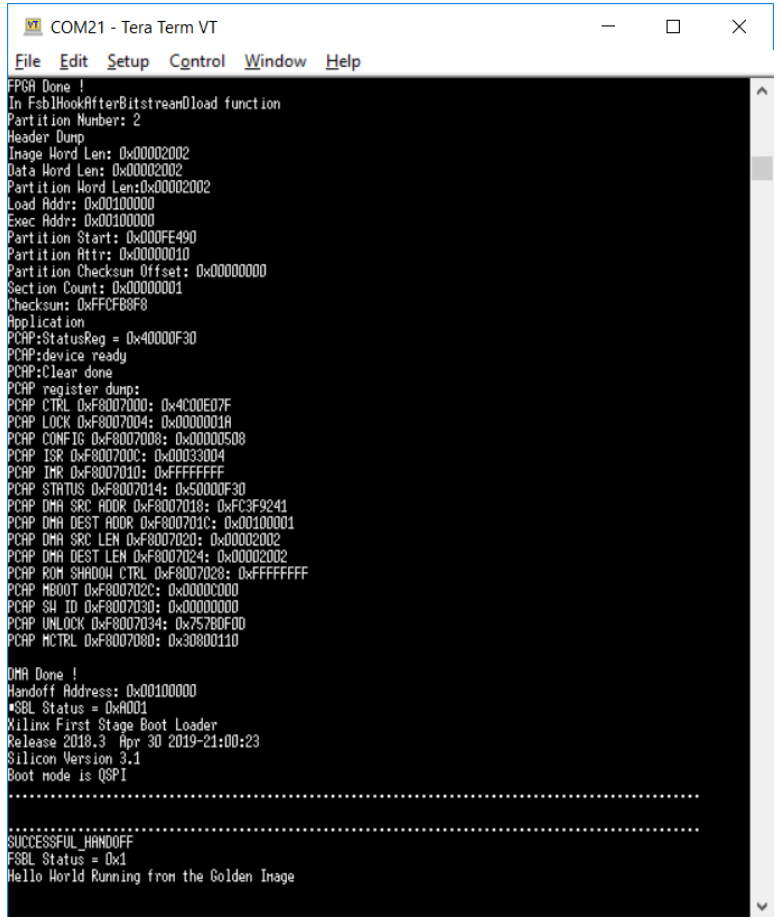


- To spoof a failure in the update image, we are going to make a modification to the updatable FSBL. Within main() in main.c, we are going to comment out the FSBL hand off and implement a fallback.

```
/*
 * FSBL handoff to valid handoff address or
 * exit in JTAG
 */
//FsblHandoff(HandoffAddress);
OutputStatus(ILLEGAL_RETURN);

FsblFallback();
```

SoC

Instead of handing off to the application, it will simulate a failure of the loading process and ensure a fallback to the golden image.



SoC Gotchas

Consider the reset type—SW2 RST on the MicroZed will perform a soft reset not power on reset so the handoff address will be already set for the fallback golden image.

When planning the location of the BIN files in the QSPI, ensure you space them sufficiently to prevent one image accidentally corrupting the other.

When planning the QSPI memory layout, leave sufficient space between the images to allow for application growth.

Ensure the fallback image is located on a 32KB address boundary.

FPGA

Creating two images—one gold image that can be fallen back to, and another which can be overwritten and updated if required in the field.

In normal operation (no MultiBot), the bitstream located at address 0x0 in the configuration memory is loaded.

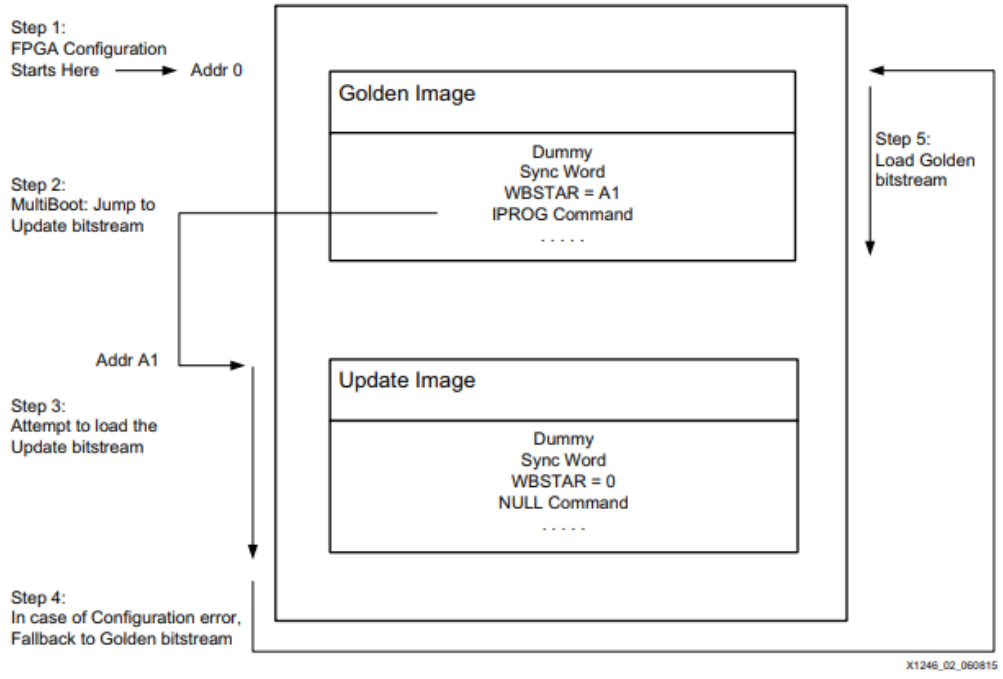
In a MultiBoot approach, there are two images in the configuration memory device. The first is located at 0x0 and is the gold image, while the normally loaded image is stored at an offset in the configuration memory.

FPGA

Crucially the gold image contains the address offset where the normal / update image resides

The address of this normal / update image is read from the gold image and stored in the gold image Warm Boot Start Address (WBSTAR)

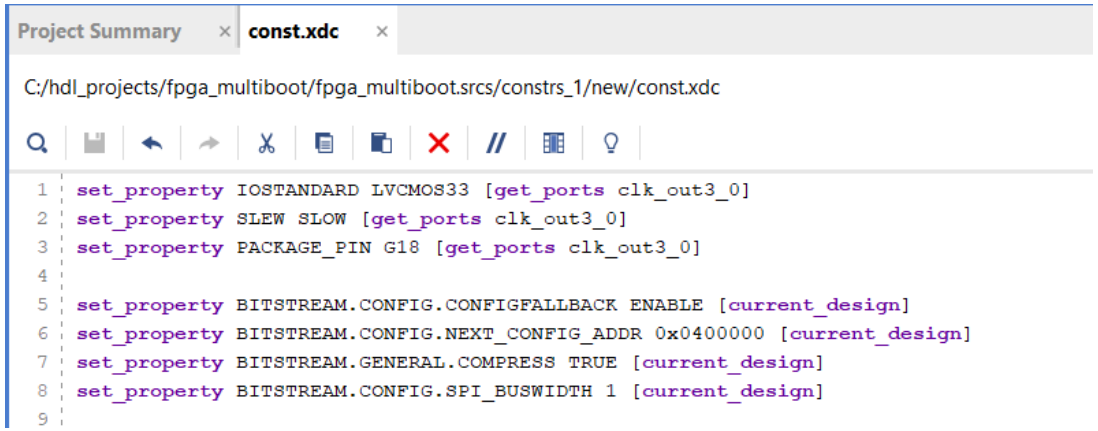
The configuration block in the FPGA can then jump to this memory address using the IPROG or internally generate pulse which initiates the jump to the specified address.



FPGA

The main difference between projects in the MultiBoot configuration is in the constraints files.

Within the constraints file for the gold image, we need to enable the fallback option. We also need to provide the location of the update image, which should be attempted to be loaded first.



FPGA

The update image, its constraint file should be as below enabling the configuration fallback but without the config address specified

Project Summary × const.xdc ×

C:/hdl_projects/fpga_multiboot/fpga_multiboot.srcs/constrs_1/new/const.xdc

```
1 set_property IOSTANDARD LVCMOS33 [get_ports clk_out3_0]
2 set_property SLEW SLOW [get_ports clk_out3_0]
3 set_property PACKAGE_PIN G18 [get_ports clk_out3_0]
4
5 set_property BITSTREAM.CONFIG.CONFIGFALLBACK ENABLE [current_design]
6 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
7 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 1 [current_design]
8
```

FPGA

Once we have both images (gold and update), we can then generate a combined MCS file which contains both images—with the gold image at address 0 and the update image at the specified offset.

The field update would require the FPGA to contain a configuration method to write a bin or hex file to the configuration memory at the update address.

write_cfgmem -format mcs -interface SPIX1 -size 16 -loadbit “up 0 golden.bit up 0x0400000 update.bit” image.mcs

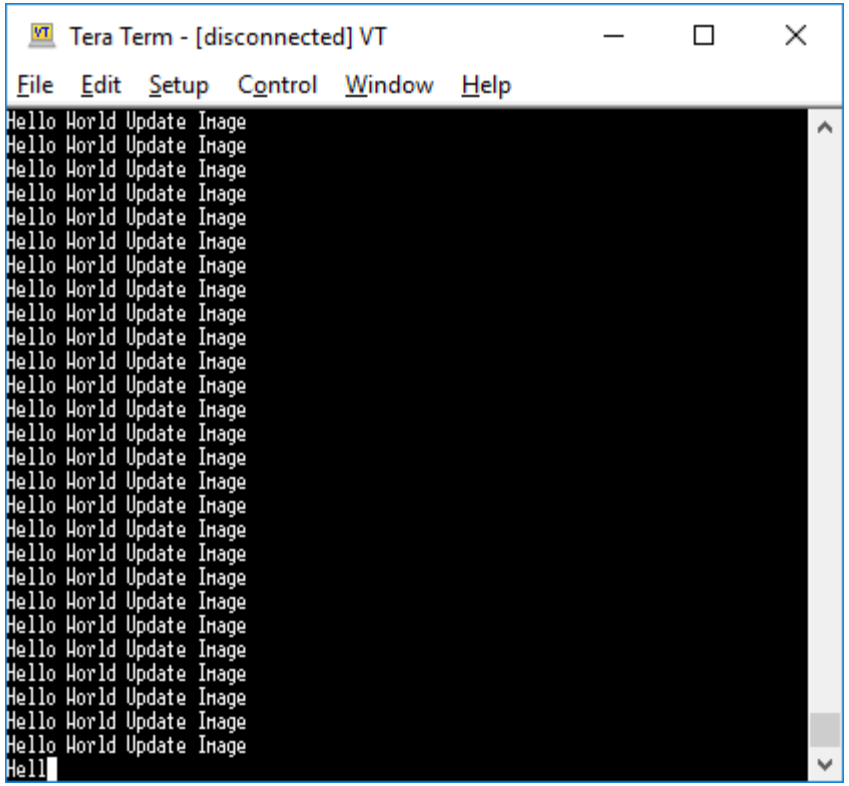
FPGA

Generated MCS Report shows the content of the generated MCS file.

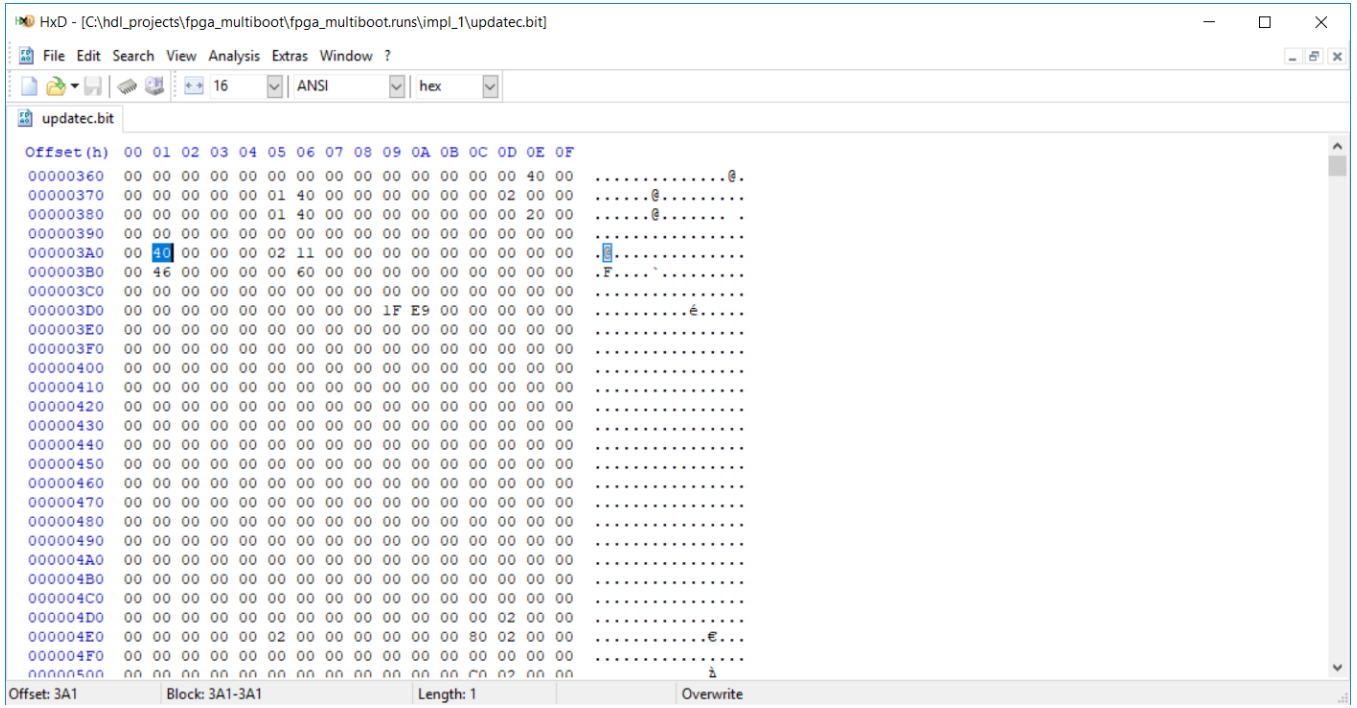
```
Command: write_cfgmem -format mcs -interface SPIX1 -size 16 -loadbit {up 0 gold.bit up 0x0400000 update.bit} image.mcs
Creating config memory files...
Creating bitstream load up from address 0x00000000
Loading bitfile gold.bit
Creating bitstream load up from address 0x00400000
Loading bitfile update.bit
Writing file ./image.mcs
Writing log file ./image.prm
==================================
Configuration Memory information
==================================
File Format        MCS
Interface          SPIX1
Size               16M
Start Address      0x00000000
End Address        0x00FFFFFF

Addr1         Addr2         Date                     File(s)
0x00000000    0x00220403    May  7 11:45:16 2019    gold.bit
0x00400000    0x00620403    May  7 11:58:09 2019    update.bit
0 Infos, 0 Warnings, 0 Critical Warnings and 0 Errors encountered.
write_cfgmem completed successfully
write_cfgmem: Time (s): cpu = 00:00:05 ; elapsed = 00:00:05 . Memory (MB): peak = 2164.539 ; gain = 17.109
```

FPGA

Update Image Loading OK

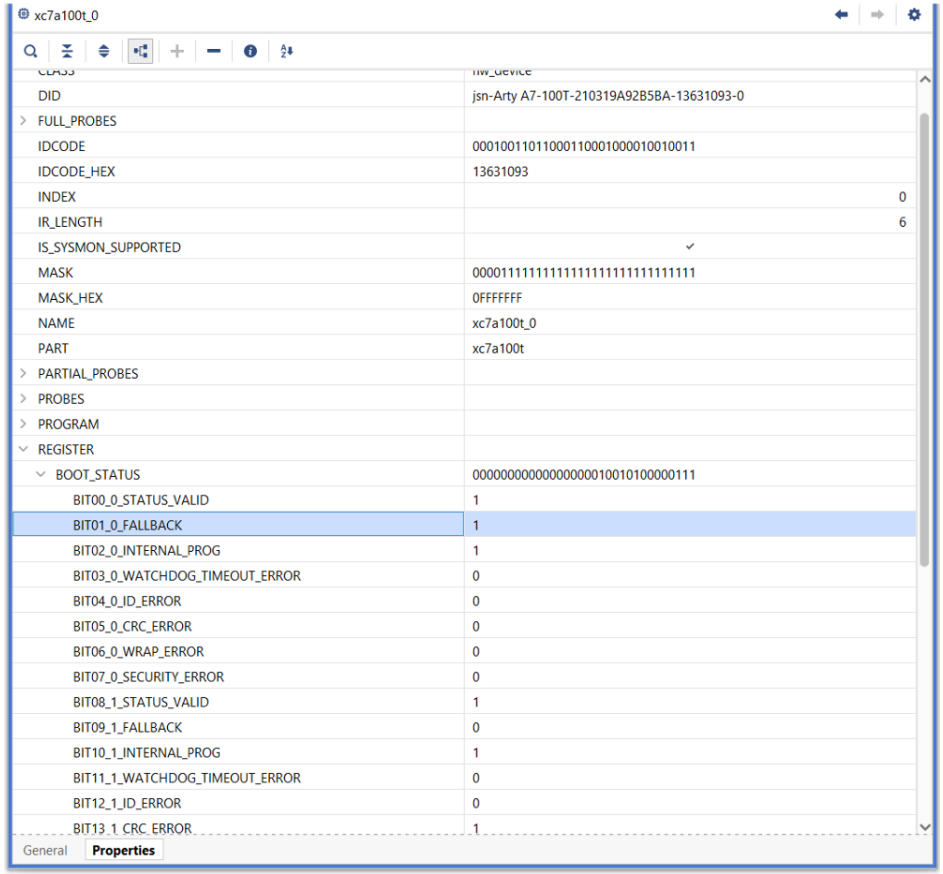


Corrupt the update and force fall back – CRC failure



FPGA

When we try and load the update image from the flash, we will see the image gold image loaded in place of the update image due to the CRC error.



References

Adiuvoengineering.com

http://microelectronics.esa.int/asic/fpga_001_01-0-2.pdf

http://microelectronics.esa.int/asic/ECSS-Q-HB-60-02A1September2016.pdf

http://microelectronics.esa.int/asic/index.html

https://www.xilinx.com/support/documentation/white_papers/wp286.pdf

https://www.xilinx.com/support/documentation/sw_manuals/xilinx2016_1/ug974-vivado-ultrascale-libraries.pdf

https://china.xilinx.com/support/documentation/sw_manuals/xilinx2018_2/ug953-vivado-7series-libraries.pdf

https://www.xilinx.com/support/documentation/ip_documentation/tmr/v1_0/pg268-tmr.pdf

https://www.xilinx.com/support/documentation/ip_documentation/sem/v4_1/pg036_sem.pdf

https://www.xilinx.com/support/documentation/user_guides/ug116.pdf

https://www.xilinx.com/support/documentation/application_notes/xapp1222-idf-for-7s-or-zynq-vivado.pdf

ADIUVO
ENGINEERING AND TRAINING, LTD.

References

https://github.com/ATaylorCEngFIET

ADIUVO

ENGINEERING AND TRAINING, LTD.

www.adiuvoengineering.com

adam@adiuvoengineering.com